```
ThisWorkbook - 1

' Last Cleaned (Export): 20-03-2023 11:34
Option Explicit
' ============================================================================

' ----------------------------------------------------------------------------
' Author:  Ronald Bax           Version: 4.00           Date: 20-03-2023
' Action:  ...
' Uses:    ...
' Param:   ...
' History: 23-01-2019 Ronald Bax     First version
' ----------------------------------------------------------------------------
Private Sub Workbook_Open()
   Dim sOrgDir As String
   Call QAGeneric.SetGlobals(sDescription:="Payroll DBS")
   Call EU_ALL_Lib.Generic_Workbook_Open(aWks:=wksStores, bWantBackup:=True, sSelCell:="G2")
   sOrgDir = Mid(wksMultiMW.Range("A3").Formula, 3, InStr(1, wksMultiMW.Range("A3").Formula, "[") - 4)
   If sOrgDir <> ThisWorkbook.Path Then
      With wksStores
         .Unprotect
         .Range("G4").Value2 = ThisWorkbook.FullName
      End With
      With wksMultiMW
         .Unprotect
         .Cells.Replace What:=sOrgDir, Replacement:=ThisWorkbook.Path, LookAt:=xlPart, SearchOrder:=xl
ByRows, MatchCase:=False, SearchFormat:=False, ReplaceFormat:=False, FormulaVersion:=xlReplaceFormula2
      End With
      Call EU_XLS_Lib.ProtectAllWorksheets(bProtect:=True)
   End If
   ' Perform other actions, specific for this Workbook ...
   wksStores.Unprotect
   wksStores.Range("G2").Value2 = Date
   Call EU_XLS_Lib.AddActCircleToBtn(aWks:=wksStores, aBtn:=wksStores.Shapes("btnPing"), bDelOld:=True
)
   wksStores.Range("$I$6").Value2 = False
   Call EU_XLS_Lib.ProtectSheet(aWks:=wksStores)
End Sub
' ============================================================================


' ============================================================================
' ============================================================================
```

wksMultiMW - 1

```
' Last Cleaned (Export): 20-03-2023 11:34
Option Explicit
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 4.00           Date: 20-03-2023
' Action:  ...
' Uses:    ...
' Param:   ...
' History: 23-01-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Public Sub ReplaceMultiInResult()
   Dim oCell   As Range
   Dim aRng    As Range
   Dim sSearch As String
   Dim iLR     As Long
   iLR = EU_XLS_Lib.LastRow(aWks:=wksResult)
   With wksResult
      .Unprotect
      .Activate
      For Each oCell In .Range("A2:A" & iLR)
         If oCell.Value <> vbNullString Then
            oCell.Activate
            sSearch = oCell.Value2 & "-" & oCell.Offset(0, 2).Value2
            With wksMultiMW
               .Activate
               .Range("E2").Activate
               Set aRng = .Range("E:E").Find(What:=sSearch, After:=ActiveCell, LookIn:=xlValues, LookA
t:=xlWhole, SearchOrder:=xlByColumns, SearchDirection:=xlNext, MatchCase:=False, SearchFormat:=False)
            End With
            .Activate
            If Not aRng Is Nothing Then
               .Range("M" & oCell.Row) = "Multi was (" & oCell.Value2 & ")"
               With oCell
                  .Value2 = wksMultiMW.Range("F" & aRng.Row).Value2
                  With .Interior
                     .Pattern = xlSolid
                     .PatternColorIndex = xlAutomatic
                     .ThemeColor = xlThemeColorAccent2
                     .TintAndShade = 0.599993896298105
                     .PatternTintAndShade = 0
                  End With
               End With
            End If
         End If
      Next oCell
   End With
   Set aRng = Nothing
   Set oCell = Nothing
End Sub

' ====================================================================


' ====================================================================
' ====================================================================
```

```
wksResult - 1

' Last Cleaned (Export): 20-03-2023 11:34
Option Explicit
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 4.00           Date: 20-03-2023
' Action:  ...
' Uses:    n.a.
' Param:   n.a.
' History: 23-01-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Public Sub GetAndProcessData()
   Const sCodeKostenpost = "U2101"
   Dim wks        As Worksheet
   Dim sDate      As String
   Dim sSQL       As String
   Dim sStoreIP   As String
   Dim iRow       As Long
   Dim iLastRow   As Long
   Dim iStLastRow As Long
   Dim bVoid      As Boolean
   Dim sFileName  As String
   Dim sSep       As String
   Dim sOneDay    As String
   Dim S          As String
   Dim sThisWB    As String
   sThisWB = ThisWorkbook.Name

   sDate = Format(wksStores.Range("G2").Value2, "yyyy-mm-dd", vbMonday)
   sSep = wksStores.Range("I5").Value2
   sOneDay = IIf(wksStores.Range("I6").Value2, "1", "0")

   sSQL = "EXEC DPEU.dbo.spDOM_ExtractPayrollNmbrsNL @ForDate='" & sDate & "', @Header=0, @JustOneDay=
" & sOneDay
   wksResult.Activate
   iStLastRow = EU_XLS_Lib.LastRow(aWks:=wksStores)
   iLastRow = 2
   For iRow = 2 To iStLastRow
      If wksStores.Range("A" & iRow).Value2 = "V" And wksStores.Range("B" & iRow).Value2 <> vbNullStri
ng Then
         wksStores.Range("E" & iRow).Value2 = vbNullString
         sStoreIP = EU_DOM_Lib.GetStoreToDNSName(sStoreNum:=wksStores.Range("B" & iRow).Value2)
         On Error Resume Next
         bVoid = EU_DB_Lib.SQLToWorksheet(aWks:=wksResult, aRng:=wksResult.Range("A" & iLastRow), sSto
reIP:=sStoreIP, sSQL:=sSQL, bHeaders:=False, bShowMsg:=False)
         On Error GoTo 0
         If bVoid Then
            wksStores.Range("E" & iRow).Value2 = "V"
         Else
            wksStores.Range("E" & iRow).Value2 = "X"
         End If
         iLastRow = EU_XLS_Lib.LastRow(aWks:=wksResult) + 1
      End If
   Next iRow

   'Replace Debt-Number for Multi-store employees
   Call wksMultiMW.ReplaceMultiInResult

   If iLastRow > 2 Then
      wksResult.Range("L2:L" & iLastRow - 1).FormulaR1C1 = "=IF(IFERROR(VLOOKUP(RC[-9]&LEFT(RC[-3],5),
Managers!C3:C4,2,FALSE),0)=0,IF(IFERROR(VLOOKUP(RC[-9],Managers!C3:C4,2,FALSE),0)=0,"""",VLOOKUP(RC[-9
],Managers!C3:C4,2,FALSE)),VLOOKUP(RC[-9]&LEFT(RC[-3],5),Managers!C3:C4,2,FALSE))"
   End If
   Call EU_XLS_Lib.AutoFitWorksheet(aWks:=wksResult)

   With wksResult
      ' ==================
      ' Save data and clean-up Temp
      ' ==================
      wksResult.Activate
      Application.DisplayAlerts = False
      On Error Resume Next
      sFileName = ActiveWorkbook.Path & "\HR-payroll.csv"
```

```
wksResult - 2

      Call EU_XLS_Lib.SaveSheetToTXTFile(aWks:=wksResult, sFilePath:=sFileName, sSepChar:=sSep)
      Application.DisplayAlerts = True
      On Error GoTo 0
      Application.Calculation = xlCalculationAutomatic
      ThisWorkbook.RefreshAll
      ' ====================
      ' Cleanup The result-set
      ' ====================
      iLastRow = EU_XLS_Lib.LastRow(aWks:=wksResult) + 1
      For iRow = iLastRow To 3 Step -1
         S = wksResult.Range("C" & iRow).Value2
         If (Len(S) = 5 And Left(S, 3) = "099") Or (Len(S) = 4 And Left(S, 2) = "99") Then
            wksResult.Rows(iRow).Delete
         End If
      Next iRow
   End With

' ====================
' Delete work sheets (if they exists)
' ====================
   Application.DisplayAlerts = False
   If Evaluate("ISREF('LooncomponentenVar'!A1)") Then Worksheets("LooncomponentenVar").Delete
   If Evaluate("ISREF('KostenverdelingLooncomponenten'!A1)") Then Worksheets("KostenverdelingLooncompo
nenten").Delete
   If Evaluate("ISREF('Temp'!A1)") Then Worksheets("Temp").Delete
   Application.DisplayAlerts = True

' ====================
' Copy Data to Nmbrs-format-output
' ====================
   wksResult.Unprotect
   Call EU_XLS_Lib.AutoFitWorksheet(aWks:=wksResult)
   wksResult.Copy After:=wksResult
   Sheets("Result (2)").Select
   ActiveSheet.Name = "Temp"

' ====================
' Cleanup Temp data - Remove the managers
' ====================
   With Sheets("Temp")
      .Activate
      iLastRow = EU_XLS_Lib.LastRow(aWks:=Sheets("Temp")) + 1
      For iRow = iLastRow To 2 Step -1
         If .Cells(iRow, 12).Value2 <> vbNullString Then .Rows(iRow).Delete
      Next iRow
   End With

' ====================
' Copy Temp to Nmbrs
' ====================
   Worksheets("Temp").Copy After:=Worksheets("Temp")
   Sheets("Temp (2)").Select
   ActiveSheet.Name = "LooncomponentenVar"
   Worksheets("Temp").Copy After:=Worksheets("LooncomponentenVar")
   Sheets("Temp (2)").Select
   ActiveSheet.Name = "KostenverdelingLooncomponenten"
   Call EU_XLS_Lib.ProtectSheet(aWks:=wksResult)

' ====================
' Cleanup Nmbrs data
' ====================
   With Sheets("LooncomponentenVar")
      .Activate
      .Range("A1").Select
      .Range("I:M").EntireColumn.Delete
      .Range("A1").Select
      .Range("A1").EntireRow.Insert
      .Range("A1").Value2 = "Import"
      .Range("E1").Value2 = "Looncomponenten"
      iLastRow = EU_XLS_Lib.LastRow(aWks:=Sheets("LooncomponentenVar")) + 1
      For iRow = iLastRow To 3 Step -1
         .Cells(iRow, 8).Value2 = vbNullString
      Next iRow
```

```
wksResult - 3

      Call EU_XLS_Lib.AutoFitWorksheet(aWks:=Sheets("LooncomponentenVar"))
   End With

   With Sheets("KostenverdelingLooncomponenten")
      .Activate
      .Range("A1").Select
      .Range("K:N").EntireColumn.Delete
      .Range("A1").Select
      .Range("A1").EntireRow.Insert
      .Range("A1").Value2 = "Import"
      .Range("E1").Value2 = "Kostenplaats"
      .Range("E2").Value2 = "PeriodeStart"
      .Range("F2").Value2 = "PeriodeEind"
      .Range("G2").Value2 = "Code"
      .Range("H2").Value2 = "Waarde"
      .Range("I2").Value2 = "Kostenplaats"
      .Range("J2").Value2 = "Kostensoort"
      iLastRow = EU_XLS_Lib.LastRow(aWks:=Sheets("KostenverdelingLooncomponenten")) + 1
      For iRow = iLastRow To 3 Step -1
         If .Range("A" & iRow).Value2 = vbNullString Then
            .Range("A" & iRow).EntireRow.Delete
         Else
            .Range("F" & iRow).Value2 = .Range("E" & iRow).Value2
            .Range("G" & iRow).Value2 = sCodeKostenpost
            .Range("H" & iRow).Value2 = Round(.Range("H" & iRow).Value2, 2)
            .Range("H" & iRow).NumberFormat = "#.00_ ;-#.00"
            .Range("J" & iRow).Value2 = Mid(.Range("I" & iRow).Value2, 1, 5)
            .Range("I" & iRow).Value2 = .Range("J" & iRow).Value2
            .Range("J" & iRow).Value2 = vbNullString
         End If
      Next iRow
      Call EU_XLS_Lib.AutoFitWorksheet(aWks:=Sheets("KostenverdelingLooncomponenten"))
   End With

' ====================
' Save data and clean-up Temp
' ====================
   Sheets("LooncomponentenVar").Activate
   Application.DisplayAlerts = False
   On Error Resume Next
   sFileName = ActiveWorkbook.Path & "\LooncomponentenVar.xlsx"
   sFileName = Replace(sFileName, ".xlsx", wksStores.Range("ExportExtention").Value2)
   ActiveSheet.Copy
   Application.ActiveWorkbook.SaveAs sFileName, ConflictResolution:=xlUserResolution, FileFormat:=51 '
 xlsx
   Application.DisplayAlerts = True
   On Error GoTo 0

   Application.Workbooks(sThisWB).Activate
   wksStores.Activate

   Sheets("KostenverdelingLooncomponenten").Activate
   Application.DisplayAlerts = False
   On Error Resume Next
   sFileName = ActiveWorkbook.Path & "\KostenverdelingLooncomponenten.xlsx"
   sFileName = Replace(sFileName, ".xlsx", wksStores.Range("ExportExtention").Value2)
   ActiveSheet.Copy
   Application.ActiveWorkbook.SaveAs sFileName, ConflictResolution:=xlUserResolution, FileFormat:=51 '
 xlsx
   Application.DisplayAlerts = True
   On Error GoTo 0

   Application.Workbooks(sThisWB).Activate
   wksStores.Activate

   sFileName = "LooncomponentenVar.xlsx"
   sFileName = Replace(sFileName, ".xlsx", wksStores.Range("ExportExtention").Value2)
   Application.Workbooks(sFileName).Close
   sFileName = "KostenverdelingLooncomponenten.xlsx"
   sFileName = Replace(sFileName, ".xlsx", wksStores.Range("ExportExtention").Value2)
   Application.Workbooks(sFileName).Close

   Application.DisplayAlerts = False
```

```
wksResult - 4

   If Evaluate("ISREF('Temp'!A1)") Then Worksheets("Temp").Delete
   Application.DisplayAlerts = True

' ===================
' Update pivot with doubles
' ===================
   With wksTelling
      .Activate
      .Unprotect
      Call EU_XLS_Lib.ClearSheet(aWks:=wksTelling, sCell:="E1", sRowCol:="col")
      .PivotTables("pivTelling").PivotCache.Refresh
      .Range("D3").Copy
      .Range("E3").PasteSpecial Paste:=xlPasteFormats, Operation:=xlNone, SkipBlanks:=False, Transpose
:=False
      Application.CutCopyMode = False
      .Range("E3").Value2 = "<="
      .Columns("E:E").Select
      With Selection
         .HorizontalAlignment = xlCenter
         .VerticalAlignment = xlCenter
         .WrapText = False
         .Orientation = 0
         .AddIndent = False
         .IndentLevel = 0
         .ShrinkToFit = False
         .ReadingOrder = xlContext
         .MergeCells = False
      End With
      .Range("E3").Select
      Selection.AutoFilter
      iLastRow = EU_XLS_Lib.LastRow(aWks:=wksTelling)
      .Range("E4:E" & iLastRow).FormulaR1C1 = "=IF(AND(RC[-2]<>"""",OR(RC[-4]=R[-1]C[-4],RC[-4]=R[1]C[
-4])),""<="","""")"
   End With

' ===================
' Return to Stores-sheet
' ===================
   With wksStores
      .Activate
      .Range("G2").Select
      If .Range("G3").Value2 <> .Range("I3").Value2 Then
         MsgBox "Not all stores imported.  Stores: " & CStr(.Range("G3").Value2) & ".  Imported: " & C
Str(.Range("I3").Value2) & ".", vbOKOnly + vbInformation
      Else
         MsgBox "Imported: " & CStr(.Range("I3").Value2) & " stores.", vbOKOnly + vbInformation
      End If
   End With

   Application.Calculation = xlCalculationAutomatic
   ThisWorkbook.RefreshAll
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 4.00           Date: 20-03-2023
' Action:  ...
' Uses:    n.a.
' Param:   n.a.
' History: 23-01-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Public Sub ActivateResultPage()
   wksStores.Activate
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 4.00           Date: 20-03-2023
' Action:  ...
' Uses:    ...
' Param:   ...
' History: 23-01-2019 Ronald Bax     First version
' --------------------------------------------------------------------
```

```
wksResult - 5

Public Sub ckbOneDay_Click()
   If Date <> wksStores.Range("G2").Value2 Or Not wksStores.Range("$I$6").Value2 Then Exit Sub
   If wksStores.Range("$I$6").Value2 Then
      MsgBox "Can't use JustOneDay for TODAY." & vbCrLf & "Change the date first.", vbOKOnly + vbCriti
cal, "wksStores.ckbOneDay_Click"
   End If
   wksStores.Activate
   wksStores.Range("G2").Select
   wksStores.Range("$I$6").Value2 = False
End Sub
' ================================================================================

' ================================================================================
' ================================================================================
```

```
wksResult2 - 1

' Last Cleaned (Export): 20-03-2023 11:34
Option Explicit
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 4.00           Date: 20-03-2023
' Action:  ...
' Uses:    n.a.
' Param:   n.a.
' History: 23-01-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Public Sub GetAndProcessData()
   Const sCodeKostenpost = "U2101"
   Dim wks        As Worksheet
   Dim sDate      As String
   Dim sSQL       As String
   Dim sStoreIP   As String
   Dim iRow       As Long
   Dim iLastRow   As Long
   Dim iStLastRow As Long
   Dim bVoid      As Boolean
   Dim sFileName  As String
   Dim sSep       As String
   Dim sOneDay    As String
   Dim S          As String
   Dim sThisWB    As String
   sThisWB = ThisWorkbook.Name

   sDate = Format(wksStores.Range("G2").Value2, "yyyy-mm-dd", vbMonday)
   sSep = wksStores.Range("I5").Value2
   sOneDay = IIf(wksStores.Range("I6").Value2, "1", "0")

   sSQL = "EXEC DPEU.dbo.spDOM_ExtractPayrollNmbrsNL @ForDate='" & sDate & "', @Header=0, @JustOneDay=
" & sOneDay
   wksResult.Activate
   iStLastRow = EU_XLS_Lib.LastRow(aWks:=wksStores)
   iLastRow = 2
   For iRow = 2 To iStLastRow
      If wksStores.Range("A" & iRow).Value2 = "V" And wksStores.Range("B" & iRow).Value2 <> vbNullStri
ng Then
         wksStores.Range("E" & iRow).Value2 = vbNullString
         sStoreIP = EU_DOM_Lib.GetStoreToDNSName(sStoreNum:=wksStores.Range("B" & iRow).Value2)
         On Error Resume Next
         bVoid = EU_DB_Lib.SQLToWorksheet(aWks:=wksResult, aRng:=wksResult.Range("A" & iLastRow), sSto
reIP:=sStoreIP, sSQL:=sSQL, bHeaders:=False, bShowMsg:=False)
         On Error GoTo 0
         If bVoid Then
            wksStores.Range("E" & iRow).Value2 = "V"
         Else
            wksStores.Range("E" & iRow).Value2 = "X"
         End If
         iLastRow = EU_XLS_Lib.LastRow(aWks:=wksResult) + 1
      End If
   Next iRow

   'Replace Debt-Number for Multi-store employees
   Call wksMultiMW.ReplaceMultiInResult

   If iLastRow > 2 Then
      wksResult.Range("L2:L" & iLastRow - 1).FormulaR1C1 = "=IF(IFERROR(VLOOKUP(RC[-9]&LEFT(RC[-3],5),
Managers!C3:C4,2,FALSE),0)=0,IF(IFERROR(VLOOKUP(RC[-9],Managers!C3:C4,2,FALSE),0)=0,"""",VLOOKUP(RC[-9
],Managers!C3:C4,2,FALSE)),VLOOKUP(RC[-9]&LEFT(RC[-3],5),Managers!C3:C4,2,FALSE))"
   End If
   Call EU_XLS_Lib.AutoFitWorksheet(aWks:=wksResult)

   With wksResult
      ' ==================
      ' Save data and clean-up Temp
      ' ==================
      wksResult.Activate
      Application.DisplayAlerts = False
      On Error Resume Next
      sFileName = ActiveWorkbook.Path & "\HR-payroll.csv"
```

```
wksResult2 - 2

      Call EU_XLS_Lib.SaveSheetToTXTFile(aWks:=wksResult, sFilePath:=sFileName, sSepChar:=sSep)
      Application.DisplayAlerts = True
      On Error GoTo 0
      Application.Calculation = xlCalculationAutomatic
      ThisWorkbook.RefreshAll
      ' ====================
      ' Cleanup The result-set
      ' ====================
      iLastRow = EU_XLS_Lib.LastRow(aWks:=wksResult) + 1
      For iRow = iLastRow To 3 Step -1
         S = wksResult.Range("C" & iRow).Value2
         If (Len(S) = 5 And Left(S, 3) = "099") Or (Len(S) = 4 And Left(S, 2) = "99") Then
            wksResult.Rows(iRow).Delete
         End If
      Next iRow
   End With

' ====================
' Delete work sheets (if they exists)
' ====================
   Application.DisplayAlerts = False
   If Evaluate("ISREF('LooncomponentenVar'!A1)") Then Worksheets("LooncomponentenVar").Delete
   If Evaluate("ISREF('KostenverdelingLooncomponenten'!A1)") Then Worksheets("KostenverdelingLooncompo
nenten").Delete
   If Evaluate("ISREF('Temp'!A1)") Then Worksheets("Temp").Delete
   Application.DisplayAlerts = True

' ====================
' Copy Data to Nmbrs-format-output
' ====================
   wksResult.Unprotect
   Call EU_XLS_Lib.AutoFitWorksheet(aWks:=wksResult)
   wksResult.Copy After:=wksResult
   Sheets("Result (2)").Select
   ActiveSheet.Name = "Temp"

' ====================
' Cleanup Temp data - Remove the managers
' ====================
   With Sheets("Temp")
      .Activate
      iLastRow = EU_XLS_Lib.LastRow(aWks:=Sheets("Temp")) + 1
      For iRow = iLastRow To 2 Step -1
         If .Cells(iRow, 12).Value2 <> vbNullString Then .Rows(iRow).Delete
      Next iRow
   End With

' ====================
' Copy Temp to Nmbrs
' ====================
   Worksheets("Temp").Copy After:=Worksheets("Temp")
   Sheets("Temp (2)").Select
   ActiveSheet.Name = "LooncomponentenVar"
   Worksheets("Temp").Copy After:=Worksheets("LooncomponentenVar")
   Sheets("Temp (2)").Select
   ActiveSheet.Name = "KostenverdelingLooncomponenten"
   Call EU_XLS_Lib.ProtectSheet(aWks:=wksResult)

' ====================
' Cleanup Nmbrs data
' ====================
   With Sheets("LooncomponentenVar")
      .Activate
      .Range("A1").Select
      .Range("I:M").EntireColumn.Delete
      .Range("A1").Select
      .Range("A1").EntireRow.Insert
      .Range("A1").Value2 = "Import"
      .Range("E1").Value2 = "Looncomponenten"
      iLastRow = EU_XLS_Lib.LastRow(aWks:=Sheets("LooncomponentenVar")) + 1
      For iRow = iLastRow To 3 Step -1
         .Cells(iRow, 8).Value2 = vbNullString
      Next iRow
```

```
wksResult2 - 3

      Call EU_XLS_Lib.AutoFitWorksheet(aWks:=Sheets("LooncomponentenVar"))
   End With

   With Sheets("KostenverdelingLooncomponenten")
      .Activate
      .Range("A1").Select
      .Range("K:N").EntireColumn.Delete
      .Range("A1").Select
      .Range("A1").EntireRow.Insert
      .Range("A1").Value2 = "Import"
      .Range("E1").Value2 = "Kostenplaats"
      .Range("E2").Value2 = "PeriodeStart"
      .Range("F2").Value2 = "PeriodeEind"
      .Range("G2").Value2 = "Code"
      .Range("H2").Value2 = "Waarde"
      .Range("I2").Value2 = "Kostenplaats"
      .Range("J2").Value2 = "Kostensoort"
      iLastRow = EU_XLS_Lib.LastRow(aWks:=Sheets("KostenverdelingLooncomponenten")) + 1
      For iRow = iLastRow To 3 Step -1
         If .Range("A" & iRow).Value2 = vbNullString Then
            .Range("A" & iRow).EntireRow.Delete
         Else
            .Range("F" & iRow).Value2 = .Range("E" & iRow).Value2
            .Range("G" & iRow).Value2 = sCodeKostenpost
            .Range("H" & iRow).Value2 = Round(.Range("H" & iRow).Value2, 2)
            .Range("H" & iRow).NumberFormat = "#.00_ ;-#.00"
            .Range("J" & iRow).Value2 = Mid(.Range("I" & iRow).Value2, 1, 5)
            .Range("I" & iRow).Value2 = .Range("J" & iRow).Value2
            .Range("J" & iRow).Value2 = vbNullString
         End If
      Next iRow
      Call EU_XLS_Lib.AutoFitWorksheet(aWks:=Sheets("KostenverdelingLooncomponenten"))
   End With

' ====================
' Save data and clean-up Temp
' ====================
   Sheets("LooncomponentenVar").Activate
   Application.DisplayAlerts = False
   On Error Resume Next
   sFileName = ActiveWorkbook.Path & "\LooncomponentenVar.xlsx"
   sFileName = Replace(sFileName, ".xlsx", wksStores.Range("ExportExtention").Value2)
   ActiveSheet.Copy
   Application.ActiveWorkbook.SaveAs sFileName, ConflictResolution:=xlUserResolution, FileFormat:=51 '
 xlsx
   Application.DisplayAlerts = True
   On Error GoTo 0

   Application.Workbooks(sThisWB).Activate
   wksStores.Activate

   Sheets("KostenverdelingLooncomponenten").Activate
   Application.DisplayAlerts = False
   On Error Resume Next
   sFileName = ActiveWorkbook.Path & "\KostenverdelingLooncomponenten.xlsx"
   sFileName = Replace(sFileName, ".xlsx", wksStores.Range("ExportExtention").Value2)
   ActiveSheet.Copy
   Application.ActiveWorkbook.SaveAs sFileName, ConflictResolution:=xlUserResolution, FileFormat:=51 '
 xlsx
   Application.DisplayAlerts = True
   On Error GoTo 0

   Application.Workbooks(sThisWB).Activate
   wksStores.Activate

   sFileName = "LooncomponentenVar.xlsx"
   sFileName = Replace(sFileName, ".xlsx", wksStores.Range("ExportExtention").Value2)
   Application.Workbooks(sFileName).Close
   sFileName = "KostenverdelingLooncomponenten.xlsx"
   sFileName = Replace(sFileName, ".xlsx", wksStores.Range("ExportExtention").Value2)
   Application.Workbooks(sFileName).Close

   Application.DisplayAlerts = False
```

```
wksResult2 - 4

   If Evaluate("ISREF('Temp'!A1)") Then Worksheets("Temp").Delete
   Application.DisplayAlerts = True

' ===================
' Update pivot with doubles
' ===================
   With wksTelling
      .Activate
      .Unprotect
      Call EU_XLS_Lib.ClearSheet(aWks:=wksTelling, sCell:="E1", sRowCol:="col")
      .PivotTables("pivTelling").PivotCache.Refresh
      .Range("D3").Copy
      .Range("E3").PasteSpecial Paste:=xlPasteFormats, Operation:=xlNone, SkipBlanks:=False, Transpose
:=False
      Application.CutCopyMode = False
      .Range("E3").Value2 = "<="
      .Columns("E:E").Select
      With Selection
         .HorizontalAlignment = xlCenter
         .VerticalAlignment = xlCenter
         .WrapText = False
         .Orientation = 0
         .AddIndent = False
         .IndentLevel = 0
         .ShrinkToFit = False
         .ReadingOrder = xlContext
         .MergeCells = False
      End With
      .Range("E3").Select
      Selection.AutoFilter
      iLastRow = EU_XLS_Lib.LastRow(aWks:=wksTelling)
      .Range("E4:E" & iLastRow).FormulaR1C1 = "=IF(AND(RC[-2]<>"""",OR(RC[-4]=R[-1]C[-4],RC[-4]=R[1]C[
-4])),""<="","""")"
   End With

' ===================
' Return to Stores-sheet
' ===================
   With wksStores
      .Activate
      .Range("G2").Select
      If .Range("G3").Value2 <> .Range("I3").Value2 Then
         MsgBox "Not all stores imported.  Stores: " & CStr(.Range("G3").Value2) & ".  Imported: " & C
Str(.Range("I3").Value2) & ".", vbOKOnly + vbInformation
      Else
         MsgBox "Imported: " & CStr(.Range("I3").Value2) & " stores.", vbOKOnly + vbInformation
      End If
   End With

   Application.Calculation = xlCalculationAutomatic
   ThisWorkbook.RefreshAll
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 4.00           Date: 20-03-2023
' Action:  ...
' Uses:    n.a.
' Param:   n.a.
' History: 23-01-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Public Sub ActivateResultPage()
   wksStores.Activate
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 4.00           Date: 20-03-2023
' Action:  ...
' Uses:    ...
' Param:   ...
' History: 23-01-2019 Ronald Bax     First version
' --------------------------------------------------------------------
```

```
wksResult2 - 5

Public Sub ckbOneDay_Click()
   If Date <> wksStores.Range("G2").Value2 Or Not wksStores.Range("$I$6").Value2 Then Exit Sub
   If wksStores.Range("$I$6").Value2 Then
      MsgBox "Can't use JustOneDay for TODAY." & vbCrLf & "Change the date first.", vbOKOnly + vbCriti
cal, "wksStores.ckbOneDay_Click"
   End If
   wksStores.Activate
   wksStores.Range("G2").Select
   wksStores.Range("$I$6").Value2 = False
End Sub
' ================================================================================

' ================================================================================
' ================================================================================
```

```
wksResult3 - 1

' Last Cleaned (Export): 20-03-2023 11:34
Option Explicit
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 4.00           Date: 20-03-2023
' Action:  ...
' Uses:    n.a.
' Param:   n.a.
' History: 23-01-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Public Sub GetAndProcessData()
   Const sCodeKostenpost = "U2101"
   Dim wks        As Worksheet
   Dim sDate      As String
   Dim sSQL       As String
   Dim sStoreIP   As String
   Dim iRow       As Long
   Dim iLastRow   As Long
   Dim iStLastRow As Long
   Dim bVoid      As Boolean
   Dim sFileName  As String
   Dim sSep       As String
   Dim sOneDay    As String
   Dim S          As String
   Dim sThisWB    As String
   sThisWB = ThisWorkbook.Name

   sDate = Format(wksStores.Range("G2").Value2, "yyyy-mm-dd", vbMonday)
   sSep = wksStores.Range("I5").Value2
   sOneDay = IIf(wksStores.Range("I6").Value2, "1", "0")

   sSQL = "EXEC DPEU.dbo.spDOM_ExtractPayrollNmbrsNL @ForDate='" & sDate & "', @Header=0, @JustOneDay=
" & sOneDay
   wksResult.Activate
   iStLastRow = EU_XLS_Lib.LastRow(aWks:=wksStores)
   iLastRow = 2
   For iRow = 2 To iStLastRow
      If wksStores.Range("A" & iRow).Value2 = "V" And wksStores.Range("B" & iRow).Value2 <> vbNullStri
ng Then
         wksStores.Range("E" & iRow).Value2 = vbNullString
         sStoreIP = EU_DOM_Lib.GetStoreToDNSName(sStoreNum:=wksStores.Range("B" & iRow).Value2)
         On Error Resume Next
         bVoid = EU_DB_Lib.SQLToWorksheet(aWks:=wksResult, aRng:=wksResult.Range("A" & iLastRow), sSto
reIP:=sStoreIP, sSQL:=sSQL, bHeaders:=False, bShowMsg:=False)
         On Error GoTo 0
         If bVoid Then
            wksStores.Range("E" & iRow).Value2 = "V"
         Else
            wksStores.Range("E" & iRow).Value2 = "X"
         End If
         iLastRow = EU_XLS_Lib.LastRow(aWks:=wksResult) + 1
      End If
   Next iRow

   'Replace Debt-Number for Multi-store employees
   Call wksMultiMW.ReplaceMultiInResult

   If iLastRow > 2 Then
      wksResult.Range("L2:L" & iLastRow - 1).FormulaR1C1 = "=IF(IFERROR(VLOOKUP(RC[-9]&LEFT(RC[-3],5),
Managers!C3:C4,2,FALSE),0)=0,IF(IFERROR(VLOOKUP(RC[-9],Managers!C3:C4,2,FALSE),0)=0,"""",VLOOKUP(RC[-9
],Managers!C3:C4,2,FALSE)),VLOOKUP(RC[-9]&LEFT(RC[-3],5),Managers!C3:C4,2,FALSE))"
   End If
   Call EU_XLS_Lib.AutoFitWorksheet(aWks:=wksResult)

   With wksResult
      ' ==================
      ' Save data and clean-up Temp
      ' ==================
      wksResult.Activate
      Application.DisplayAlerts = False
      On Error Resume Next
      sFileName = ActiveWorkbook.Path & "\HR-payroll.csv"
```

```
wksResult3 - 2

      Call EU_XLS_Lib.SaveSheetToTXTFile(aWks:=wksResult, sFilePath:=sFileName, sSepChar:=sSep)
      Application.DisplayAlerts = True
      On Error GoTo 0
      Application.Calculation = xlCalculationAutomatic
      ThisWorkbook.RefreshAll
      ' ====================
      ' Cleanup The result-set
      ' ====================
      iLastRow = EU_XLS_Lib.LastRow(aWks:=wksResult) + 1
      For iRow = iLastRow To 3 Step -1
         S = wksResult.Range("C" & iRow).Value2
         If (Len(S) = 5 And Left(S, 3) = "099") Or (Len(S) = 4 And Left(S, 2) = "99") Then
            wksResult.Rows(iRow).Delete
         End If
      Next iRow
   End With

' ====================
' Delete work sheets (if they exists)
' ====================
   Application.DisplayAlerts = False
   If Evaluate("ISREF('LooncomponentenVar'!A1)") Then Worksheets("LooncomponentenVar").Delete
   If Evaluate("ISREF('KostenverdelingLooncomponenten'!A1)") Then Worksheets("KostenverdelingLooncompo
nenten").Delete
   If Evaluate("ISREF('Temp'!A1)") Then Worksheets("Temp").Delete
   Application.DisplayAlerts = True

' ====================
' Copy Data to Nmbrs-format-output
' ====================
   wksResult.Unprotect
   Call EU_XLS_Lib.AutoFitWorksheet(aWks:=wksResult)
   wksResult.Copy After:=wksResult
   Sheets("Result (2)").Select
   ActiveSheet.Name = "Temp"

' ====================
' Cleanup Temp data - Remove the managers
' ====================
   With Sheets("Temp")
      .Activate
      iLastRow = EU_XLS_Lib.LastRow(aWks:=Sheets("Temp")) + 1
      For iRow = iLastRow To 2 Step -1
         If .Cells(iRow, 12).Value2 <> vbNullString Then .Rows(iRow).Delete
      Next iRow
   End With

' ====================
' Copy Temp to Nmbrs
' ====================
   Worksheets("Temp").Copy After:=Worksheets("Temp")
   Sheets("Temp (2)").Select
   ActiveSheet.Name = "LooncomponentenVar"
   Worksheets("Temp").Copy After:=Worksheets("LooncomponentenVar")
   Sheets("Temp (2)").Select
   ActiveSheet.Name = "KostenverdelingLooncomponenten"
   Call EU_XLS_Lib.ProtectSheet(aWks:=wksResult)

' ====================
' Cleanup Nmbrs data
' ====================
   With Sheets("LooncomponentenVar")
      .Activate
      .Range("A1").Select
      .Range("I:M").EntireColumn.Delete
      .Range("A1").Select
      .Range("A1").EntireRow.Insert
      .Range("A1").Value2 = "Import"
      .Range("E1").Value2 = "Looncomponenten"
      iLastRow = EU_XLS_Lib.LastRow(aWks:=Sheets("LooncomponentenVar")) + 1
      For iRow = iLastRow To 3 Step -1
         .Cells(iRow, 8).Value2 = vbNullString
      Next iRow
```

```
wksResult3 - 3

      Call EU_XLS_Lib.AutoFitWorksheet(aWks:=Sheets("LooncomponentenVar"))
   End With

   With Sheets("KostenverdelingLooncomponenten")
      .Activate
      .Range("A1").Select
      .Range("K:N").EntireColumn.Delete
      .Range("A1").Select
      .Range("A1").EntireRow.Insert
      .Range("A1").Value2 = "Import"
      .Range("E1").Value2 = "Kostenplaats"
      .Range("E2").Value2 = "PeriodeStart"
      .Range("F2").Value2 = "PeriodeEind"
      .Range("G2").Value2 = "Code"
      .Range("H2").Value2 = "Waarde"
      .Range("I2").Value2 = "Kostenplaats"
      .Range("J2").Value2 = "Kostensoort"
      iLastRow = EU_XLS_Lib.LastRow(aWks:=Sheets("KostenverdelingLooncomponenten")) + 1
      For iRow = iLastRow To 3 Step -1
         If .Range("A" & iRow).Value2 = vbNullString Then
            .Range("A" & iRow).EntireRow.Delete
         Else
            .Range("F" & iRow).Value2 = .Range("E" & iRow).Value2
            .Range("G" & iRow).Value2 = sCodeKostenpost
            .Range("H" & iRow).Value2 = Round(.Range("H" & iRow).Value2, 2)
            .Range("H" & iRow).NumberFormat = "#.00_ ;-#.00"
            .Range("J" & iRow).Value2 = Mid(.Range("I" & iRow).Value2, 1, 5)
            .Range("I" & iRow).Value2 = .Range("J" & iRow).Value2
            .Range("J" & iRow).Value2 = vbNullString
         End If
      Next iRow
      Call EU_XLS_Lib.AutoFitWorksheet(aWks:=Sheets("KostenverdelingLooncomponenten"))
   End With

' ====================
' Save data and clean-up Temp
' ====================
   Sheets("LooncomponentenVar").Activate
   Application.DisplayAlerts = False
   On Error Resume Next
   sFileName = ActiveWorkbook.Path & "\LooncomponentenVar.xlsx"
   sFileName = Replace(sFileName, ".xlsx", wksStores.Range("ExportExtention").Value2)
   ActiveSheet.Copy
   Application.ActiveWorkbook.SaveAs sFileName, ConflictResolution:=xlUserResolution, FileFormat:=51 '
 xlsx
   Application.DisplayAlerts = True
   On Error GoTo 0

   Application.Workbooks(sThisWB).Activate
   wksStores.Activate

   Sheets("KostenverdelingLooncomponenten").Activate
   Application.DisplayAlerts = False
   On Error Resume Next
   sFileName = ActiveWorkbook.Path & "\KostenverdelingLooncomponenten.xlsx"
   sFileName = Replace(sFileName, ".xlsx", wksStores.Range("ExportExtention").Value2)
   ActiveSheet.Copy
   Application.ActiveWorkbook.SaveAs sFileName, ConflictResolution:=xlUserResolution, FileFormat:=51 '
 xlsx
   Application.DisplayAlerts = True
   On Error GoTo 0

   Application.Workbooks(sThisWB).Activate
   wksStores.Activate

   sFileName = "LooncomponentenVar.xlsx"
   sFileName = Replace(sFileName, ".xlsx", wksStores.Range("ExportExtention").Value2)
   Application.Workbooks(sFileName).Close
   sFileName = "KostenverdelingLooncomponenten.xlsx"
   sFileName = Replace(sFileName, ".xlsx", wksStores.Range("ExportExtention").Value2)
   Application.Workbooks(sFileName).Close

   Application.DisplayAlerts = False
```

```
wksResult3 - 4

   If Evaluate("ISREF('Temp'!A1)") Then Worksheets("Temp").Delete
   Application.DisplayAlerts = True

' ===================
' Update pivot with doubles
' ===================
   With wksTelling
      .Activate
      .Unprotect
      Call EU_XLS_Lib.ClearSheet(aWks:=wksTelling, sCell:="E1", sRowCol:="col")
      .PivotTables("pivTelling").PivotCache.Refresh
      .Range("D3").Copy
      .Range("E3").PasteSpecial Paste:=xlPasteFormats, Operation:=xlNone, SkipBlanks:=False, Transpose
:=False
      Application.CutCopyMode = False
      .Range("E3").Value2 = "<="
      .Columns("E:E").Select
      With Selection
         .HorizontalAlignment = xlCenter
         .VerticalAlignment = xlCenter
         .WrapText = False
         .Orientation = 0
         .AddIndent = False
         .IndentLevel = 0
         .ShrinkToFit = False
         .ReadingOrder = xlContext
         .MergeCells = False
      End With
      .Range("E3").Select
      Selection.AutoFilter
      iLastRow = EU_XLS_Lib.LastRow(aWks:=wksTelling)
      .Range("E4:E" & iLastRow).FormulaR1C1 = "=IF(AND(RC[-2]<>"""",OR(RC[-4]=R[-1]C[-4],RC[-4]=R[1]C[
-4])),""<="","""")"
   End With

' ===================
' Return to Stores-sheet
' ===================
   With wksStores
      .Activate
      .Range("G2").Select
      If .Range("G3").Value2 <> .Range("I3").Value2 Then
         MsgBox "Not all stores imported.  Stores: " & CStr(.Range("G3").Value2) & ".  Imported: " & C
Str(.Range("I3").Value2) & ".", vbOKOnly + vbInformation
      Else
         MsgBox "Imported: " & CStr(.Range("I3").Value2) & " stores.", vbOKOnly + vbInformation
      End If
   End With

   Application.Calculation = xlCalculationAutomatic
   ThisWorkbook.RefreshAll
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax          Version: 4.00          Date: 20-03-2023
' Action:  ...
' Uses:    n.a.
' Param:   n.a.
' History: 23-01-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Public Sub ActivateResultPage()
   wksStores.Activate
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax          Version: 4.00          Date: 20-03-2023
' Action:  ...
' Uses:    ...
' Param:   ...
' History: 23-01-2019 Ronald Bax     First version
' --------------------------------------------------------------------
```

```
wksResult3 - 5

Public Sub ckbOneDay_Click()
   If Date <> wksStores.Range("G2").Value2 Or Not wksStores.Range("$I$6").Value2 Then Exit Sub
   If wksStores.Range("$I$6").Value2 Then
      MsgBox "Can't use JustOneDay for TODAY." & vbCrLf & "Change the date first.", vbOKOnly + vbCriti
cal, "wksStores.ckbOneDay_Click"
   End If
   wksStores.Activate
   wksStores.Range("G2").Select
   wksStores.Range("$I$6").Value2 = False
End Sub
' ================================================================================

' ================================================================================
' ================================================================================
```

```
' ====================================================================
'                              EU_ALL_Lib.bas
' ====================================================================
' Author:  Ronald Bax          Version: 2.30          Date: 25-07-2019
' Action:  Library to add generic Workbook-functions like MacroShortcuts,
'          ShowSysInfo, Import and Export Modules and Save-Backup on Open
' Uses:    EU_SYS_Lib, EU_VBP_Lib, EU_WIN_Lib and EU_XLS_Lib
' History: 20-03-2019 Ronald Bax     First version
' ====================================================================
' Public  Function GetVersion() As String
' Public  Function GetDTUpdate() As Date
' Public  Function LibFileExists(sLibDir As String) As Boolean
' Public  Function GetLibFileDate(sLibDir As String) As Date
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Private Function IsSheetLocked(sMe As String) As Boolean
' Private Sub      CreateMacroShortCuts()
' Private Sub      DeleteMacroShortCuts()
' Private Sub      AddLibrariesRequired()
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Public  Sub      SysInfo()
' Public  Sub      HideSysInfo()
' Public  Sub      DeleteSysInfo()
' Public  Sub      Export_All_Code()
' Public  Sub      Import_All_Code()
' Public  Sub      Update_Libraries()
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Public  Sub      Generic_Workbook_Open(aWks As Worksheet, ByVal bWantBackup As Boolean, Optional sSe
lCell As String = vbNullString)
' Public  Sub      Generic_Workbook_BeforeClose(bCancel As Boolean)
' ====================================================================
Option Explicit

' ====================================================================
'                       Constants, Globals and Types
' ====================================================================
Private Const EU_ALL_Lib_LVersion As String = "2.30"
Private Const EU_ALL_Lib_DTUpdate As String = "25-07-2019"
Private Const EU_ALL_Lib_FileName As String = "EU_ALL_Lib.bas"

Private Const GDELETE_DELAY As String = "00:00:03"  ' Delay for application.ontime()
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
Global GsMainSheet As String
Global GsMainCell  As String
' ====================================================================

' ====================================================================
'                           PUBLIC ROUTINES
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax          Version: 2.30          Date: 25-07-2019
' Action:  Get info about this library
' Uses:    EU_WIN_Lib.FileExists
'          EU_WIN_Lib.GetFileDateTime
' Param:   sLibDir
' History: 02-02-2016 Ronald Bax     First version
'          02-02-2019 Ronald Bax     2.00 Update
' --------------------------------------------------------------------
Public Function GetVersion() As String
   GetVersion = EU_ALL_Lib_LVersion
End Function
' --------------------------------------------------------------------
Public Function GetDTUpdate() As Date
   GetDTUpdate = CDate(EU_ALL_Lib_DTUpdate)
End Function
' --------------------------------------------------------------------
Public Function LibFileExists(sLibDir As String) As Boolean
   LibFileExists = EU_WIN_Lib.FileExists(sFilePath:=sLibDir & EU_ALL_Lib_FileName)
End Function
' --------------------------------------------------------------------
Public Function GetLibFileDate(sLibDir As String) As Date
   GetLibFileDate = EU_WIN_Lib.GetFileDateTime(sFilePath:=sLibDir & EU_ALL_Lib_FileName)
End Function
```

```
' ====================================================================

' ====================================================================
'                          PRIVATE ROUTINES
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 2.30           Date: 25-07-2019
' Action:  Check if sheet is locked and show message
' Uses:    EU_XLS_Lib.addVBIDEExt
' Param:   n.a.
' History: 20-03-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Private Function IsSheetLocked(sMe As String) As Boolean
   IsSheetLocked = ThisWorkbook.VBProject.Protection <> vbext_pp_none
   If IsSheetLocked Then
      MsgBox "The workbook is password-protected. Please unlock.", vbCritical + vbOKOnly, sMe
   End If
End Function
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 2.30           Date: 25-07-2019
' Action:  Add/delete the macro-shortcuts
' Uses:    EU_XLS_Lib.addVBIDEExt
' Param:   n.a.
' History: 20-03-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Private Sub CreateMacroShortCuts()
   Application.OnKey "+^{P}", "EU_ALL_Lib.SysInfo"
   Application.MacroOptions Macro:="EU_ALL_Lib.SysInfo", Description:="CTRL-SHIFT-P Show System-inform
ation"
End Sub
' ====================================================================
Private Sub DeleteMacroShortCuts()
   Application.OnKey "+^{P}"
   Application.OnKey "+^{I}", "" ' Formerly used
   Application.OnKey "+^{O}", "" ' Formerly used
   Application.OnKey "+^{L}", "" ' Formerly used
   Application.OnKey "+^{I}" ' Formerly used
   Application.OnKey "+^{O}" ' Formerly used
   Application.OnKey "+^{L}" ' Formerly used
End Sub
' ====================================================================
Private Sub AddLibrariesRequired()
   Call EU_XLS_Lib.addVBIDEExt
End Sub
' ====================================================================

' ====================================================================
'                          PUBLIC ROUTINES
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 2.30           Date: 25-07-2019
' Action:  Show sysinfo, and Import/Export modules and sheets
' Uses:    EU_SYS_Lib.GetSysInfo
'          EU_VBP_Lib.Export_All_Code
'          EU_VBP_Lib.Import_All_Code
'          EU_VBP_Lib.UpdateAllEULibs
'          EU_VBP_Lib.GetStandardLibraryPath
' Param:   n.a.
' History: 20-03-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Public Sub SysInfo()
   Dim wks    As Worksheet
   Dim bFound As Boolean
   bFound = False
   For Each wks In ThisWorkbook.Worksheets
      If wks.Name = "SysInfo" Then
         bFound = True
         Exit For
      End If
```

```
   Next wks
   If bFound Then
      ThisWorkbook.Sheets("SysInfo").Visible = True
      ThisWorkbook.Sheets("SysInfo").Activate
   End If
   If Not EU_ALL_Lib.IsSheetLocked(sMe:="EU_ALL_Lib.SysInfo") Then Call EU_SYS_Lib.GetSysInfo(bShow:=T
rue)
   Set wks = Nothing
End Sub
' ====================================================================
Public Sub HideSysInfo()
   ThisWorkbook.Sheets(1).Activate
   On Error Resume Next
   wksSysInfo.Visible = False
   On Error GoTo 0
End Sub
' ====================================================================
Public Sub DeleteSysInfo()
   Dim sCmd As String
   If EU_ALL_Lib.IsSheetLocked(sMe:="EU_ALL_Lib.DeleteSysInfo") Then Exit Sub
   ThisWorkbook.Sheets(1).Activate
   sCmd = "EU_SYS_Lib.MacroDeleteSysInfo"
   Application.OnTime Now() + TimeValue(GDELETE_DELAY), sCmd ' This deletes the module
End Sub
' ====================================================================
Public Sub Export_All_Code()
   If EU_ALL_Lib.IsSheetLocked(sMe:="EU_ALL_Lib.Export_All_Code") Then Exit Sub
   Call EU_VBP_Lib.Export_All_Code
   Call EU_SYS_Lib.GetSysInfo(bShow:=True)
End Sub
' ====================================================================
Public Sub Import_All_Code()
   If EU_ALL_Lib.IsSheetLocked(sMe:="EU_ALL_Lib.Import_All_Code") Then Exit Sub
   Call EU_VBP_Lib.Import_All_Code(bIncLibs:=False)
   Call EU_SYS_Lib.GetSysInfo(bShow:=True)
End Sub
' ====================================================================
Public Sub Update_Libraries()
   If EU_ALL_Lib.IsSheetLocked(sMe:="EU_ALL_Lib.Update_Libraries") Then Exit Sub
   If EU_VBP_Lib.GetStandardLibraryPath() = vbNullString Then Exit Sub
   ActiveWorkbook.Save
   Call EU_VBP_Lib.UpdateAllEULibs
   Call EU_WIN_Lib.Pause(iSeconds:=10)
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 2.30           Date: 25-07-2019
' Action:  Create a backup and remove older backups if present
' Uses:    EU_XLS_Lib.ShowProcessing
'          EU_XLS_Lib.Workbook_Open_AutoBackup
' Param:   aWks        --
'          bWantBackup --
'          sSelCell    --
' History: 20-03-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Public Sub Generic_Workbook_Open(aWks As Worksheet, ByVal bWantBackup As Boolean, Optional sSelCell As
 String = vbNullString)
   Call EU_XLS_Lib.ShowProcessing(bSwitchOn:=False)
   If Not aWks Is Nothing Then
      GsMainSheet = aWks.Name
      GsMainCell = sSelCell
      Call EU_XLS_Lib.Workbook_Open_AutoBackup(aWks:=aWks, bWantBackup:=bWantBackup, sSelCell:=GsMainC
ell)
   End If
   Call EU_ALL_Lib.CreateMacroShortCuts
   Call EU_XLS_Lib.ShowProcessing(bSwitchOn:=True)
   aWks.Activate
   aWks.Range(GsMainCell).Select
End Sub
' ====================================================================
Public Sub Generic_Workbook_BeforeClose(ByRef bCancel As Boolean)
   Call EU_ALL_Lib.HideSysInfo
```

```
   Call EU_ALL_Lib.DeleteMacroShortCuts
End Sub
' ====================================================================


' ====================================================================
' ====================================================================
```

```
' ====================================================================
'                              EU_DB_lib.bas
' ====================================================================
' Author:  Ronald Bax          Version: 3.00          Date: 14-03-2021
' Action:  Library with ADO DB functions
' Uses:    EU_XLS_Lib
' History: 11-11-2016 Ronald Bax     First version
'          02-02-2017 Ronald Bax     1.20 General update
' ====================================================================
' Note:    This Lib is dependent of EU_WIN_Lib and EU_XLS_Lib
' ====================================================================
' Public Function GetVersion() As String
' Public Function GetDTUpdate() As Date
' Public Function LibFileExists(sLibDir As String) As Boolean
' Public Function GetLibFileDate(sLibDir As String) As Date
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Public Function ExecSQL(sqlStr As String, ByVal bWantResults As Boolean, Optional ByVal bShowMsg As
Boolean = True) As Boolean
' Public Function SQLToWorksheet(aWks As Worksheet, sStartCell As Range, sStoreIP As String, sSQL As S
tring, _
'                Optional ByVal bHeaders As Boolean = False, Optional ByVal bShowMsg As Boolean = Tru
e) As Boolean
' Public Function getPingResult(sHost As String) As String
' ====================================================================
Option Explicit

' ====================================================================
'                          Constants and Types
' ====================================================================
Private Const EU_DB_Lib_LVersion As String = "3.00"
Private Const EU_DB_Lib_DTUpdate As String = "14-03-2021"
Private Const EU_DB_Lib_FileName As String = "EU_DB_Lib.bas"
' ====================================================================

' ====================================================================
'                                  GLOBALS
' ====================================================================
Global GGadoConnString As String
Global GadoConn As Object
Global GadoRecs As Object
Global GadoSQL As Object

Public Const DefHdrColor As Long = 16773330 ' RGB(210, 240, 255) -- light blue
Global GhdrColor As Long  ' if not set or 0, it will be DefHdrColor
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax          Version: 2.30          Date: 25-07-2019
' Action:  Get info about this library
' Uses:    .
' Param:   .
' History: 02-02-2016 Ronald Bax     First version
'          02-02-2019 Ronald Bax     2.00 Update
' --------------------------------------------------------------------
Public Function GetVersion() As String
   GetVersion = EU_DB_Lib_LVersion
End Function
' --------------------------------------------------------------------
Public Function GetDTUpdate() As Date
   GetDTUpdate = CDate(EU_DB_Lib_DTUpdate)
End Function
' --------------------------------------------------------------------
Public Function LibFileExists(sLibDir As String) As Boolean
   LibFileExists = EU_WIN_Lib.FileExists(sFilePath:=sLibDir & EU_DB_Lib_FileName)
End Function
' --------------------------------------------------------------------
Public Function GetLibFileDate(sLibDir As String) As Date
   GetLibFileDate = EU_WIN_Lib.GetFileDateTime(sFilePath:=sLibDir & EU_DB_Lib_FileName)
End Function
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax          Version: 2.30          Date: 25-07-2019
```

EU_DB_Lib - 2

```
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 02-02-2016 Ronald Bax     First version
' ---------------------------------------------------------------------
Public Function ExecSQL(sqlStr As String, ByVal bWantResults As Boolean, Optional ByVal bShowMsg As Bo
olean = True) As Boolean
   Const cMe = "EU_DB_Lib.ExecSQL"
   On Error GoTo ErrHandle
   ' Init the ADO objects
   If GadoConn Is Nothing Then Set GadoConn = CreateObject("ADODB.Connection")
   If GadoRecs Is Nothing Then Set GadoRecs = CreateObject("ADODB.Recordset")
   If GadoSQL Is Nothing Then Set GadoSQL = CreateObject("ADODB.Command")
   If GadoConn.State = 1 Then
      GadoConn.Close
   End If
   ' Create the connection
   If GadoConn.State = 0 Then
      GadoConn.Open GGadoConnString
      GadoConn.CursorLocation = 3 'adUseClient
   End If
   ' Set GadoSQL's values
   With GadoSQL
      .CommandText = sqlStr
      .CommandType = 1 'adCmdText
      .ActiveConnection = GadoConn
   End With
   ' Execute the sql command, reurning values if we neeed to
   If bWantResults = True Then
      Set GadoRecs = GadoSQL.Execute
   Else
      GadoSQL.Execute
   End If
   ExecSQL = True
   Exit Function
ErrHandle:
   If bShowMsg = True Then MsgBox "ExecSQL. Err: " & Err.Description, vbCritical + vbOKOnly, cMe
   ExecSQL = False
End Function
' =====================================================================

' ---------------------------------------------------------------------
' Author:  Ronald Bax           Version: 2.30           Date: 25-07-2019
' Action:  ...
' Uses:    EU_XLS_Lib.getRangeFromCellToEnd()
'          EU_DB_Lib.ExecSQL()
' Param:   ...
' History: 18-02-2016 Ronald Bax     First version
' ---------------------------------------------------------------------
Public Function SQLToWorksheet(aWks As Worksheet, aRng As Range, sStoreIP As String, sSQL As String, O
ptional ByVal bHeaders As Boolean = False, Optional ByVal bShowMsg As Boolean = True) As Boolean
   Dim rng       As Range
   Dim sStart    As String
   Dim iIdx      As Long
   Dim iOfs      As Long
   Dim C         As Long
   Dim bSaveStat As Boolean
   Dim bProt     As Boolean

   bProt = aWks.ProtectContents
   aWks.Unprotect
   bSaveStat = Application.ScreenUpdating
   Application.ScreenUpdating = False
   sStart = aRng.Address
   If GhdrColor = 0 Then GhdrColor = DefHdrColor
   GGadoConnString = "Provider=SQLOLEDB.1;Password=maint;Persist Security Info=True;User ID=maint;Init
ial Catalog=DPEU;Data Source=" & sStoreIP
   ' Write results to sheet
   iIdx = aRng.Row
   iOfs = aRng.Column - 1
   SQLToWorksheet = EU_DB_Lib.ExecSQL(sqlStr:=sSQL, bWantResults:=True, bShowMsg:=bShowMsg)
   If GadoRecs.Fields.Count > 0 Then
      Set rng = EU_XLS_Lib.FromCellToEnd(aWks:=aWks, sCell:=sStart)
```

```
      rng.Borders.LineStyle = xlLineStyleNone
      rng.EntireRow.Delete
      With aWks
         If Not GadoRecs.EOF And bHeaders = True Then
            For C = 1 To GadoRecs.Fields.Count
               .Cells(iIdx, C + iOfs).Interior.Color = GhdrColor
               .Cells(iIdx, C + iOfs).Value2 = GadoRecs.Fields(C - 1).Name
               .Cells(iIdx, C + iOfs).Borders.LineStyle = xlContinuous
               .Cells(iIdx, C + iOfs).Borders.Weight = xlThin
            Next
            iIdx = iIdx + 1
         End If
         While Not GadoRecs.EOF
            For C = 1 To GadoRecs.Fields.Count
               .Cells(iIdx, C + iOfs).Interior.Color = vbWhite
               .Cells(iIdx, C + iOfs).Value = GadoRecs.Fields(C - 1) ' must be Value
               .Cells(iIdx, C + iOfs).Borders.LineStyle = xlContinuous
               .Cells(iIdx, C + iOfs).Borders.Weight = xlThin
            Next
            GadoRecs.MoveNext
            iIdx = iIdx + 1
         Wend
         GadoRecs.Close
      End With
   End If
   Set rng = EU_XLS_Lib.FromCellToEnd(aWks:=aWks, sCell:=sStart)
   rng.Cells.EntireColumn.AutoFit
   aWks.Select
   aWks.Range(sStart).Select
   Application.ScreenUpdating = bSaveStat
   If bProt Then aWks.Protect
   Set rng = Nothing
End Function
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 2.30           Date: 25-07-2019
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 02-02-2016 Ronald Bax     First version
' --------------------------------------------------------------------
Public Function getPingResult(sHost As String) As String
   Dim oPing   As Object
   Dim oStatus As Object
   Dim sRes    As String
   Set oPing = GetObject("winmgmts:{impersonationLevel=impersonate}").ExecQuery("Select * from Win32_P
ingStatus Where Address = '" & sHost & "'")
   sRes = vbNullString
   For Each oStatus In oPing
      Select Case oStatus.StatusCode
         Case 0: sRes = "Connected"
         Case 11001: sRes = "Buffer too small"
         Case 11002: sRes = "Destination net unreachable"
         Case 11003: sRes = "Destination host unreachable"
         Case 11004: sRes = "Destination protocol unreachable"
         Case 11005: sRes = "Destination port unreachable"
         Case 11006: sRes = "No resources"
         Case 11007: sRes = "Bad option"
         Case 11008: sRes = "Hardware error"
         Case 11009: sRes = "Packet too big"
         Case 11010: sRes = "Request timed out"
         Case 11011: sRes = "Bad request"
         Case 11012: sRes = "Bad route"
         Case 11013: sRes = "Time-To-Live (TTL) expired transit"
         Case 11014: sRes = "Time-To-Live (TTL) expired reassembly"
         Case 11015: sRes = "Parameter problem"
         Case 11016: sRes = "Source quench"
         Case 11017: sRes = "Option too big"
         Case 11018: sRes = "Bad destination"
         Case 11032: sRes = "Negotiating IPSEC"
         Case 11050: sRes = "General failure"
         Case Else:  sRes = "Unknown host"
```

```
      End Select
      getPingResult = sRes
   Next
   Set oStatus = Nothing
   Set oPing = Nothing
End Function
' ========================================================================


' ========================================================================
' ========================================================================
```

```
' ====================================================================
'                             EU_DOM_Lib.bas
' ====================================================================
' Author:  Ronald Bax           Version: 3.00          Date: 14-03-2021
' Action:  Library with Dominos specific functions
' Uses:    EU_WIN_Lib
' History: 25-11-2016 Ronald Bax     First version
'          02-02-2017 Ronald Bax     1.20 General update
'          02-02-2019 Ronald Bax     2.00 Update
' ====================================================================
' Note:    This Lib is dependent of EU_WIN_Lib
' ====================================================================
' Public Function GetVersion() As String
' Public Function GetDTUpdate() As Date
' Public Function LibFileExists(sLibDir As String) As Boolean
' Public Function GetLibFileDate(sLibDir As String) As Date
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Public Function GetCountry() As String
' Public Function GetLanguage() As String
' Public Function GetStoreToCountry(sStoreNum As String) As String
' Public Function GetStoreToDNSName(sStoreNum As String) As String
' Public Function GetCountryDateMask(Optional ByVal bYMD As Boolean = False) As String
' Public Function GetCountryDateTimeMask(Optional ByVal bYMD As Boolean = False) As String
' Public Function GetLocCountryDateMask(Optional ByVal bYMD As Boolean = False) As String
' Public Function GetLocCountryDateTimeMask(Optional ByVal bYMD As Boolean = False) As String
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Public Function IsLeapYear(Optional ByVal iYear As Integer = 0) As Boolean
' Public Function GetEasterSunday(Optional ByVal iYear As Integer = 0) As Date
' Public Function IsHoliday(ByVal dtDate As Date, Optional sCountry As String = vbNullString, Optional
 ByVal bRefresh As Boolean=False) As String
' Public Function GetWeek_ISO8601(Optional ByVal dtDate As Date) As Integer
' Public Function GetWeekStr_ISO8601(Optional ByVal dtDate As Date) As String
' Public Sub GetHolidays(aRng As Range, Optional ByVal iYear As Integer = 0, Optional sCountry As Stri
ng = vbNullString, Optional ByVal bRefresh As Boolean=False)
' Public Sub GetFiscalYear(aRng As Range, Optional ByVal iYear As Integer = 0, Optional ByVal bRefresh
 As Boolean = False)
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Private Function GetArrayDimension(ByRef aArray() As String) As Byte
' Private Sub FillHolidayArray(Optional iYear As Integer, Optional sCountry As String)
' Private Sub FillFiscalYearArray(Optional ByVal iYear As Integer = 0)
' Private Sub AlphabeticallySortArray(ByRef aArray() As String)
' ====================================================================
Option Explicit

' ====================================================================
'                          Constants and Types
' ====================================================================
Private Const EU_DOM_Lib_LVersion As String = "3.00"
Private Const EU_DOM_Lib_DTUpdate As String = "14-03-2021"
Private Const EU_DOM_Lib_FileName As String = "EU_DOM_Lib.bas"
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
Private GarrHolidays(12, 1) As String    ' Holiday(x) + Date , Holiday(x) + Name
Private GarrFiscalYear(12, 1) As String  ' 12 periods x (startdate + enddate)
' ====================================================================
Private Declare PtrSafe Function GetLocaleInfo Lib "kernel32" Alias "GetLocaleInfoA" (ByVal Locale As
Long, ByVal LCType As Long, ByVal lpLCData As String, ByVal cchData As Long) As Long
Private Const LOCALE_USER_DEFAULT As Long = &H400
Private Const LOCALE_STIMEFORMAT  As Long = &H1003
Private Const LOCALE_SSHORTDATE   As Long = &H1F
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00          Date: 14-03-2021
' Action:  Get info about this library
' Uses:    .
' Param:   .
' History: 02-02-2016 Ronald Bax     First version
'          02-02-2019 Ronald Bax     2.00 Update
' --------------------------------------------------------------------
Public Function GetVersion() As String
   GetVersion = EU_DOM_Lib_LVersion
End Function
' --------------------------------------------------------------------
```

```
Public Function GetDTUpdate() As Date
   GetDTUpdate = CDate(EU_DOM_Lib_DTUpdate)
End Function
' ---------------------------------------------------------------------
Public Function LibFileExists(sLibDir As String) As Boolean
   LibFileExists = EU_WIN_Lib.FileExists(sFilePath:=sLibDir & EU_DOM_Lib_FileName)
End Function
' ---------------------------------------------------------------------
Public Function GetLibFileDate(sLibDir As String) As Date
   GetLibFileDate = EU_WIN_Lib.GetFileDateTime(sFilePath:=sLibDir & EU_DOM_Lib_FileName)
End Function
' =====================================================================

' ---------------------------------------------------------------------
' Author:  Ronald Bax          Version: 3.00          Date: 14-03-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 02-02-2016 Ronald Bax     First version
' ---------------------------------------------------------------------
Public Function getCountry() As String
   Select Case Application.International(xlCountrySetting)
      Case 1:    getCountry = "US"  ' The United States of America  English
      Case 31:   getCountry = "NL"  ' The Netherlands               Dutch
      Case 32:   getCountry = "BE"  ' Belgium                       Dutch/French
      Case 33:   getCountry = "FR"  ' France                        French
      Case 44:   getCountry = "GB"  ' United Kindom                 English
      Case 45:   getCountry = "DK"  ' Denmark                       Danish
      Case 49:   getCountry = "DE"  ' Germany                       German
      Case 61:   getCountry = "AU"  ' Australia                     English
      Case 352:  getCountry = "LU"  ' Luxembourg                    French
      Case Else: getCountry = "??"
   End Select
End Function
' =====================================================================

' ---------------------------------------------------------------------
' Author:  Ronald Bax          Version: 3.00          Date: 14-03-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 02-02-2016 Ronald Bax     First version
' ---------------------------------------------------------------------
Public Function getLanguage() As String
   Select Case Application.LanguageSettings.LanguageID(msoLanguageIDUI)
      Case "1031": getLanguage = "de-DE" ' "&H0407"  German
      Case "1033": getLanguage = "en-US" ' "&H0409"  English (US)
      Case "1036": getLanguage = "fr-FR" ' "&H040C"  French
      Case "1043": getLanguage = "nl-NL" ' "&H0413"  Dutch
      Case "2057": getLanguage = "en-GB" ' "&H0809"  English (British)
      Case "2067": getLanguage = "nl-BE" ' "&H0813"  Dutch (Belgium)
      Case "2060": getLanguage = "fr-BE" ' "&H080C"  French (Belgium)
      Case "3081": getLanguage = "en-AU" ' "&H0C09"  English (Australian)
      Case Else:   getLanguage = "??"
   End Select
End Function
' =====================================================================

' ---------------------------------------------------------------------
' Author:  Ronald Bax          Version: 3.00          Date: 14-03-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 02-02-2016 Ronald Bax     First version
' ---------------------------------------------------------------------
Public Function GetStoreToCountry(sStoreNum As String) As String
   Dim sCountry As String
   Select Case sStoreNum
      Case 88100, 88110, 88190: sCountry = "NL"
      Case 88200, 88210:        sCountry = "BE"
      Case 88300, 88310:        sCountry = "FR"
      Case 88400, 88410, 88420: sCountry = "DE"
      Case 88500, 88510:        sCountry = "LU"
```

```
      Case 88600, 88610:        sCountry = "DK"
      Case Is < 27010:          sCountry = vbNullString
      Case 27010 To 27150:      sCountry = "DK"
      Case 30540 To 31139:      sCountry = "NL"
      Case 31140 To 31639:      sCountry = "BE"
      Case 31640 To 31659:      sCountry = "LU"
      Case 31660 To 33659:      sCountry = "FR"
      Case 33710 To 34659:      sCountry = "DE"
      Case Else:                sCountry = vbNullString
   End Select
   GetStoreToCountry = IIf(sCountry <> vbNullString, UCase(sCountry), vbNullString)
End Function
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 02-02-2016 Ronald Bax     First version
' --------------------------------------------------------------------
Public Function GetStoreToDNSName(sStoreNum As String) As String
   Dim sCountry As String
   sCountry = EU_DOM_Lib.GetStoreToCountry(sStoreNum:=sStoreNum)
   GetStoreToDNSName = IIf(sCountry <> vbNullString, "dp" & LCase(sCountry) & sStoreNum & ".dpev.net",
 vbNullString)
End Function
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 02-02-2016 Ronald Bax     First version
' --------------------------------------------------------------------
Public Function GetCountryDateMask(Optional ByVal bYMD As Boolean = False) As String
   Dim sDTSep     As String
   Dim sDayCode   As String
   Dim sMonthCode As String
   Dim sYearCode  As String
   With Application
      sDTSep = .International(xlDateSeparator)
      sDayCode = .International(xlDayCode)
      sMonthCode = .International(xlMonthCode)
     sYearCode = .International(xlYearCode)
   End With
   GetCountryDateMask = IIf(bYMD, String(4, sYearCode) & String(2, sMonthCode) & String(2, sDayCode),
_
                            String(2, sDayCode) & sDTSep & String(2, sMonthCode) & sDTSep & Stri
ng(4, sYearCode))
End Function
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  ...
' Uses:    EU_DOM_Lib.getCountryDateMask()
' Param:   ...
' History: 02-02-2016 Ronald Bax     First version
' --------------------------------------------------------------------
Public Function GetCountryDateTimeMask(Optional ByVal bYMD As Boolean = False) As String
   Dim sTMSep    As String
   Dim sHourCode As String
   Dim sMinCode  As String
   With Application
      sTMSep = .International(xlTimeSeparator)
      sHourCode = .International(xlHourCode)
      sMinCode = .International(xlMinuteCode)
   End With
   GetCountryDateTimeMask = IIf(bYMD, EU_DOM_Lib.GetCountryDateMask(bYMD:=bYMD) & String(2, sHourCode)
 & String(2, sMinCode), _
                                EU_DOM_Lib.GetCountryDateMask(bYMD:=bYMD) & vbSpace & String(2,
```

```
sHourCode) & ":" & String(2, sMinCode))
End Function
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 02-02-2016 Ronald Bax     First version
' --------------------------------------------------------------------
Public Function GetLocCountryDateMask(Optional ByVal bYMD As Boolean = False) As String
   Dim sDTSep     As String
   Dim sDayCode   As String
   Dim sMonthCode As String
   Dim sYearCode  As String
   Dim sDateMask  As String
   Dim lRet       As Long

   sDateMask = Space(255)
   lRet = GetLocaleInfo(LOCALE_USER_DEFAULT, LOCALE_SSHORTDATE, sDateMask, Len(sDateMask))
   sDateMask = Left(sDateMask, lRet - 1)

   With Application
      sDTSep = .International(xlDateSeparator)
      sDayCode = .International(xlDayCode)
      sMonthCode = .International(xlMonthCode)
      sYearCode = .International(xlYearCode)
      If Len(sDateMask) = 10 And Mid(sDateMask, 3, 1) = Mid(sDateMask, 6, 1) Then
         sDTSep = Mid(sDateMask, 3, 1)
         sDayCode = Mid(sDateMask, 1, 2)
         sMonthCode = Mid(sDateMask, 4, 2)
         sYearCode = Mid(sDateMask, 7, 4)
      End If
      If Len(sDateMask) = 10 And Mid(sDateMask, 5, 1) = Mid(sDateMask, 8, 1) Then
         sDTSep = Mid(sDateMask, 5, 1)
         sDayCode = Mid(sDateMask, 9, 2)
         sMonthCode = Mid(sDateMask, 6, 2)
         sYearCode = Mid(sDateMask, 1, 4)
      End If
   End With
   GetLocCountryDateMask = IIf(bYMD, String(4, sYearCode) & String(2, sMonthCode) & String(2, sDayCode
), _
                               String(2, sDayCode) & sDTSep & String(2, sMonthCode) & sDTSep & S
tring(4, sYearCode))
End Function
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  ...
' Uses:    EU_DOM_Lib.getCountryDateMask()
' Param:   ...
' History: 02-02-2016 Ronald Bax     First version
' --------------------------------------------------------------------
Public Function GetLocCountryDateTimeMask(Optional ByVal bYMD As Boolean = False) As String
   Dim sTMSep    As String
   Dim sHourCode As String
   Dim sMinCode  As String
   Dim sTimeMask As String
   Dim lRet      As Long

   sTimeMask = Space(255)
   lRet = GetLocaleInfo(LOCALE_USER_DEFAULT, LOCALE_STIMEFORMAT, sTimeMask, Len(sTimeMask))
   sTimeMask = Left(sTimeMask, lRet - 1)

   With Application
      sTMSep = .International(xlTimeSeparator)
      sHourCode = .International(xlHourCode)
      sMinCode = .International(xlMinuteCode)
      If Len(sTimeMask) = 8 And Mid(sTimeMask, 3, 1) = Mid(sTimeMask, 6, 1) Then
         sTMSep = Mid(sTimeMask, 3, 1)
         sHourCode = Mid(sTimeMask, 1, 2)
```

```
         sMinCode = Mid(sTimeMask, 4, 2)
      End If
   End With
   GetLocCountryDateTimeMask = IIf(bYMD, GetLocCountryDateMask(bYMD:=bYMD) & String(2, sHourCode) & St
ring(2, sMinCode), _
                                   GetLocCountryDateMask(bYMD:=bYMD) & vbSpace & String(2, sHour
Code) & ":" & String(2, sMinCode))
End Function
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  Determine for what year we need to calculate
' Uses:    n.a.
' Param:   ...
' History: 13-03-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Public Function IsLeapYear(Optional ByVal iYear As Integer = 0) As Boolean
   Dim y As Integer
   y = IIf(iYear <= 0, Year(Now), iYear)
   IsLeapYear = ((Round(y / 4) = (y / 4)) And Not (Round(y / 100) = (y / 100))) Or ((Round(y / 400) =
(y / 400)))
End Function
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 13-03-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Public Function GetEasterSunday(Optional ByVal iYear As Integer = 0) As Date
   Dim y As Integer
   Dim A As Integer, b As Integer, C As Integer, d As Integer, E As Integer, F As Integer, G As Intege
r
   Dim H As Integer, I As Integer, J As Integer, K As Integer, L As Integer, M As Integer
   ' Determine for what year we need to calculate
   y = IIf(iYear <= 0, Year(Now), iYear)
   ' Calculate the date of eastern
   ' Eastern is on the first Sunday after the first full-moon after start of spring
   A = y Mod 19
   b = y \ 100
   C = y Mod 100
   d = b \ 4
   E = b Mod 4
   F = (b + 8) \ 25
   G = (b - F + 1) \ 3
   H = (19 * A + b - d - G + 15) Mod 30
   I = C \ 4
   J = C Mod 4
   K = (32 + 2 * E + 2 * I - H - J) Mod 7
   L = (A + 11 * H + 22 * K) \ 451
   M = H + K - 7 * L + 114
   GetEasterSunday = CDate(y & "-" & (M \ 31) & "-" & ((M Mod 31) + 1))
End Function
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  .
' Uses:    .
' Param:   .
' History: 13-03-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Public Function IsHoliday(ByVal dtDate As Date, Optional sCountry As String, Optional ByVal bRefresh A
s Boolean = False) As String
   Dim I As Integer
   If CStr(dtDate) = "00:00:00" Then dtDate = Now()
   dtDate = CDate(Format(CStr(dtDate), "yyyy-mm-dd")) ' Only the datepart of this datetime...
   If GarrHolidays(0, 0) = vbNullString Or bRefresh Then
      Call EU_DOM_Lib.FillHolidayArray(iYear:=Year(dtDate), sCountry:=sCountry)
      If GarrHolidays(0, 0) <> vbNullString Then Exit Function
```

```
   End If
   For I = LBound(GarrHolidays) To UBound(GarrHolidays)
      If GarrHolidays(I, 0) <> vbNullString Then
         If CDate(GarrHolidays(I, 0)) = dtDate Then
         IsHoliday = GarrHolidays(I, 1)
         Exit Function
         End If
      End If
   Next I
   IsHoliday = vbNullString
End Function
' ===========================================================================

' ---------------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  .
' Uses:    .
' Param:   .
' History: 13-03-2019 Ronald Bax     First version
' ---------------------------------------------------------------------------
Public Function GetWeek_ISO8601(Optional ByVal dtDate As Date = CDate("00:00:00")) As Integer
   Dim iWeek As Integer
   If CStr(dtDate) = "00:00:00" Then dtDate = Now()
   dtDate = CDate(Format(CStr(dtDate), "yyyy-mm-dd")) ' Only the datepart of this datetime...
   iWeek = Format(dtDate - Weekday(dtDate, vbMonday) + 4, "ww", vbMonday, vbFirstFourDays)
   If Month(dtDate) = 1 And iWeek > 50 Then
      iWeek = 0 - iWeek
   End If
   If Month(dtDate) = 12 And iWeek < 2 Then
      iWeek = 0 - iWeek
   End If
   GetWeek_ISO8601 = iWeek
End Function
' ===========================================================================

' ---------------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  .
' Uses:    .
' Param:   .
' History: 13-03-2019 Ronald Bax     First version
' ---------------------------------------------------------------------------
Public Function GetWeekStr_ISO8601(Optional ByVal dtDate As Date = CDate("00:00:00")) As String
   Dim I As Integer
   Dim y As Integer
   If CStr(dtDate) = "00:00:00" Then dtDate = Now()
   dtDate = CDate(Format(CStr(dtDate), "yyyy-mm-dd")) ' Only the datepart of this datetime...
   y = Year(dtDate)
   I = EU_DOM_Lib.GetWeek_ISO8601(dtDate:=dtDate)
   If I < 0 Then
      y = IIf(I = -1, y + 1, y - 1)
      I = 0 - I
   End If
   GetWeekStr_ISO8601 = vbNullString & y & "-" & Right("0" & I, 2)
End Function
' ===========================================================================

' ---------------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  .
' Uses:    .
' Param:   .
' History: 13-03-2019 Ronald Bax     First version
' ---------------------------------------------------------------------------
Public Sub GetHolidays(aRng As Range, Optional ByVal iYear As Integer = 0, Optional sCountry As String
, Optional ByVal bRefresh As Boolean = False)
   Const cMe = "EU_DOM_Lib.GetHolidays"
   Dim y     As Integer
   Dim iRows As Integer
   Dim I     As Integer
   Dim sMsg  As String
   ' Determine for what year we need to calculate and Fill the array
   y = IIf(iYear <= 0, Year(Now), iYear)
```

```
   If GarrHolidays(0, 0) = vbNullString Or bRefresh Then
      Call EU_DOM_Lib.FillHolidayArray(iYear:=y, sCountry:=sCountry)
   End If
   ' Test if given range is big enough and is empty
   iRows = 0
   For I = LBound(GarrHolidays) To UBound(GarrHolidays)
      iRows = I
      If GarrHolidays(I, 0) = vbNullString Then
         Exit For
      End If
   Next I
   sMsg = vbNullString
   With aRng
      If .Rows.Count < iRows Then sMsg = "The OutputRange should cover at least " & iRows & " rows!."
      If .Columns.Count <> 2 Then sMsg = "The OutputRange should cover 2 Columns!."
      If WorksheetFunction.CountA(aRng) <> 0 Then sMsg = "The OutputRange must be empty!."
      If sMsg <> vbNullString Then
         .Worksheet.Activate
         .Select
         MsgBox sMsg, vbCritical + vbOKOnly, cMe
         Exit Sub
      End If
      ' Display the results
      If GarrHolidays(0, 0) <> vbNullString Then
         For I = LBound(GarrHolidays) To UBound(GarrHolidays)
            If GarrHolidays(I, 0) = vbNullString Then Exit For
            .Worksheet.Cells(.Row + I, .Column + 0).Value2 = GarrHolidays(I, 0)
            .Worksheet.Cells(.Row + I, .Column + 1).Value2 = GarrHolidays(I, 1)
         Next I
         .Worksheet.Cells(.Row, .Column).Select
      End If
   End With
End Sub
' ============================================================================

' ----------------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  .
' Uses:    .
' Param:   .
' History: 13-03-2019 Ronald Bax     First version
' ----------------------------------------------------------------------------
Public Sub GetFiscalYear(aRng As Range, Optional ByVal iYear As Integer = 0, Optional ByVal bRefresh A
s Boolean = False, Optional ByVal bTimesheet As Boolean = False)
   Const cMe = "EU_DOM_Lib.GetFiscalYear"
   Dim y     As Integer
   Dim I     As Integer
   Dim R     As Integer
   Dim C     As Integer
   Dim sMsg  As String

   ' Determine for what year we need to calculate and Fill the array
   y = IIf(iYear <= 0, Year(Now), iYear)
   If GarrFiscalYear(0, 0) = vbNullString Or bRefresh Then
      Call EU_DOM_Lib.FillFiscalYearArray(iYear:=y, bTimesheet:=bTimesheet)
   End If
   ' Test if given range is big enough and is empty
   sMsg = vbNullString
   If aRng.Rows.Count <> 12 Then sMsg = "The OutputRange should cover 12 rows!."
   If aRng.Columns.Count <> 3 Then sMsg = "The OutputRange should cover 3 Columns!."
   If WorksheetFunction.CountA(aRng) <> 0 Then sMsg = "The OutputRange must be empty!."
   If sMsg <> vbNullString Then
      aRng.Worksheet.Activate
      aRng.Select
      MsgBox sMsg, vbCritical + vbOKOnly, cMe
      Exit Sub
   End If
   ' Display the results
   If GarrFiscalYear(0, 0) <> vbNullString Then
      For I = LBound(GarrFiscalYear) To UBound(GarrFiscalYear)
         If GarrFiscalYear(I, 0) = vbNullString Then Exit For
         C = aRng.Column
         R = aRng.Row
```

```
         aRng.Worksheet.Cells(R + I, C + 0).Value2 = "FY " & y & "-" & y + 1 & " P " & Right("0" & (I
+ 1), 2)
         aRng.Worksheet.Cells(R + I, C + 1).Value2 = GarrFiscalYear(I, 0)
         aRng.Worksheet.Cells(R + I, C + 2).Value2 = GarrFiscalYear(I, 1)
      Next I
      aRng.Worksheet.Cells(R, C).Select
   End If
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00          Date: 14-03-2021
' Action:  .
' Uses:    .
' Param:   .
' History: 13-03-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Private Function GetArrayDimension(ByRef aArray() As String) As Byte
   Dim I     As Integer
   Dim S     As String
   Dim iADim As Byte
   On Error Resume Next
   I = 0
   Do
      S = CStr(aArray(0, I))
      If Err.Number > 0 Then
         iADim = I
         On Error GoTo 0
         Exit Do
      Else
         I = I + 1
      End If
   Loop
   If iADim = 0 Then iADim = 1
   GetArrayDimension = iADim
End Function
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00          Date: 14-03-2021
' Action:  .
' Uses:    .
' Param:   .
' History: 13-03-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Private Sub FillHolidayArray(Optional ByVal iYear As Integer = 0, Optional sCountry As String = vbNull
String)
   Const cMe = "EU_DOM_Lib.FillHolidayArray"
   Dim y        As Integer
   Dim sCntry   As String
   Dim sLang    As String
   Dim dtEaster As Date
   Dim I        As Integer

   ' Determine for what year we need to calculate, and country & language
   y = IIf(iYear <= 0, Year(Now), iYear)
   sCntry = IIf(Len(sCountry) = 2, UCase(sCountry), EU_DOM_Lib.getCountry)
   sLang = EU_DOM_Lib.getLanguage
   dtEaster = EU_DOM_Lib.GetEasterSunday(iYear:=y)
   ' Initialise
   For I = LBound(GarrHolidays) To UBound(GarrHolidays)
      GarrHolidays(I, 0) = vbNullString
      GarrHolidays(I, 1) = vbNullString
   Next I
   ' Fill arrays
   If sCntry = "NL" Then
      GarrHolidays(0, 0) = Format(CDate(y & "-01-01"), "yyyy-mm-dd"): GarrHolidays(0, 1) = "Nieuwjaar"
          ' New year
      GarrHolidays(1, 0) = Format(CDate(y & "-04-27"), "yyyy-mm-dd"): GarrHolidays(1, 1) = "Koningsdag
"         ' Kings Day
      GarrHolidays(2, 0) = Format(CDate(y & "-12-25"), "yyyy-mm-dd"): GarrHolidays(2, 1) = "1e Kerstda
g"        ' 1 Christmas
      GarrHolidays(3, 0) = Format(CDate(y & "-12-26"), "yyyy-mm-dd"): GarrHolidays(3, 1) = "2e Kerstda
```

```
g"        ' 2 Christmas
      GarrHolidays(4, 0) = Format(dtEaster, "yyyy-mm-dd"): GarrHolidays(4, 1) = "1e Paasdag"
          ' 1 Eastern
      GarrHolidays(5, 0) = Format(dtEaster + 1, "yyyy-mm-dd"): GarrHolidays(5, 1) = "2e Paasdag"
          ' 2 Eastern
      GarrHolidays(6, 0) = Format(dtEaster + 39, "yyyy-mm-dd"): GarrHolidays(6, 1) = "Hemelvaart"
          ' Ascension-day
      GarrHolidays(7, 0) = Format(dtEaster + 49, "yyyy-mm-dd"): GarrHolidays(7, 1) = "1e Pinksterdag"
          ' 1 Pentecost
      GarrHolidays(8, 0) = Format(dtEaster + 50, "yyyy-mm-dd"): GarrHolidays(8, 1) = "2e Pinksterdag"
          ' 2 Pentecost
      ' -- Correct Kingsday. If this is on Sunday, it will be the day before
      If Weekday(CDate(y & "-04-27"), vbMonday) = 7 Then
         GarrHolidays(1, 0) = Format(CDate(y & "-04-26"), "yyyy-mm-dd")
      End If
      If (y Mod 5) = 0 Then
         GarrHolidays(9, 0) = Format(CDate(y & "-05-05"), "yyyy-mm-dd"): GarrHolidays(9, 1) = "Bevrijd
ingsdag"
      End If
   End If
   If sCntry = "BE" Then
      GarrHolidays(0, 0) = Format(CDate(y & "-01-01"), "yyyy-mm-dd"): GarrHolidays(0, 1) = "Nieuwjaar"
           ' New year
      GarrHolidays(1, 0) = Format(CDate(y & "-05-01"), "yyyy-mm-dd"): GarrHolidays(1, 1) = "Dag van de
 Arbeid"   ' Labourday
      GarrHolidays(2, 0) = Format(CDate(y & "-07-21"), "yyyy-mm-dd"): GarrHolidays(2, 1) = "Nationale
feestdag"  ' Nat.Holiday
      GarrHolidays(3, 0) = Format(CDate(y & "-08-15"), "yyyy-mm-dd"): GarrHolidays(3, 1) = "OLV Hemelv
aart"      ' Ascention
      GarrHolidays(4, 0) = Format(CDate(y & "-11-01"), "yyyy-mm-dd"): GarrHolidays(4, 1) = "Allerheili
gen"       ' All Hallows
      GarrHolidays(5, 0) = Format(CDate(y & "-11-11"), "yyyy-mm-dd"): GarrHolidays(5, 1) = "Wapenstils
tand 1918" ' Truce
      GarrHolidays(6, 0) = Format(CDate(y & "-12-25"), "yyyy-mm-dd"): GarrHolidays(6, 1) = "1e Kerstda
g"         ' 1 Christmas
      GarrHolidays(7, 0) = Format(dtEaster, "yyyy-mm-dd"): GarrHolidays(7, 1) = "1e Paasdag"
           ' 1 Eastern
      GarrHolidays(8, 0) = Format(dtEaster + 1, "yyyy-mm-dd"): GarrHolidays(8, 1) = "2e Paasdag"
           ' 2 Eastern
      GarrHolidays(9, 0) = Format(dtEaster + 39, "yyyy-mm-dd"): GarrHolidays(9, 1) = "Hemelvaart"
           ' Ascension-day
      GarrHolidays(10, 0) = Format(dtEaster + 49, "yyyy-mm-dd"): GarrHolidays(10, 1) = "1e Pinksterdag
"          ' 1 Pentecost
      GarrHolidays(11, 0) = Format(dtEaster + 50, "yyyy-mm-dd"): GarrHolidays(11, 1) = "2e Pinksterdag
"          ' 2 Pentecost
      If sLang = "fr-BE" Then
         GarrHolidays(0, 1) = "Nouvel an"      ' New year
         GarrHolidays(1, 1) = "Travail"        ' Labourday
         GarrHolidays(2, 1) = "Fête Nationale" ' Nat.Holiday
         GarrHolidays(3, 1) = "Assomption"     ' Ascention
         GarrHolidays(4, 1) = "Toussaint"      ' All Hallows
         GarrHolidays(5, 1) = "Armistice 1918" ' Truce
         GarrHolidays(6, 1) = "Noël"           ' 1 Christmas
         GarrHolidays(7, 1) = "Pàques 1"       ' 1 Eastern
         GarrHolidays(8, 1) = "Pàques 2"       ' 2 Eastern
         GarrHolidays(9, 1) = "Ascension"      ' Ascension-day
         GarrHolidays(10, 1) = "Pentecôte 1"   ' Pentecost 1
         GarrHolidays(11, 1) = "Pentecôte 2"   ' Pentecost 2
      End If
   End If
   If sCntry = "LU" Then
      GarrHolidays(0, 0) = Format(CDate(y & "-01-01"), "yyyy-mm-dd"): GarrHolidays(0, 1) = "Nouvel an"
           ' New year
      GarrHolidays(1, 0) = Format(CDate(y & "-05-01"), "yyyy-mm-dd"): GarrHolidays(1, 1) = "Travail"
           ' Labor-day
      GarrHolidays(2, 0) = Format(CDate(y & "-06-23"), "yyyy-mm-dd"): GarrHolidays(2, 1) = "Fête Natio
nale"      ' Grand Dukes Official Birthday
      GarrHolidays(3, 0) = Format(CDate(y & "-08-15"), "yyyy-mm-dd"): GarrHolidays(3, 1) = "Assomption
"          ' Maria Ascension-day
      GarrHolidays(4, 0) = Format(CDate(y & "-11-01"), "yyyy-mm-dd"): GarrHolidays(4, 1) = "Toussaint"
           ' All Hallows
      GarrHolidays(5, 0) = Format(CDate(y & "-12-25"), "yyyy-mm-dd"): GarrHolidays(5, 1) = "Noël"
           ' Christmas 1
```

```
      GarrHolidays(6, 0) = Format(CDate(y & "-12-26"), "yyyy-mm-dd"): GarrHolidays(6, 1) = "Saint-Etie
nne"        ' Christmas 2
      GarrHolidays(7, 0) = Format(dtEaster - 2, "yyyy-mm-dd"): GarrHolidays(7, 1) = "Vendredi Saint"
            ' Good Friday
      GarrHolidays(8, 0) = Format(dtEaster, "yyyy-mm-dd"): GarrHolidays(8, 1) = "Pàques 1"
            ' Eastern 1
      GarrHolidays(9, 0) = Format(dtEaster + 1, "yyyy-mm-dd"): GarrHolidays(9, 1) = "Pàques 2"
            ' Eastern 2
      GarrHolidays(10, 0) = Format(dtEaster + 39, "yyyy-mm-dd"): GarrHolidays(10, 1) = "Ascension"
            ' Ascension-day
      GarrHolidays(11, 0) = Format(dtEaster + 49, "yyyy-mm-dd"): GarrHolidays(11, 1) = "Pentecôte 1"
            ' Pentecost 1
      GarrHolidays(12, 0) = Format(dtEaster + 50, "yyyy-mm-dd"): GarrHolidays(12, 1) = "Pentecôte 2"
            ' Pentecost 2
   End If
   If sCntry = "DE" Then
      GarrHolidays(0, 0) = Format(CDate(y & "-01-01"), "yyyy-mm-dd"): GarrHolidays(0, 1) = "Neujahr"
            ' New year
      GarrHolidays(1, 0) = Format(CDate(y & "-05-01"), "yyyy-mm-dd"): GarrHolidays(1, 1) = "Erster Mai
"           ' Labor-day
      GarrHolidays(2, 0) = Format(CDate(y & "-10-03"), "yyyy-mm-dd"): GarrHolidays(2, 1) = "Deutschen
Einheit"    ' German Union
      GarrHolidays(3, 0) = Format(CDate(y & "-12-25"), "yyyy-mm-dd"): GarrHolidays(3, 1) = "1. Weihnac
htstag"     ' Christmas 1
      GarrHolidays(4, 0) = Format(CDate(y & "-12-26"), "yyyy-mm-dd"): GarrHolidays(4, 1) = "2. Weihnac
htstag"     ' Christmas 2
      GarrHolidays(5, 0) = Format(dtEaster - 2, "yyyy-mm-dd"): GarrHolidays(5, 1) = "Karfreitag"
            ' Good Friday
      GarrHolidays(6, 0) = Format(dtEaster, "yyyy-mm-dd"): GarrHolidays(6, 1) = "Ostersonntag"
            ' 1 Eastern
      GarrHolidays(7, 0) = Format(dtEaster + 1, "yyyy-mm-dd"): GarrHolidays(7, 1) = "Ostermontag"
            ' 2 Eastern
      GarrHolidays(8, 0) = Format(dtEaster + 39, "yyyy-mm-dd"): GarrHolidays(8, 1) = "Christi Himmelfa
hrt"        ' Ascension-day
      GarrHolidays(9, 0) = Format(dtEaster + 49, "yyyy-mm-dd"): GarrHolidays(9, 1) = "Pfingstsonntag"
            ' 1 Pentecost
      GarrHolidays(10, 0) = Format(dtEaster + 50, "yyyy-mm-dd"): GarrHolidays(10, 1) = "Pfingstmontag"
            ' 2 Pentecost
   End If
   If sCntry = "FR" Then
      GarrHolidays(0, 0) = Format(CDate(y & "-01-01"), "yyyy-mm-dd"): GarrHolidays(0, 1) = "Nouvel an"
           ' New year
      GarrHolidays(1, 0) = Format(CDate(y & "-05-01"), "yyyy-mm-dd"): GarrHolidays(1, 1) = "Travail"
           ' Labor-day
      GarrHolidays(2, 0) = Format(CDate(y & "-05-08"), "yyyy-mm-dd"): GarrHolidays(2, 1) = "Victoire"
           ' National holiday
      GarrHolidays(3, 0) = Format(CDate(y & "-07-14"), "yyyy-mm-dd"): GarrHolidays(3, 1) = "Fête Natio
nale"       ' National holiday
      GarrHolidays(4, 0) = Format(CDate(y & "-08-15"), "yyyy-mm-dd"): GarrHolidays(4, 1) = "Assomption
"           ' Maria Ascension-day
      GarrHolidays(5, 0) = Format(CDate(y & "-11-01"), "yyyy-mm-dd"): GarrHolidays(5, 1) = "Toussaint"
           ' All Hallows
      GarrHolidays(6, 0) = Format(CDate(y & "-11-11"), "yyyy-mm-dd"): GarrHolidays(6, 1) = "Armistice"
           ' Truce
      GarrHolidays(7, 0) = Format(CDate(y & "-12-25"), "yyyy-mm-dd"): GarrHolidays(7, 1) = "Noël"
           ' 1 Christmas
      GarrHolidays(8, 0) = Format(dtEaster, "yyyy-mm-dd"): GarrHolidays(8, 1) = "Pàques 1"
           ' 1 Eastern
      GarrHolidays(9, 0) = Format(dtEaster + 1, "yyyy-mm-dd"): GarrHolidays(9, 1) = "Pàques 2"
           ' 2 Eastern
      GarrHolidays(10, 0) = Format(dtEaster + 39, "yyyy-mm-dd"): GarrHolidays(10, 1) = "Ascension"
           ' Ascension-day
      GarrHolidays(11, 0) = Format(dtEaster + 49, "yyyy-mm-dd"): GarrHolidays(11, 1) = "Pentecôte 1"
           ' 1 Pentecost
      GarrHolidays(12, 0) = Format(dtEaster + 50, "yyyy-mm-dd"): GarrHolidays(12, 1) = "Pentecôte 2"
           ' 2 Pentecost
   End If
   If sCntry = "DK" Then
      GarrHolidays(0, 0) = Format(CDate(y & "-01-01"), "yyyy-mm-dd"): GarrHolidays(0, 1) = "Nytårsdag"
            ' New year
      GarrHolidays(1, 0) = Format(CDate(y & "-06-05"), "yyyy-mm-dd"): GarrHolidays(1, 1) = "Grundlovsd
ag"         ' Constitution Day (Denmark)
      GarrHolidays(2, 0) = Format(CDate(y & "-12-25"), "yyyy-mm-dd"): GarrHolidays(2, 1) = "Juledag"
```

```
            ' Christmas 1
      GarrHolidays(3, 0) = Format(CDate(y & "-12-26"), "yyyy-mm-dd"): GarrHolidays(3, 1) = "Anden jule
dag"        ' Christmas 2
      GarrHolidays(4, 0) = Format(dtEaster - 3, "yyyy-mm-dd"): GarrHolidays(4, 1) = "Skærtorsdag"
            ' Thursday before Eastern
      GarrHolidays(5, 0) = Format(dtEaster - 2, "yyyy-mm-dd"): GarrHolidays(5, 1) = "Langfredag"
            ' Good Friday
      GarrHolidays(6, 0) = Format(dtEaster, "yyyy-mm-dd"): GarrHolidays(6, 1) = "Påskedag"
            ' 1 Eastern
      GarrHolidays(7, 0) = Format(dtEaster + 1, "yyyy-mm-dd"): GarrHolidays(7, 1) = "Anden påskedag"
            ' 2 Eastern
      GarrHolidays(8, 0) = Format(dtEaster + 39, "yyyy-mm-dd"): GarrHolidays(8, 1) = "Himmelfartsdag"
            ' Ascension-day
      GarrHolidays(9, 0) = Format(dtEaster + 49, "yyyy-mm-dd"): GarrHolidays(9, 1) = "Pinsedag"
            ' 1 Pentecost
      GarrHolidays(10, 0) = Format(dtEaster + 50, "yyyy-mm-dd"): GarrHolidays(10, 1) = "Anden Pinsedag
"           ' 2 Pentecost
   End If
   If GarrHolidays(0, 0) = vbNullString Then
      MsgBox "No data for country '" & sCntry & "' present!!.", vbCritical + vbOKOnly, cMe
      Exit Sub
   End If
   Call EU_DOM_Lib.AlphabeticallySortArray(aArray:=GarrHolidays)
End Sub
' ===========================================================================

' ---------------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  .
' Uses:    .
' Param:   .
' History: 13-03-2019 Ronald Bax     First version
' ---------------------------------------------------------------------------
Private Sub FillFiscalYearArray(Optional ByVal iYear As Integer = 0, Optional ByVal bTimesheet As Bool
ean = False)
   Dim y           As Integer
   Dim I           As Integer
   Dim dtFirst     As Date
   Dim iCorrection As Integer

   ' Determine for what year we need to calculate, and country & language
   y = IIf(iYear <= 0, Year(Now), iYear)
   ' Initialise
   For I = LBound(GarrFiscalYear) To UBound(GarrFiscalYear)
      GarrFiscalYear(I, 0) = vbNullString
      GarrFiscalYear(I, 1) = vbNullString
   Next I
   ' Determine first day of the period
   ' Make sure it's a Monday and it starts in week 27
   dtFirst = CDate(y & "-07-01")
   If Weekday(dtFirst, vbMonday) <> 1 Then
      dtFirst = DateAdd("d", 1 - Weekday(dtFirst, vbMonday), dtFirst)
   End If
   If EU_DOM_Lib.GetWeek_ISO8601(dtDate:=dtFirst) > 27 Then dtFirst = DateAdd("d", -7, dtFirst)
   If EU_DOM_Lib.GetWeek_ISO8601(dtDate:=dtFirst) < 27 Then dtFirst = DateAdd("d", 7, dtFirst)
   ' Correct for years with 53 weeks ; Then P4 is 5 weeks in stead of 4 weeks
   iCorrection = IIf(EU_DOM_Lib.GetWeek_ISO8601(dtDate:=CDate(y & "-12-31")) = 53, 34, 27)
' =============
'   OCTOBER 2020 Finance watnt to have the extra week not in 2020-2021 but in 2021-2022
' =============
If iCorrection = 34 And y = 2020 Then iCorrection = 27
If iCorrection = 27 And y = 2021 Then
   iCorrection = 34
   dtFirst = DateAdd("d", -7, dtFirst)
End If
' =============
' Fill arrays
   If bTimesheet = True Then
      GarrFiscalYear(0, 0) = Format(dtFirst, "yyyy-mm-dd"): GarrFiscalYear(0, 1) = Format(CDate(GarrFi
scalYear(0, 0)) + 20, "yyyy-mm-dd") '  3 weeks
   Else
      GarrFiscalYear(0, 0) = Format(dtFirst, "yyyy-mm-dd"): GarrFiscalYear(0, 1) = Format(CDate(GarrFi
scalYear(0, 0)) + 27, "yyyy-mm-dd") '  4 weeks
```

```
   End If
   GarrFiscalYear(1, 0) = Format(CDate(GarrFiscalYear(0, 1)) + 1, "yyyy-mm-dd"): GarrFiscalYear(1, 1)
= Format(CDate(GarrFiscalYear(1, 0)) + 27, "yyyy-mm-dd")      '  4 weeks
   GarrFiscalYear(2, 0) = Format(CDate(GarrFiscalYear(1, 1)) + 1, "yyyy-mm-dd"): GarrFiscalYear(2, 1)
= Format(CDate(GarrFiscalYear(2, 0)) + 34, "yyyy-mm-dd")      '  5 weeks
   GarrFiscalYear(3, 0) = Format(CDate(GarrFiscalYear(2, 1)) + 1, "yyyy-mm-dd"): GarrFiscalYear(3, 1)
= Format(CDate(GarrFiscalYear(3, 0)) + iCorrection, "yyyy-mm-dd") '  4 weeks / 5 weeks
   GarrFiscalYear(4, 0) = Format(CDate(GarrFiscalYear(3, 1)) + 1, "yyyy-mm-dd"): GarrFiscalYear(4, 1)
= Format(CDate(GarrFiscalYear(4, 0)) + 27, "yyyy-mm-dd")      '  4 weeks
   GarrFiscalYear(5, 0) = Format(CDate(GarrFiscalYear(4, 1)) + 1, "yyyy-mm-dd"): GarrFiscalYear(5, 1)
= Format(CDate(GarrFiscalYear(5, 0)) + 34, "yyyy-mm-dd")      '  5 weeks
   GarrFiscalYear(6, 0) = Format(CDate(GarrFiscalYear(5, 1)) + 1, "yyyy-mm-dd"): GarrFiscalYear(6, 1)
= Format(CDate(GarrFiscalYear(6, 0)) + 27, "yyyy-mm-dd")      '  4 weeks
   GarrFiscalYear(7, 0) = Format(CDate(GarrFiscalYear(6, 1)) + 1, "yyyy-mm-dd"): GarrFiscalYear(7, 1)
= Format(CDate(GarrFiscalYear(7, 0)) + 27, "yyyy-mm-dd")      '  4 weeks
   GarrFiscalYear(8, 0) = Format(CDate(GarrFiscalYear(7, 1)) + 1, "yyyy-mm-dd"): GarrFiscalYear(8, 1)
= Format(CDate(GarrFiscalYear(8, 0)) + 34, "yyyy-mm-dd")      '  5 weeks
   GarrFiscalYear(9, 0) = Format(CDate(GarrFiscalYear(8, 1)) + 1, "yyyy-mm-dd"): GarrFiscalYear(9, 1)
= Format(CDate(GarrFiscalYear(9, 0)) + 27, "yyyy-mm-dd")   '  4 weeks
   GarrFiscalYear(10, 0) = Format(CDate(GarrFiscalYear(9, 1)) + 1, "yyyy-mm-dd"): GarrFiscalYear(10, 1
) = Format(CDate(GarrFiscalYear(10, 0)) + 27, "yyyy-mm-dd")  '  4 weeks
   If bTimesheet = True Then
      GarrFiscalYear(11, 0) = Format(CDate(GarrFiscalYear(10, 1)) + 1, "yyyy-mm-dd"): GarrFiscalYear(1
1, 1) = Format(CDate(GarrFiscalYear(11, 0)) + 41, "yyyy-mm-dd")  '  6 weeks
   Else
      GarrFiscalYear(11, 0) = Format(CDate(GarrFiscalYear(10, 1)) + 1, "yyyy-mm-dd"): GarrFiscalYear(1
1, 1) = Format(CDate(GarrFiscalYear(11, 0)) + 34, "yyyy-mm-dd")  '  5 weeks
   End If
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  Sort an array filled with strings alphabetically (A-Z)
'          SOURCE:  www.TheSpreadsheetGuru.com/the-code-vault
' Uses:    .
' Param:   .
' History: 13-03-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Private Sub AlphabeticallySortArray(ByRef aArray() As String)
   Const cMe = "EU_DOM_Lib.AlphabeticallySortArray"
   Dim x        As Long
   Dim y        As Long
   Dim TempTxt1 As String
   Dim TempTxt2 As String
   Dim iArrDim  As Byte
   iArrDim = EU_DOM_Lib.GetArrayDimension(aArray:=aArray)
   If iArrDim > 3 Then
      MsgBox "Only use sting-arras of 1 or 2 dimensions!", vbCritical + vbOKOnly, cMe
      Exit Sub
   End If
   'Alphabetize Sheet Names in Array List
   For x = LBound(aArray) To UBound(aArray)
      For y = x To UBound(aArray)
         If aArray(y, 0) <> vbNullString And aArray(x, 0) <> vbNullString Then
            If UCase(aArray(y, 0)) < UCase(aArray(x, 0)) Then
               TempTxt1 = aArray(x, 0)
               TempTxt2 = aArray(y, 0)
               aArray(x, 0) = TempTxt2
               aArray(y, 0) = TempTxt1
               If iArrDim = 2 Then
                  TempTxt1 = aArray(x, 1)
                  TempTxt2 = aArray(y, 1)
                  aArray(x, 1) = TempTxt2
                  aArray(y, 1) = TempTxt1
               End If
            End If
         End If
      Next y
   Next x
End Sub
' ====================================================================
```

```
' ====================================================================
' ====================================================================
```

```
' ================================================================
' Author:  Ronald Bax          Version: 3.00          Date: 14-03-2021
' Action:  Library to create and show SysInfo and Interface to the VBP Functions
' Uses:    EU_VBP_Lib, EU_WIN_Lib
' History: 18-02-2019 Ronald Bax     First version
' ================================================================
' Public  Function GetVersion() As String
' Public  Function GetDTUpdate() As Date
' Public  Function LibFileExists(sLibDir As String) As Boolean
' Public  Function GetLibFileDate(sLibDir As String) As Date
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Private Function AddSheetToWorkbook(aWB As Workbook, sSheetName As String) As String
' Private Function IsProjOfActiveWorkbook() As Boolean
' Private Function PrepSysInfoSheet() As Boolean
' Private Function GetVersionFromFile(sFilePath As String, aRng As Range) As Boolean
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Public  Sub      GetSysInfo()
' Public  Sub      MacroDeleteSysInfo()
' Public  Sub      DeleteSysInfo()
' ================================================================
Option Explicit

' ================================================================
'                         Constants and Types
' ================================================================
Private Const EU_SYS_Lib_LVersion As String = "3.00"
Private Const EU_SYS_Lib_DTUpdate As String = "12-07-2021"
Private Const EU_SYS_Lib_FileName As String = "EU_SYS_Lib.bas"
' ================================================================

' ----------------------------------------------------------------
' Author:  Ronald Bax          Version: 3.00          Date: 14-03-2021
' Action:  Get info about this library
' Uses:    EU_WIN_Lib.FileExists
'          EU_WIN_Lib.GetFileDateTime
' Param:   sLibDir
' History: 02-02-2016 Ronald Bax     First version
'          02-02-2019 Ronald Bax     2.00 Update
' ----------------------------------------------------------------
Public Function GetVersion() As String
   GetVersion = EU_SYS_Lib_LVersion
End Function
' ----------------------------------------------------------------
Public Function GetDTUpdate() As Date
   GetDTUpdate = CDate(EU_SYS_Lib_DTUpdate)
End Function
' ----------------------------------------------------------------
Public Function LibFileExists(sLibDir As String) As Boolean
   LibFileExists = EU_WIN_Lib.FileExists(sFilePath:=sLibDir & EU_SYS_Lib_FileName)
End Function
' ----------------------------------------------------------------
Public Function GetLibFileDate(sLibDir As String) As Date
   GetLibFileDate = EU_WIN_Lib.GetFileDateTime(sFilePath:=sLibDir & EU_SYS_Lib_FileName)
End Function
' ================================================================

' ================================================================
'                          PRIVATE FUNCTIONS
' ================================================================

' ----------------------------------------------------------------
' Author:  Ronald Bax          Version: 3.00          Date: 14-03-2021
' Action:  Returns the CodeName of the added sheet or an empty String if the workbook could not be ope
ned.
' Uses:    n.a.
' Param:   aWB        --
'          sSheetName --
' History: 02-02-2016 Ronald Bax     First version
'          02-02-2019 Ronald Bax     2.00 Update
' ----------------------------------------------------------------
Private Function AddSheetToWorkbook(aWb As Workbook, sSheetName As String) As String
   Dim wks As Worksheet
   AddSheetToWorkbook = vbNullString
```

```
   If Not aWb Is Nothing Then
      Set wks = aWb.Sheets.Add(After:=aWb.Sheets(aWb.Sheets.Count))
      wks.Name = sSheetName
      AddSheetToWorkbook = wks.CodeName ' ws.CodeName = sheetName: cannot assign to read only property
   End If
   Set wks = Nothing
End Function
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  Check if ActiveWorkbook is the one that has this ActiveVBProject
' Uses:    n.a.
' Param:   EU_WIN_Lib.Path2UNC
' History: 14-03-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Private Function IsProjOfActiveWorkbook() As Boolean
   Dim oProject As VBIDE.VBProject
   Set oProject = Application.VBE.ActiveVBProject
   On Error Resume Next ' This can throw if the workbook has never been saved.
   IsProjOfActiveWorkbook = (EU_WIN_Lib.Path2UNC(sFilePath:=oProject.Filename) = EU_WIN_Lib.Path2UNC(s
FilePath:=ActiveWorkbook.FullName))
   On Error GoTo 0
   Set oProject = Nothing
End Function
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  Format the layout of the SysInfo-sheet
' Uses:    EU_SYS_Lib.AddSheetToWorkbook
' Param:   n.a.
' History: 14-03-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Private Function PrepSysInfoSheet() As Boolean
   Const cMe = "EU_SYS_Lib.PrepSysInfoSheet"
   Dim aStr()    As String
   Dim oProject  As VBIDE.VBProject
   Dim WS        As Worksheet
   Dim wks       As Worksheet
   Dim sCodeName As String
   Dim sCompName As String
   Dim sFdt      As String
   Dim sLib      As String
   Dim sUser     As String

   PrepSysInfoSheet = False
   Set oProject = Application.VBE.ActiveVBProject
   Set WS = Nothing
   For Each wks In ActiveWorkbook.Worksheets
      If wks.Name = "SysInfo" And wks.CodeName = "wksSysInfo" Then Set WS = wks
   Next wks
   Set wks = Nothing

   If WS Is Nothing Then
      sCodeName = "wksSysInfo"
      sCompName = "SysInfo"
      sCompName = EU_SYS_Lib.AddSheetToWorkbook(aWb:=ActiveWorkbook, sSheetName:=sCompName)
      If sCompName = vbNullString Then
         MsgBox "Sheet '" & sCompName & "' could not be added.", vbCritical + vbOKOnly, cMe
         Exit Function
      End If
      oProject.VBComponents(sCompName).Name = sCodeName
   End If
   Set WS = ActiveWorkbook.Worksheets("SysInfo")
   Set oProject = Nothing

   With WS
      .Visible = xlSheetVisible
      .Activate
      .Unprotect
      Range(.Range("A10"), .Range("A10").SpecialCells(xlLastCell)).ClearContents
      ' Setup Row-labels
```

```
      ReDim aStr(8)  ' 0 - 8 = 9 entries
      aStr() = Split("Workbook|FilePath|FileDT|LibraryDir|User||||Modules", "|", -1, vbBinaryCompare)
      .Range("A1:A9").Value2 = Application.Transpose(aStr)
      ' Setup Column-labels
      ReDim aStr(12)  ' 12 - 8 = 13 entries
      aStr() = Split("Module|Sheetname|Hid.|Type|Library|#Lines|Intern.Ver|Intern.Date|#SrcLines|SrcVe
r|ScrDate|FileDate|Dif.", "|", -1, vbBinaryCompare)
      .Range("B9:N9").Value2 = aStr
      ' Merge cells
      .Range("B1:J1").MergeCells = False
      .Range("B1:J1").Merge
      .Range("B1:J1").MergeCells = True
      .Range("B2:J2").MergeCells = False
      .Range("B2:J2").Merge
      .Range("B2:J2").MergeCells = True
      .Range("B4:J4").MergeCells = False
      .Range("B4:J4").Merge
      .Range("B4:J4").MergeCells = True
      ' Fill-in the data
      sFdt = FileDateTime(ActiveWorkbook.FullName)
      sLib = EU_VBP_Lib.GetStandardLibraryPath
      sUser = Environ("USERNAME")
      ReDim aStr(12)  ' 12 - 8 = 13 entries
      aStr() = Split(ActiveWorkbook.Name & "|" & ActiveWorkbook.FullName & "|" & sFdt & "|" & sLib & "
|" & sUser, "|", -1, vbBinaryCompare)
      .Range("B1:B5").Value2 = Application.Transpose(aStr)
      ' Colour headers
      With .Range("B9:M9").Interior
         .Pattern = xlSolid
         .PatternColorIndex = xlAutomatic
         .ThemeColor = xlThemeColorAccent1
         .TintAndShade = 0.8
         .PatternTintAndShade = 0
      End With
      With .Range("J9:M9").Interior
         .Pattern = xlSolid
         .PatternColorIndex = xlAutomatic
         .ThemeColor = xlThemeColorAccent2
         .TintAndShade = 0.8
         .PatternTintAndShade = 0
      End With
      ' Correct alignment
      .Columns("B:F").HorizontalAlignment = xlLeft
      .Columns("D:D").HorizontalAlignment = xlCenter
      .Columns("G:G").HorizontalAlignment = xlRight
      .Columns("H:L").HorizontalAlignment = xlCenter
      .Columns("J:J").HorizontalAlignment = xlRight
      .Columns("M:M").HorizontalAlignment = xlRight
      .Columns("N:N").HorizontalAlignment = xlCenter
      .Columns("H:H").NumberFormat = "@"
      .Columns("K:K").NumberFormat = "@"
      .Range("B3").NumberFormat = "ddd dd/mm/yyyy"
      .Range("B2").HorizontalAlignment = xlLeft
      .Range("B4").HorizontalAlignment = xlLeft
      .Range("A10").Select
      ActiveWindow.FreezePanes = True
   End With
   Set WS = Nothing
   PrepSysInfoSheet = True
End Function
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  Get Version, Version-date and lines-count from a module-file
' Uses:    n.a.
' Param:   sFilePath -- file to analyse
'          aRng      -- where to show the results
' History: 14-03-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Private Function GetVersionFromFile(sFilePath As String, aRng As Range) As Boolean
   Dim sResVer As String
   Dim sResDt  As String
```

```
   Dim sLine   As String
   Dim S       As String
   Dim I       As Long
   Dim iCnt    As Long

   iCnt = 0
   sResVer = vbNullString
   sResDt = vbNullString
   On Error GoTo ExitErr
   Open sFilePath For Input As #1
   Do Until EOF(1)
      Line Input #1, sLine
      I = InStr(1, sLine, "Version:", vbTextCompare)
      If (sResVer = vbNullString) And (I > 0) Then
         S = LTrim(RTrim(Mid(sLine, I + 8, 6)))
         sResVer = S
         I = InStr(1, sLine, "Date:", vbTextCompare)
         If I > 0 Then
            S = LTrim(RTrim(Mid(sLine, I + 5, 11)))
            sResDt = S
         End If
      End If
      iCnt = iCnt + 1
      If InStr(1, sLine, "Attribute ", vbTextCompare) > 0 And _
         ((InStr(1, sLine, ".VB_Description ", vbTextCompare) > 0) Or (InStr(1, sLine, ".VB_ProcData."
, vbTextCompare) > 0)) Then
         iCnt = iCnt - 1
      End If
   Loop
   Close #1
   If UCase(Right(sFilePath, 4)) = ".BAS" Then iCnt = iCnt - 1
   aRng.Offset(0, 0).Value2 = iCnt
   aRng.Offset(0, 1).Value2 = sResVer
   aRng.Offset(0, 2).Value2 = CDate(sResDt)
ExitErr:
   GetVersionFromFile = (iCnt > 0)
End Function
' ===============================================================================

' ===============================================================================
'                            PUBLIC FUNCTIONS
' ===============================================================================

' -------------------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  Create SysInfo-sheet if not present, and show all info
' Uses:    EU_VBP_Lib.GetStandardLibraryPath
'          EU_VBP_Lib.GetSourceDir
'          EU_SYS_Lib.GetVersionFromFile
'          EU_XLS_Lib.DrawBorders
'          EU_XLS_Lib.HeaderFooterAddStandardWks
'          EU_XLS_Lib.PageInLandscapeWks
'          EU_XLS_Lib.AutoFitWorksheet
' Param:   bShow -- Not only update the sheet, bur also show it
' History: 14-03-2019 Ronald Bax     First version
' -------------------------------------------------------------------------------
Public Sub GetSysInfo(Optional ByVal bShow As Boolean = True)
   Const cMe = "EU_VBP_Lib.GetSysInfo"
   Const cWksName = "SysInfo"
   Dim oComp    As VBComponent
   Dim OldWS    As Worksheet
   Dim wks      As Worksheet
   Dim rngData  As Range
   Dim btn      As Shape
   Dim iLine    As Long
   Dim iRow     As Long
   Dim I        As Integer
   Dim sLine    As String
   Dim sFile    As String
   Dim sName    As String
   Dim S        As String
   Dim bLib     As Boolean
   Dim bPresent As Boolean
```

```
   ' Check if ActiveWorkbook is the one that has this ActiveVBProject
   If Not EU_SYS_Lib.IsProjOfActiveWorkbook() Then
      MsgBox "The VBProject does not belong to the ActiveWorkbook.", vbCritical + vbOKOnly, cMe
      Exit Sub
   End If
   ' Remember oldWS
   Set OldWS = Application.ActiveSheet
   If Not EU_SYS_Lib.PrepSysInfoSheet() Then
      MsgBox "Could not create/find worksheet '" & cWksName & "'.", vbCritical + vbOKOnly, cMe
      Set OldWS = Nothing
      Exit Sub
   End If
   ' Fill-in the details for each module
   Application.ScreenUpdating = False
   With ActiveWorkbook.Worksheets(cWksName)
      .Visible = xlSheetVisible
      .Activate
      .Unprotect
      ' Module details
      iRow = 10
      With Application.VBE.ActiveVBProject
         For Each oComp In .VBComponents
            With ActiveWorkbook.Worksheets(cWksName)
               bLib = False
               .Range("B" & iRow).Value2 = oComp.Name
               If oComp.Type = vbext_ct_Document Then
                  For Each wks In Application.Worksheets
                     If UCase(wks.CodeName) = UCase(oComp.Name) Then
                        .Range("C" & iRow).Value2 = wks.Name
                        .Range("D" & iRow).Value2 = IIf(wks.Visible, vbNullString, "V")
                        Exit For
                     End If
                  Next wks
               End If
               Select Case oComp.Type
                  Case vbext_ct_ClassModule: S = "ClassModule"
                  Case vbext_ct_StdModule: S = "Module"
                  Case vbext_ct_MSForm: S = "Form"
                  Case vbext_ct_Document: S = "Worksheet"
                  Case Else: S = "*unknown*"
               End Select
               If oComp.Type = vbext_ct_Document And oComp.Name = "ThisWorkbook" Then S = "ThisWorkboo
k"
               .Range("E" & iRow).Value2 = S

               If oComp.Type = vbext_ct_StdModule Then
                  bLib = (UCase(Left(oComp.Name, 3)) = "EU_" And UCase(Right(oComp.Name, 4)) = UCase("
_Lib"))
                  If bLib Then
                     .Range("F" & iRow).Value2 = "EU Library"
                  End If
               End If
               If oComp.CodeModule.CountOfLines > 0 Then .Range("G" & iRow).Value2 = oComp.CodeModule.
CountOfLines
               For iLine = oComp.CodeModule.CountOfLines To 1 Step -1
                  sLine = RTrim(oComp.CodeModule.Lines(iLine, 1))
                  I = InStr(1, sLine, oComp.Name & "_LVersion As String", vbTextCompare)
                  If I > 0 Then
                     sLine = Mid(sLine, I)
                     I = InStr(1, sLine, """", vbTextCompare)
                     .Range("H" & iRow).Value2 = Replace(Mid(sLine, I), """", "")
                  Else
                     I = InStr(1, sLine, "Version:", vbTextCompare)
                     If I > 0 Then
                        S = LTrim(RTrim(Mid(sLine, I + 8, 6)))
                        .Range("H" & iRow).Value2 = S
                        I = InStr(1, sLine, "Date:", vbTextCompare)
                        If I > 0 Then
                           S = LTrim(RTrim(Mid(sLine, I + 5, 11)))
                           .Range("I" & iRow).Value2 = CDate(S)
                        End If
                     End If
```

```
                End If
            Next iLine
            sFile = vbNullString
            If bLib Then
               sName = EU_VBP_Lib.GetStandardLibraryPath & "\" & oComp.Name
            Else
               sName = EU_VBP_Lib.GetSourceDir(bForceCreate:=True) & oComp.Name
            End If
            S = sName & ".sheet.cls"
            If EU_WIN_Lib.FileExists(sFilePath:=S) Then sFile = S
            S = sName & ".cls"
            If EU_WIN_Lib.FileExists(sFilePath:=S) Then sFile = S
            S = sName & ".bas"
            If EU_WIN_Lib.FileExists(sFilePath:=S) Then sFile = S
            S = sName & ".frm"
            If EU_WIN_Lib.FileExists(sFilePath:=S) Then sFile = S
            If sFile <> vbNullString Then
               Call EU_SYS_Lib.GetVersionFromFile(sFilePath:=sFile, aRng:=.Range("J" & iRow))
               .Range("M" & iRow).Value = EU_WIN_Lib.GetFileDateTime(sFilePath:=sFile) ' must be Va
lue
            End If
            iRow = iRow + 1
         End With
      Next oComp
   End With

   ' Sort the results
   iRow = .Range("A10").SpecialCells(xlLastCell).Row
   Set rngData = Range(.Range("B10"), .Range("A10").SpecialCells(xlLastCell))
   .Sort.SortFields.Clear
   .Sort.SortFields.Add2 key:=Range("F10:F" & iRow), SortOn:=xlSortOnValues, Order:=xlAscending, Da
taOption:=xlSortNormal
   .Sort.SortFields.Add2 key:=Range("E10:E" & iRow), SortOn:=xlSortOnValues, Order:=xlAscending, Da
taOption:=xlSortNormal
   .Sort.SortFields.Add2 key:=Range("G10:G" & iRow), SortOn:=xlSortOnValues, Order:=xlDescending, D
ataOption:=xlSortNormal
   .Sort.SortFields.Add2 key:=Range("B10:B" & iRow), SortOn:=xlSortOnValues, Order:=xlAscending, Da
taOption:=xlSortNormal
   With .Sort
      .SetRange rngData
      .Header = xlGuess
      .MatchCase = False
      .Orientation = xlTopToBottom
      .SortMethod = xlPinYin
      .Apply
   End With
   ' Create 4 buttons
   bPresent = False
   For Each btn In .Shapes
      If btn.Name = "btnSysInfo" Then bPresent = True
   Next btn
   If Not bPresent Then
      With .Buttons.Add(.Range("B6").Left + 3, .Range("B6").Top + 3, 80, 22)
         .OnAction = "EU_ALL_Lib.SysInfo"
         .Characters.Text = "Show SysInfo"
         .Name = "btnSysInfo"
      End With
      With .Buttons.Add(.Range("C6").Left + 3, .Range("C6").Top + 3, 80, 22)
         .OnAction = "EU_ALL_Lib.HideSysInfo"
         .Characters.Text = "Hide SysInfo"
         .Name = "btnDelSysInfo"
      End With
      With .Buttons.Add(.Range("E6").Left + 3, .Range("E6").Top + 3, 80, 22)
         .OnAction = "EU_ALL_Lib.DeleteSysInfo"
         .Characters.Text = "Delete SysInfo"
         .Name = "btnHideSysInfo"
      End With
      With .Buttons.Add(.Range("L2").Left + 5, .Range("L2").Top + 3, 80, 22)
         .OnAction = "EU_ALL_Lib.Update_Libraries"
         .Characters.Text = "Reload Libs"
         .Name = "btnUpdLibs"
      End With
      With .Buttons.Add(.Range("L4").Left + 5, .Range("L4").Top + 3, 80, 22)
```

```
            .OnAction = "EU_ALL_Lib.Export_All_Code"
            .Characters.Text = "Export all code"
            .Name = "btnExpAll"
         End With
         With .Buttons.Add(.Range("L6").Left + 5, .Range("L6").Top + 3, 80, 22)
            .OnAction = "EU_ALL_Lib.Import_All_Code"
            .Characters.Text = "Import all code"
            .Name = "btnImpAll"
         End With
      End If
      Set btn = Nothing
      ' Create Sum-formula
      iRow = .Range("A10").SpecialCells(xlLastCell).Row - 8
      .Range("G8").FormulaR1C1 = "=SUM(R[2]C:R[" & iRow & "]C)"
      .Range("J8").FormulaR1C1 = "=SUM(R[2]C:R[" & iRow & "]C)"
      ' Create formula in "N" column
      .Range("N10").FormulaR1C1 = "=IF(CONCATENATE(RC[-7],RC[-6],RC[-5])=CONCATENATE(RC[-4],RC[-3],RC[
-2]),""""",""X"")"
      .Range("N10").AutoFill Destination:=Range("N10:N" & .Range("N10").SpecialCells(xlLastCell).Row),
 Type:=xlFillDefault
      .Range("A1").Select ' Focus on top
      ActiveWindow.ScrollColumn = 1
      ActiveWindow.ScrollRow = 1
      Call EU_XLS_Lib.DrawBorders(aRng:=Range(.Range("B9"), .Range("M" & .Range("M9").SpecialCells(xlL
astCell).Row)), bOnOff:=True)
      Call EU_XLS_Lib.HeaderFooterAddStandardWks(aWks:=Worksheets(cWksName))
      Call EU_XLS_Lib.PageInLandscapeWks(aWks:=Worksheets(cWksName))
      Call EU_XLS_Lib.AutoFitWorksheet(aWks:=Worksheets(cWksName))
      If .Range("B:B").ColumnWidth < 17 Then .Range("B:B").ColumnWidth = 17
      If .Range("C:C").ColumnWidth < 12 Then .Range("C:C").ColumnWidth = 12
      If .Range("E:E").ColumnWidth < 17 Then .Range("E:E").ColumnWidth = 17
      .Buttons("btnSysInfo").Width = 80
      .Buttons("btnUpdLibs").Width = 80
      .Buttons("btnExpAll").Width = 80
      .Buttons("btnImpAll").Width = 80
      .Buttons("btnDelSysInfo").Width = 80
      .Buttons("btnHideSysInfo").Width = 80
      For I = .Range("M9").SpecialCells(xlLastCell).Row To 10 Step -1
         If .Range("B" & I).Value2 = vbNullString Then .Range("B" & I).Rows.EntireRow.Delete
      Next I
      .Range("I:I,L:L,M:M").NumberFormat = "dd/mm/yyyy"
      .Protect DrawingObjects:=True, Contents:=True, Scenarios:=True, AllowFiltering:=True, AllowUsing
PivotTables:=True
      If Not bShow Then
         OldWS.Activate
         .Visible = xlSheetHidden
      Else
         .Visible = xlSheetVisible
         Worksheets(cWksName).Activate
      End If
   End With
   Application.ScreenUpdating = True
   Set rngData = Nothing
   Set wks = Nothing
   Set oComp = Nothing
   Set OldWS = Nothing
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  Delete SysInfo-sheet
' Uses:    n.a.
' Param:   n.a.
' History: 14-03-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Public Sub MacroDeleteSysInfo()
   Application.DisplayAlerts = False
   On Error Resume Next
   ThisWorkbook.Sheets("SysInfo").Delete
   On Error GoTo 0
   Application.DisplayAlerts = True
End Sub
```

```
' ========================================================================


' ========================================================================
' ========================================================================
```

EU_VBP_Lib - 1

```
' ====================================================================
' Author:  Ronald Bax          Version: 3.00          Date: 08-11-2022
' Action:  Library with functions to update (import/export) VB modules and EU-Libraries
' Uses:    n.a.
' History: 21-10-2016 Ronald Bax     First version
'          25-10-2016 Ronald Bax     1.10 Update libraries added
'          26-10-2016 Ronald Bax     1.12 BeforeSave and AfterSave fixed
'          02-02-2019 Ronald Bax     2.00 Update
'          02-02-2019 Ronald Bax     2.00 Update
'          19-03-2020 Ronald Bax     2.40 GetModuleInfoFromWB updated
' ====================================================================
' Note:    This Lib is GENERIC and INDEPENDENT of any other libs/sheets
'          Sourcecode based in part on vbaDeveloper-master (https://github.com/hilkoc/vbaDeveloper)
' ====================================================================
' Public  Function GetVersion() As String
' Public  Function GetDTUpdate() As Date
' Public  Function LibFileExists(sLibDir As String) As Boolean
' Public  Function GetLibFileDate(sLibDir As String) As Date
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Private Sub      ExportComponent(sExportDir As String, aComp As VBComponent, Optional sExtension As
String = ".cls")
' Private Sub      ExportLines(sExportDir As String, aComp As VBComponent)
' Private Sub      CreateCompareBacth(sFilePath As String, ByVal bShow As Boolean)
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Private Function ComponentExists(aVBAProject As VBIDE.VBProject, ByVal sComponentName As String) As
Boolean
' Private Function HasCodeToExport(aComp As VBComponent) As Boolean
' Private Function AddModuleToWorkbook(aVBAProject As VBIDE.VBProject, sCompName As String) As String
' Private Function AddSheetToWorkbook(aWB As Workbook, sSheetName As String) As String
' Private Sub      Import_Code_From_File(sFilePath As String)
' Private Sub      ImportLinesFromFile(ByVal sCompNameAndImportFile As String)
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Private Function GetFileDateTime(sFilePath As String) As Date
' Private Function DirExists(sDirPath As String, Optional ByVal bForceCreate As Boolean = False) As Bo
olean
' Private Function FileExists(sFilePath As String) As Boolean
' Private Function ListfilesInDir(sDirPath As String, Optional sMask As String = "*.*", Optional sUnMa
sk As String = vbNullString ) As Variant
' Private Function Path2UNC(sFilePath As String) As String
' Private Sub      Pause(ByVal iSeconds As Single)
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Public  Function GetSourceDir(Optional ByVal bForceCreate As Boolean = False) As String
' Public  Function GetStandardLibraryPath() As String
' Public  Sub      CleanAllModules(Optional sAction As String = vbNullString )
' Public  Sub      UpdateEULib(sLibPath As String)
' Public  Sub      UpdateAllEULibs()
' Public  Sub      Export_All_Code()
' Public  Sub      Import_All_Code(Optional ByVal bIncLibs As Boolean = False)
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Private Sub      GetFullProcDeclarations()
' Public  Sub      ClearGlobalCollections()
' Public  Sub      GetModuleInfoFromWB(aWB As Workbook, sPW As String)
' ====================================================================
Option Explicit

' ====================================================================
'                          Constants and Types
' ====================================================================
Private Const EU_VBP_Lib_LVersion As String = "2.50"
Private Const EU_VBP_Lib_DTUpdate As String = "19-03-2020"
Private Const EU_VBP_Lib_FileName As String = "EU_VBP_Lib.bas"

Private Const LibPath1 = "K:\Technology\50 - DEVELOPMENT\50 EXCEL VBA\02-ProdLibs\"
Private Const LibPath2 = "V:\Technology\50 - DEVELOPMENT\50 EXCEL VBA\02-ProdLibs\"

Private Const GIMPORT_DELAY As String = "00:00:03"    ' Delay for application.ontime()
Private Const GCONTINUE_DELAY As String = "00:00:03"  ' Delay for EU-libs application.ontime()
Private Const GTIMEOUT As Single = 5                  ' Try for 5 seconds to see if Module is gone or
has reappeared

Public GCollModules    As New Collection  ' Collection of all modules
Public GCollProcedures As New Collection  ' Collection of all Procedures
Public GCollSourceCode As New Collection  ' Collection of all Sourcecode-lines
```

```
Public GCollProcDecl   As New Collection  ' Collection of all Procedures
' ================================================================

' ----------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 08-11-2022
' Action:  Get info about this library
' Uses:    EU_VBP_Lib.FileExists       --  Copy of the same func in EU_WIN_Lib
'          EU_VBP_Lib.GetFileDateTime  --  Copy of the same func in EU_WIN_Lib
' Param:   sLibDir
' History: 02-02-2016 Ronald Bax     First version
'          02-02-2019 Ronald Bax     2.00 Update
' ----------------------------------------------------------------
Public Function GetVersion() As String
   GetVersion = EU_VBP_Lib_LVersion
End Function
' ----------------------------------------------------------------
Public Function GetDTUpdate() As Date
   GetDTUpdate = CDate(EU_VBP_Lib_DTUpdate)
End Function
' ----------------------------------------------------------------
Public Function LibFileExists(sLibDir As String) As Boolean
   LibFileExists = EU_VBP_Lib.FileExists(sFilePath:=sLibDir & EU_VBP_Lib_FileName)
End Function
' ----------------------------------------------------------------
Public Function GetLibFileDate(sLibDir As String) As Date
   GetLibFileDate = EU_VBP_Lib.GetFileDateTime(sFilePath:=sLibDir & EU_VBP_Lib_FileName)
End Function
' ================================================================

' ================================================================
'                           PRIVATE FUNCTIONS
' ================================================================

' ----------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 08-11-2022
' Action:  To export everything else but sheets
' Uses:    n.a.
' Param:   sExportDir --
'          aComp      --
'          sExtension --
' History: 02-02-2019 Ronald Bax     First version
' ----------------------------------------------------------------
Private Sub ExportComponent(sExportDir As String, aComp As VBComponent, Optional sExtension As String
= ".cls")
    aComp.Export sExportDir & aComp.Name & sExtension
End Sub
' ================================================================

' ----------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 08-11-2022
' Action:  To export sheets
' Uses:    n.a.
' Param:   ...
' History: 02-02-2019 Ronald Bax     First version
' ----------------------------------------------------------------
Private Sub ExportLines(sExportDir As String, aComp As VBComponent)
   Dim sFileName  As String
   Dim oFSO       As Object
   Dim oOutStream As Object
   sFileName = sExportDir & aComp.Name & ".sheet.cls"
   Set oFSO = CreateObject("Scripting.FileSystemObject")
   Set oOutStream = oFSO.CreateTextFile(sFileName, True, False)
   oOutStream.Write (aComp.CodeModule.Lines(1, aComp.CodeModule.CountOfLines))
   oOutStream.Close
   Set oOutStream = Nothing
   Set oFSO = Nothing
End Sub
' ================================================================

' ----------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 08-11-2022
' Action:  Create a batch-file to compare the EU_ libs with the Actual version
' Uses:    n.a.
```

```
' Param:   sFilePath -- Name of batch-file to create
'          bShow     -- If true, open Notepad or notepad++ to show the result
' History: 02-02-2016 Ronald Bax     First version
'          02-02-2019 Ronald Bax     2.00 Update
' ------------------------------------------------------------------------
Private Sub CreateCompareBacth(sFilePath As String, ByVal bShow As Boolean)
   Dim S(1 To 50) As String
   Dim I          As Integer
   S(1) = "@ECHO OFF"
   S(2) = "   "
   S(3) = "   %~d0"
   S(4) = "   CD %~dp0"
   S(5) = "   "
   S(6) = "   IF !%1!==!! GOTO Start"
   S(7) = "   SET SD=\Technology\50 - DEVELOPMENT\50 EXCEL VBA\02-ProdLibs"
   S(8) = "   IF EXIST ""%SD%\EU*.*"" GOTO EachFile"
   S(9) = "   SET SD=E:\Data\Data\20 - DEVELOP\50 EXCEL VBA\02-ProdLibs"
   S(10) = "   IF EXIST ""%SD%\EU*.*"" GOTO EachFile"
   S(11) = "   ECHO."
   S(12) = "   ECHO Directory not found: %SD%"
   S(13) = "   ECHO."
   S(14) = "   PAUSE"
   S(15) = "   GOTO Uit"
   S(16) = "   "
   S(17) = ":Start"
   S(18) = "   ECHO OFF>CompareLibs.out"
   S(19) = "   ECHO ---------------------------------------------------- >>CompareLibs.out"
   S(20) = "   ECHO Comparing Project Lib-Files with Standard Lib-files. >>CompareLibs.out"
   S(21) = "   ECHO ProjectDir: %~dp0 >>CompareLibs.out"
   S(22) = "   ECHO ---------------------------------------------------- >>CompareLibs.out"
   S(23) = "   ECHO. >>CompareLibs.out"
   S(24) = "   ECHO. >>CompareLibs.out"
   S(25) = "   FOR %%G IN (""EU*.*"") DO CALL %0 %%G"
   S(26) = "   ECHO. >>CompareLibs.out"
   S(27) = "   ECHO ---------------------------------------------------- >>CompareLibs.out"
   If bShow Then
      S(28) = "   IF EXIST ""C:\Program Files (x86)\Notepad++\notepad++.exe"" GOTO NOTEPAD"
      S(29) = "   IF EXIST ""C:\Program Files\Notepad++\notepad++.exe"" GOTO NOTEPAD"
      S(30) = "   START NOTEPAD CompareLibs.out"
      S(31) = "   GOTO Uit"
      S(32) = "   "
      S(33) = ":NOTEPAD"
      S(34) = "   IF EXIST ""C:\Program Files (x86)\Notepad++\notepad++.exe"" START ""C:\Program Files
\Notepad++\notepad++.exe"" CompareLibs.out"
      S(35) = "   IF EXIST ""C:\Program Files\Notepad++\notepad++.exe"" START ""C:\Program Files\Notep
ad++\notepad++.exe"" CompareLibs.out"
   End If
   S(36) = "   GOTO Uit"
   S(37) = "   "
   S(38) = ":EachFile"
   S(39) = "   ECHO."
   S(40) = "   ECHO Process:  %1"
   S(41) = "   ECHO."
   S(42) = "   FC /C ""%SD%\%1"" %1 >>CompareLibs.out"
   S(43) = "   GOTO Uit"
   S(44) = "   "
   S(45) = ":Uit"
   Open sFilePath For Output As #1
   For I = 1 To 50
      If S(I) <> vbNullString Then Print #1, S(I)
   Next I
   Close #1
End Sub
' ========================================================================

' ------------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 08-11-2022
' Action:  Check if a given Component exists
' Uses:    n.a.
' Param:   aVBAProject     --
'          sComponentName --
' History: 02-02-2016 Ronald Bax     First version
'          02-02-2019 Ronald Bax     2.00 Update
```

```
' ---------------------------------------------------------------------
Private Function ComponentExists(aVBAProject As VBIDE.VBProject, sComponentName As String) As Boolean
   Dim C As VBComponent
   ComponentExists = False
   On Error GoTo Err
   Set C = aVBAProject.VBComponents(sComponentName)
   ComponentExists = True
Err:
   Set C = Nothing
End Function
' =====================================================================

' ---------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00          Date: 08-11-2022
' Action:  Check if component has anything to export
' Uses:    n.a.
' Param:   aComp --
' History: 02-02-2019 Ronald Bax     First version
' ---------------------------------------------------------------------
Private Function HasCodeToExport(aComp As VBComponent) As Boolean
   Dim sFirstLine As String
   HasCodeToExport = True
   If aComp.CodeModule.CountOfLines <= 2 Then
      sFirstLine = Trim(aComp.CodeModule.Lines(1, 1))
      HasCodeToExport = Not (sFirstLine = vbNullString Or sFirstLine = "Option Explicit")
   End If
End Function
' =====================================================================

' ---------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00          Date: 08-11-2022
' Action:  Add a new module to the workbook
' Uses:    n.a.
' Param:   aVBAProject  --
'          sCompName --
' History: 02-02-2016 Ronald Bax     First version
'          02-02-2019 Ronald Bax     2.00 Update
' ---------------------------------------------------------------------
Private Function AddModuleToWorkbook(aVBAProject As VBIDE.VBProject, sCompName As String) As String
   Dim aModule As VBComponent
   On Error Resume Next
   Set aModule = aVBAProject.VBComponents.Add(vbext_ct_StdModule)
   aModule.Name = sCompName
   On Error GoTo 0
   AddModuleToWorkbook = vbNullString
   If EU_VBP_Lib.ComponentExists(aVBAProject:=aVBAProject, sComponentName:=sCompName) Then
      AddModuleToWorkbook = sCompName
   End If
   Set aModule = Nothing
End Function
' =====================================================================

' ---------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00          Date: 08-11-2022
' Action:  Returns the CodeName of the added sheet or an empty String if the workbook could not be ope
ned.
' Uses:    n.a.
' Param:   aWB        --
'          sSheetName --
' History: 02-02-2016 Ronald Bax     First version
'          02-02-2019 Ronald Bax     2.00 Update
' ---------------------------------------------------------------------
Private Function AddSheetToWorkbook(aWb As Workbook, sSheetName As String) As String
   Dim WS As Worksheet
   AddSheetToWorkbook = vbNullString
   If Not aWb Is Nothing Then
      Set WS = aWb.Sheets.Add(After:=aWb.Sheets(aWb.Sheets.Count))
      WS.Name = sSheetName
      ' ws.CodeName = sheetName: cannot assign to read only property
      AddSheetToWorkbook = WS.CodeName
   End If
   Set WS = Nothing
End Function
```

```
' ====================================================================
' --------------------------------------------------------------------
' Author:  Ronald Bax          Version: 3.00          Date: 08-11-2022
' Action:  Import code from Module-file
' Uses:    EU_VBP_Lib.Path2UNC         --  Copy of the same func in EU_WIN_Lib
' Param:   sFilePath --
' History: 02-02-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Private Sub Import_Code_From_File(sFilePath As String)
   Const cMe = "EU_VBP_Lib.Import_Code_From_File"
   Dim oProject       As VBIDE.VBProject
   Dim oComp          As VBComponent
   Dim sProjFileName  As String
   Dim sProjFileDir   As String
   Dim sActWBFileName As String
   Dim sImportDir     As String
   Dim sImportFile    As String
   Dim sImportExt     As String
   Dim sCompName      As String
   Dim sCodeName      As String
   Dim sCmd           As String
   Dim I              As Integer
   ' Check if ActiveWorkbook is the one that has this ActiveVBProject
   Set oProject = Application.VBE.ActiveVBProject
   On Error Resume Next
   ' This can throw if the workbook has never been saved.
   sProjFileName = EU_VBP_Lib.Path2UNC(sFilePath:=oProject.Filename)
   On Error GoTo 0
   sActWBFileName = EU_VBP_Lib.Path2UNC(sFilePath:=ActiveWorkbook.FullName)
   If sProjFileName <> sActWBFileName Then
      MsgBox "The VBProject does not belong to the ActiveWorkbook.", vbCritical + vbOKOnly, cMe
      Exit Sub
   End If
   ' Test if this is a new workbook. In that case, skip it
   If sProjFileName = vbNullString Then
      MsgBox "No file name for project '" & oProject.Name & "'. Import aborted.", vbCritical + vbOKOnl
y, cMe
      Exit Sub
   End If
   ' Determine the directory of the sProjFileDir
   I = InStrRev(sProjFileName, Application.PathSeparator, , vbTextCompare)
   sProjFileDir = IIf(I > 0, Mid(sProjFileName, 1, I), vbNullString)
   ' Get the Import-Dir and test if the ImportDir actually exists
   I = InStrRev(sFilePath, Application.PathSeparator, , vbTextCompare)
   If I <> 0 Then
      ' If sFilePath is only the filename, then add the directory of the sProjFileName
      sImportDir = EU_VBP_Lib.Path2UNC(sFilePath:=Left(sFilePath, I))
   Else
      sImportDir = EU_VBP_Lib.Path2UNC(sFilePath:=sProjFileDir & "Source\")
   End If
   If Not EU_VBP_Lib.DirExists(sDirPath:=sImportDir, bForceCreate:=False) Then
      MsgBox "Directory '" & sImportDir & "' does not exist.", vbCritical + vbOKOnly, cMe
      Exit Sub
   End If
   ' Get the FileName and ModuleName from the FilePath
   I = InStrRev(sFilePath, Application.PathSeparator, , vbTextCompare)
   If I > 0 Then sImportFile = Mid(sFilePath, I + 1, Len(sFilePath) - I) Else sImportFile = sFilePath
   sCompName = Left(sImportFile, InStr(sImportFile, ".") - 1)
   sImportExt = Replace(sImportFile, sCompName, vbNullString)
   sImportFile = sImportDir & sImportFile
   ' Test if the import-file actualy exists
   If Not EU_VBP_Lib.FileExists(sFilePath:=sImportFile) Then
      MsgBox "File '" & sImportFile & "' does not exist.", vbCritical + vbOKOnly, cMe
      Exit Sub
   End If
   If Not EU_VBP_Lib.ComponentExists(aVBAProject:=oProject, sComponentName:=sCompName) Then
      If sImportExt = ".sheet.cls" Then
         ' Create a sheet to import this code into. We cannot set the ws.codeName property which is re
ad-only,
         ' instead we set its vbComponent.name which leads to the same  result.
         sCodeName = sCompName
         If Left(sCompName, 3) = "wks" Then sCompName = Mid(sCompName, 4)
         sCompName = EU_VBP_Lib.AddSheetToWorkbook(aWb:=ActiveWorkbook, sSheetName:=sCompName)
```

```
         If sCompName = vbNullString Then
            MsgBox "Sheet '" & sCompName & "' could not be added.", vbCritical + vbOKOnly, cMe
            Exit Sub
         End If
         oProject.VBComponents(sCompName).Name = sCodeName
         sCompName = sCodeName
      ElseIf sImportExt = ".cls" Or sImportExt = ".bas" Or sImportExt = ".frm" Then
         If Not EU_VBP_Lib.AddModuleToWorkbook(aVBAProject:=oProject, sCompName:=sCompName) = sCompNam
e Then
            MsgBox "Component '" & sCompName & "' could not be added.", vbCritical + vbOKOnly, cMe
            Exit Sub
         End If
      Else
         MsgBox "File '" & sImportFile & "' could not be imported.", vbCritical + vbOKOnly, cMe
         Exit Sub
      End If
   End If
   ' Now Import all lines for this module/sheet
   Set oComp = oProject.VBComponents(sCompName)
   sCmd = "'EU_VBP_Lib.ImportLinesFromFile(""" & sCompName & "||" & sImportFile & """)'"
   Application.OnTime Now() + TimeValue(GIMPORT_DELAY), sCmd ' This imports the module
   Call EU_VBP_Lib.Pause(iSeconds:=3)
   Set oComp = Nothing
   Set oProject = Nothing
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 08-11-2022
' Action:  Import code-lines from Module-file
' Uses:    n.a.
' Param:   sCompNameAndImportFile --
' History: 02-02-2016 Ronald Bax     First version
'          02-02-2019 Ronald Bax     2.00 Update
' --------------------------------------------------------------------
Private Sub ImportLinesFromFile(sCompNameAndImportFile As String)
   Dim oProject    As VBIDE.VBProject
   Dim oComp       As VBComponent
   Dim sCompName   As String
   Dim sImportFile As String
   Dim I           As Integer
   I = InStr(1, sCompNameAndImportFile, "||")
   sCompName = Mid(sCompNameAndImportFile, 1, I - 1)
   sImportFile = Mid(sCompNameAndImportFile, I + 2)
   Set oProject = Application.VBE.ActiveVBProject
   Set oComp = oProject.VBComponents(sCompName)
   ' At this point compilation errors may cause a crash, so we ignore those.
   On Error Resume Next
   oComp.CodeModule.DeleteLines 1, oComp.CodeModule.CountOfLines
   oComp.CodeModule.AddFromFile sImportFile
   On Error GoTo 0
   Debug.Print "Imported lines into '" & sCompName & "' from '" & sImportFile & "'."
   Set oComp = Nothing
   Set oProject = Nothing
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Copy of the same function in EU_WIN_Lib !!!
' --------------------------------------------------------------------
' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 08-11-2022
' Action:  Get DateTime of given file
' Uses:    EU_VBP_Lib.FileExists       --  Copy of the same func in EU_WIN_Lib
'          EU_VBP_Lib.GetFileDateTime  --  Copy of the same func in EU_WIN_Lib
' Param:   sFilePath --
' History: 22-01-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Private Function GetFileDateTime(sFilePath As String) As Date
   Const cMe = "EU_VBP_Lib.GetFileDateTime"
   Dim MinDate As Date
   MinDate = DateSerial(1900, 1, 1)
   If EU_VBP_Lib.FileExists(sFilePath:=sFilePath) Then
```

```
      GetFileDateTime = FileDateTime(sFilePath)
   Else
      MsgBox "File not found '" & sFilePath & "'.", vbCritical + vbOKOnly, cMe
      GetFileDateTime = MinDate
   End If
End Function
' ====================================================================

' --------------------------------------------------------------------
' Copy of the same function in EU_WIN_Lib !!!
' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 08-11-2022
' Action:  Returns if sDirPath Exists as Directory
' Uses:    n.a.
' Param:   sDirPath     -- Name to process
'          bForceCreate -- True: Try to create the Dir if not already there
' History: 02-02-2017 Ronald Bax     First version
' --------------------------------------------------------------------
Private Function DirExists(sDirPath As String, Optional ByVal bForceCreate As Boolean = False) As Bool
ean
   Dim bRes As Boolean
   Dim sDir As String
   Dim oFSO As Object
   sDir = IIf((Len(sDirPath) > 3) And (Right(sDirPath, 1) = Application.PathSeparator), Mid(sDirPath,
1, Len(sDirPath) - 1), sDirPath)
   Set oFSO = CreateObject("Scripting.FileSystemObject")
   bRes = oFSO.FolderExists(sDir)
   If bRes = False And bForceCreate = True Then
      On Error Resume Next
      oFSO.CreateFolder sDir
      On Error GoTo 0
   End If
   DirExists = oFSO.FolderExists(sDir)
   Set oFSO = Nothing
End Function
' ====================================================================

' --------------------------------------------------------------------
' Copy of the same function in EU_WIN_Lib !!!
' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 08-11-2022
' Action:  Returns if sFilePath Exists as File
' Uses:    n.a.
' Param:   sFilePath -- Name to process
' History: 02-02-2017 Ronald Bax     First version
' --------------------------------------------------------------------
Private Function FileExists(ByVal sFilePath As String) As Boolean
   Dim oFSO As Object
   Set oFSO = CreateObject("Scripting.FileSystemObject")
   FileExists = oFSO.FileExists(sFilePath)
   Set oFSO = Nothing
End Function
' ====================================================================

' --------------------------------------------------------------------
' Copy of the same function in EU_WIN_Lib !!!
' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 08-11-2022
' Action:  Converts the mapped drive path in sFilePath, to an UNC path if one exists.
'          If not, just returns sFilePath
' Uses:    n.a.
' Param:   sFilePath
' History: 02-02-2016 Ronald Bax     First version
' --------------------------------------------------------------------
Private Function Path2UNC(sFilePath As String) As String
   Dim sDrive As String
   Dim I      As Long
   sDrive = UCase(Left(sFilePath, 2))
   Path2UNC = sFilePath
   With CreateObject("WScript.Network").EnumNetworkDrives
      For I = 0 To .Count - 1 Step 2
         If .Item(I) = sDrive Then
            Path2UNC = .Item(I + 1) & Mid(sFilePath, 3)
```

```
            Exit For
         End If
      Next
   End With
End Function
' ====================================================================

' --------------------------------------------------------------------
' Copy of the same function in EU_WIN_Lib !!!
' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00          Date: 08-11-2022
' Action:  List files in given Directory
' Uses:    n.a.
' Param:   sDirPath --
'          sMask    --
'          sUnMask  --
' History: 02-02-2016 Ronald Bax     First version
' --------------------------------------------------------------------
Private Function ListfilesInDir(sDirPath As String, Optional sMask As String = "*.*", Optional sUnMask
 As String = vbNullString) As Variant
   Dim sDir    As String
   Dim sFile   As String
   Dim vaArray As Variant
   Dim I       As Integer
   sDir = LTrim(RTrim(sDirPath))
   If Right(sDir, 1) <> Application.PathSeparator Then sDir = sDir & Application.PathSeparator
   If sMask = vbNullString Then sMask = "*.*"
   If sUnMask = "*.*" Then sMask = vbNullString
   sFile = Dir(sDir & sMask)   ' Call with path "initializes" the Dir function and returns the first f
ile
   I = 0
   Do Until sFile = vbNullString ' Call it again until there are no more files
      If Not sFile Like sUnMask Then I = I + 1
      sFile = Dir              ' Subsequent calls without params return next file
   Loop
   ReDim vaArray(1 To I)
   I = 1
   sFile = Dir(sDir & sMask)
   Do Until sFile = vbNullString
      If Not sFile Like sUnMask Then
         vaArray(I) = sDir & sFile
         I = I + 1
      End If
      sFile = Dir
   Loop
   ListfilesInDir = vaArray
End Function
' ====================================================================

' --------------------------------------------------------------------
' Copy of the same function in EU_WIN_Lib !!!
' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00          Date: 08-11-2022
' Action:  Wait for iSeconds seconds while yielding to Windows
' Uses:    n.a.
' Param:   iSeconds --
' History: 02-02-2016 Ronald Bax     First version
' --------------------------------------------------------------------
Private Sub Pause(ByVal iSeconds As Single)
   Dim sngEnd As Single
   sngEnd = Timer + iSeconds
   While Timer < sngEnd
      DoEvents
   Wend
End Sub
' ====================================================================

' ====================================================================
'                            PUBLIC FUNCTIONS
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00          Date: 08-11-2022
```

```
' Action:  ...
' Uses:    n.a.
' Param:   bForceCreate --
' History: 02-02-2019 Ronald Bax     First version
' ---------------------------------------------------------------------
Public Function GetSourceDir(Optional ByVal bForceCreate As Boolean = False) As String
   Const cMe = "EU_VBP_Lib.GetSourceDir"
   Dim oProject           As VBIDE.VBProject
   Dim sVBProjectFileName As String
   Dim sDir               As String
   Dim I                  As Integer
   sDir = vbNullString
   Set oProject = Application.VBE.ActiveVBProject
   On Error Resume Next ' This can throw if the workbook has never been saved.
   sVBProjectFileName = oProject.Filename
   On Error GoTo 0
   If sVBProjectFileName = vbNullString Then ' If this is a new workbook, we skip it
      MsgBox "No file name for project '" & oProject.Name & "'. Export aborted", vbCritical + vbOKOnly
, cMe
      GoTo ExitFunc
   End If
   I = InStrRev(sVBProjectFileName, Application.PathSeparator, , vbTextCompare)
   If I > 0 Then sVBProjectFileName = Mid(sVBProjectFileName, 1, I)
   sDir = sVBProjectFileName & "Source\"
   If Not EU_VBP_Lib.DirExists(sDirPath:=sDir, bForceCreate:=bForceCreate) Then
      If bForceCreate Then
         MsgBox "Directory '" & sDir & "' could not be created.", vbCritical + vbOKOnly, cMe
      Else
         MsgBox "Directory '" & sDir & "' not found.", vbCritical + vbOKOnly, cMe
      End If
   End If
ExitFunc:
   GetSourceDir = sDir
   Set oProject = Nothing
End Function
' =====================================================================

' ---------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00          Date: 08-11-2022
' Action:  Return the path to the Library-files
' Uses:    n.a.
' Param:   n.a.
' History: 01-02-2019 Ronald Bax     First version
' ---------------------------------------------------------------------
Public Function GetStandardLibraryPath() As String
   Const cMe = "EU_VBP_Lib.GetStandardLibraryPath"
   Dim sLibDir As String
   GetStandardLibraryPath = vbNullString
   sLibDir = LibPath1
   If Right(sLibDir, 1) <> Application.PathSeparator Then sLibDir = sLibDir & Application.PathSeparato
r
   If Not EU_VBP_Lib.LibFileExists(sLibDir:=sLibDir) Then
      sLibDir = LibPath2
      If Right(sLibDir, 1) <> Application.PathSeparator Then sLibDir = sLibDir & Application.PathSepar
ator
      If Not EU_VBP_Lib.LibFileExists(sLibDir:=sLibDir) Then
         MsgBox "The LibraryDirectory could not be found.", vbCritical + vbOKOnly, cMe
         Exit Function
      End If
   End If
   GetStandardLibraryPath = sLibDir
End Function
' =====================================================================

' ---------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00          Date: 08-11-2022
' Action:  Clean-up of all Modules
' Uses:    n.a.
' Param:   sAction --
' History: 02-02-2016 Ronald Bax     First version
'          02-02-2019 Ronald Bax     2.00 Update
' ---------------------------------------------------------------------
Public Sub CleanAllModules(Optional sAction As String = vbNullString)
```

```
   Dim oComp              As VBComponent
   Dim iLine              As Long
   Dim iCnt               As Long
   Dim sLine              As String
   Dim sChg               As String
   Dim bLastRealLineFound As Boolean
   With Application.VBE.ActiveVBProject
      For Each oComp In .VBComponents
         If oComp.Name <> "EU_VBP_Lib" Then
            iCnt = oComp.CodeModule.CountOfLines
            If iCnt <= 3 Then
               If RTrim(oComp.CodeModule.Lines(1, 1)) = "Option Explicit" Then
                  oComp.CodeModule.DeleteLines 1, 1
               End If
            End If
            bLastRealLineFound = False
            For iLine = oComp.CodeModule.CountOfLines To 1 Step -1
               sLine = RTrim(oComp.CodeModule.Lines(iLine, 1))
               ' Update or add a first line with last update-action
               If iLine = 1 Then
                  If (UCase(Left(oComp.Name, 3)) <> "EU_") And (UCase(Right(oComp.Name, 4)) <> UCase("
_Lib")) Then
                     If oComp.CodeModule.CountOfLines >= 3 Then
                        sChg = "' Last Cleaned (" & sAction & "): " & Format(Now, "dd-mm-yyyy hh:nn")
                        If Left(sLine, 14) = "' Last Cleaned" Then
                           oComp.CodeModule.ReplaceLine 1, sChg
                        Else
                           oComp.CodeModule.InsertLines 1, sChg
                        End If
                        sLine = sChg
                     End If
                  End If
               End If
               ' Remove empty lines at the end of the source-ode
               If sLine = vbNullString And Not bLastRealLineFound Then
                  Debug.Print "Removing empty lines at EOF: '" & oComp.Name & "'."
                  oComp.CodeModule.DeleteLines iLine, 1
               Else
                  bLastRealLineFound = True
               End If
               ' Update the line if it is different
               If sLine <> oComp.CodeModule.Lines(iLine, 1) Then
                  oComp.CodeModule.ReplaceLine iLine, sLine
               End If
            Next iLine
         End If
      Next
   End With
   Set oComp = Nothing
End Sub
' ===========================================================================

' ---------------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 08-11-2022
' Action:  Reload the given EU_ library
' Uses:    n.a.
' Param:   sLibPath --
' History: 02-02-2016 Ronald Bax     First version
'          02-02-2019 Ronald Bax     2.00 Update
' ---------------------------------------------------------------------------
Public Sub UpdateEULib(sLibPath As String)
   Dim oldStatusBar As Boolean
   With Application
      oldStatusBar = .DisplayStatusBar
      .Cursor = xlWait
      .DisplayStatusBar = True
      .StatusBar = "Reloading " + sLibPath + " ..."
      Call EU_VBP_Lib.Import_Code_From_File(sFilePath:=sLibPath)
      .StatusBar = False
      .DisplayStatusBar = oldStatusBar
      .Cursor = xlDefault
   End With
End Sub
```

```
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 08-11-2022
' Action:  Reload ALL EU_ libraries
' Uses:    n.a.
' Param:   n.a.
' History: 02-02-2016 Ronald Bax     First version
'          02-02-2019 Ronald Bax     2.00 Update
' --------------------------------------------------------------------
Public Sub UpdateAllEULibs()
   Dim oProject     As VBIDE.VBProject
   Dim oComp        As VBComponent
   Dim sCmd         As String
   Dim sLibDir      As String
   Dim oldStatusBar As Boolean
   Set oProject = Application.VBE.ActiveVBProject
   sLibDir = EU_VBP_Lib.GetStandardLibraryPath()
   For Each oComp In oProject.VBComponents
      If UCase(Left(oComp.Name, 3)) = "EU_" And UCase(Right(oComp.Name, 4)) = UCase("_Lib") Then
         If (UCase(oComp.Name) <> UCase("EU_ALL_Lib")) And (UCase(oComp.Name) <> UCase("EU_VBP_Lib"))
Then
            With Application
               oldStatusBar = .DisplayStatusBar
               .Cursor = xlWait
               .DisplayStatusBar = True
               .StatusBar = "Reloading: " + oComp.Name + " ..."
            End With
            sCmd = "'EU_VBP_Lib.UpdateEULib(""" & sLibDir & oComp.Name & ".bas"")'"
            Application.OnTime Now() + TimeValue(GCONTINUE_DELAY), sCmd
            With Application
               .StatusBar = False
               .DisplayStatusBar = oldStatusBar
               .Cursor = xlDefault
            End With
         End If
      End If
   Next oComp
ExitFun:
   Set oComp = Nothing
   Set oProject = Nothing
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 08-11-2022
' Action:  Export all components
' Uses:    n.a.
' Param:   n.a.
' History: 18-02-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Public Sub Export_All_Code()
   Dim oProject     As VBIDE.VBProject
   Dim oComp        As VBComponent
   Dim sExportDir   As String
   Dim I            As Integer
   Dim oldStatusBar As Boolean
   Set oProject = Application.VBE.ActiveVBProject
   sExportDir = EU_VBP_Lib.GetSourceDir(bForceCreate:=True)
   If sExportDir = vbNullString Then GoTo ExitFun
   Call EU_VBP_Lib.CleanAllModules(sAction:="Export")
   For Each oComp In oProject.VBComponents
      If EU_VBP_Lib.HasCodeToExport(aComp:=oComp) Then
         With Application
            oldStatusBar = .DisplayStatusBar
            .Cursor = xlWait
            .DisplayStatusBar = True
            .StatusBar = "Exporting " + oComp.Name + " ..."
         End With
         Select Case oComp.Type
            Case vbext_ct_ClassModule
                 Call EU_VBP_Lib.ExportComponent(sExportDir:=sExportDir, aComp:=oComp)
            Case vbext_ct_StdModule
```

```
                Call EU_VBP_Lib.ExportComponent(sExportDir:=sExportDir, aComp:=oComp, sExtension:=".b
as")
            Case vbext_ct_MSForm
                Call EU_VBP_Lib.ExportComponent(sExportDir:=sExportDir, aComp:=oComp, sExtension:=".f
rm")
            Case vbext_ct_Document
                Call EU_VBP_Lib.ExportLines(sExportDir:=sExportDir, aComp:=oComp)
            Case Else
                ' Raise "Unkown component type"
        End Select
        With Application
           .StatusBar = False
           .DisplayStatusBar = oldStatusBar
           .Cursor = xlDefault
        End With
      End If
   Next oComp
   Call EU_VBP_Lib.CreateCompareBacth(sFilePath:=sExportDir & "CompareLibs.bat", bShow:=True)
ExitFun:
   Set oComp = Nothing
   Set oProject = Nothing
End Sub
' ===============================================================================

' -------------------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 08-11-2022
' Action:  Import all code
' Uses:    n.a.
' Param:   bIncLibs --
' History: 18-02-2019 Ronald Bax     First version
' -------------------------------------------------------------------------------
Public Sub Import_All_Code(Optional ByVal bIncLibs As Boolean = False)
   Dim Comp         As VBComponent
   Dim sImportDir   As String
   Dim I            As Integer
   Dim S            As String
   Dim sUnMask      As String
   Dim vaArray      As Variant
   Dim oldStatusBar As Boolean
   sImportDir = EU_VBP_Lib.GetSourceDir(bForceCreate:=False)
   If sImportDir = vbNullString Then Exit Sub
   sUnMask = IIf(bIncLibs, vbNullString, "EU_*.*")
   vaArray = EU_VBP_Lib.ListfilesInDir(sDirPath:=sImportDir, sMask:="*.*", sUnMask:=sUnMask)
   For I = 1 To UBound(vaArray)
      S = Mid(vaArray(I), Len(sImportDir))
      S = Mid(S, InStr(S, ".") + 1)
      If S = "sheet.cls" Or S = "cls" Or S = "bas" Or S = "frm" Then
         S = vaArray(I)
         With Application
            oldStatusBar = .DisplayStatusBar
            .Cursor = xlWait
            .DisplayStatusBar = True
            .StatusBar = "Importing " + S + " ..."
         End With
         Call EU_VBP_Lib.Import_Code_From_File(sFilePath:=S)
         With Application
            .StatusBar = False
            .DisplayStatusBar = oldStatusBar
            .Cursor = xlDefault
         End With
      End If
   Next I
   Call EU_VBP_Lib.CleanAllModules(sAction:="Import")
End Sub
' ===============================================================================

' -------------------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 08-11-2022
' Action:  Compare Procedures and SourceCode to get the full declarations
' Uses:    n.a.
' Param:   n.a.
' Note:    Public Global Collections required:
'          GCollProcedures, GCollModules, GCollSourceCode and GCollProcDecl
```

```
' History: 23-07-2019 Ronald Bax     First version
' ---------------------------------------------------------------------
Private Sub GetFullProcDeclarations()
   Dim arrProc() As String
   Dim arrSrce() As String
   Dim sSrceLine As String
   Dim I, J      As Integer
   For I = 1 To GCollProcedures.Count  ' For each procedure
      arrProc = Split(GCollProcedures(I), "|")
      For J = 1 To GCollSourceCode.Count ' For each sourceline
         arrSrce = Split(GCollSourceCode(J), "|")
         If arrProc(0) = arrSrce(0) Then
            sSrceLine = Trim(arrSrce(1))
            If Len(sSrceLine) > 0 And Left(sSrceLine, 1) <> "'" Then
               If InStr(1, arrSrce(1), "Function ", vbTextCompare) > 0 Or InStr(1, arrSrce(1), "Sub ",
 vbTextCompare) > 0 Then
                  If InStr(1, arrSrce(1), " " & arrProc(1) & "(", vbTextCompare) > 0 Then
                     GCollProcDecl.Add arrProc(0) & "|" & sSrceLine
                     Exit For ' done with searching the sourcecode; do next procedure now
                  End If
               End If
            End If
         End If
      Next J
   Next I
End Sub
' =====================================================================

' ---------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00          Date: 08-11-2022
' Action:  Clear the 4 global collections
' Uses:    n.a.
' Param:   n.a.
' Note:    Public Global Collections required:
'          GCollProcedures, GCollModules, GCollSourceCode and GCollProcDecl
' History: 18-06-2019 Ronald Bax     First version
' ---------------------------------------------------------------------
Public Sub ClearGlobalCollections()
   Dim I As Integer
   For I = GCollProcedures.Count To 1 Step -1
      GCollProcedures.Remove (I)
   Next I
   For I = GCollModules.Count To 1 Step -1
      GCollModules.Remove (I)
   Next I
   For I = GCollSourceCode.Count To 1 Step -1
      GCollSourceCode.Remove (I)
   Next I
   For I = GCollProcDecl.Count To 1 Step -1
      GCollProcDecl.Remove (I)
   Next I
End Sub
' =====================================================================

' ---------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00          Date: 08-11-2022
' Action:
' Uses:    EU_VBP_Lib.Path2UNC         --  Copy of the same func in EU_WIN_Lib
' Param:   n.a.
' Note:    Public Global Collections required:
'          GCollProcedures, GCollModules, GCollSourceCode and GCollProcDecl
' History: 18-06-2019 Ronald Bax     First version
' ---------------------------------------------------------------------
Public Sub GetModuleInfoFromWB(aWb As Workbook, sPW As String)
   Const cMe = "GetModuleInfoFromWB"
   Dim oVBAProject  As VBIDE.VBProject
   Dim myVBAProject As VBIDE.VBProject
   Dim vbComp       As VBIDE.VBComponent
   Dim objCode      As VBIDE.CodeModule
   Dim iComp        As Long
   Dim I            As Long
   Dim sLine        As String
   Dim sModName     As String
```

```
   Dim sModTrueName As String
   Dim strProcName  As String
   ' Check if the Workbook exists
   If aWb Is Nothing Then
      MsgBox "Workbook-parameter invalid or Workbook not open.", vbOKOnly + vbCritical, cMe
      Exit Sub
   End If
   ' Research if it's VBA project
   Set myVBAProject = Nothing
   For Each oVBAProject In Application.VBE.VBProjects
      If EU_VBP_Lib.Path2UNC(sFilePath:=oVBAProject.Filename) = EU_VBP_Lib.Path2UNC(sFilePath:=aWb.Ful
lName) Then
         Set myVBAProject = oVBAProject
         GbPWProtected = False
         If myVBAProject.Protection = 1 Then
            GbPWProtected = True
            Set Application.VBE.ActiveVBProject = myVBAProject
            SendKeys ("{BACKSPACE}{BACKSPACE}{BACKSPACE}{BACKSPACE}{BACKSPACE}{BACKSPACE}{BACKSPACE}{B
ACKSPACE}")
            SendKeys sPW
            SendKeys ("{ENTER}{ENTER}")
            Application.VBE.CommandBars(1).FindControl(ID:=2578, recursive:=True).Execute
            Application.Wait (Now + TimeValue("0:00:1"))
         End If
      End If
   Next oVBAProject
   If myVBAProject Is Nothing Then
      MsgBox "getModuleInfoFromWB: VBProject not found.", vbOKOnly + vbCritical, cMe
      Exit Sub
   End If
   ' Now get all data from the VBProject into the Global-Collections
   Call EU_VBP_Lib.Pause(iSeconds:=1)
   Call EU_VBP_Lib.ClearGlobalCollections
   With myVBAProject
      For Each vbComp In .VBComponents
         If vbComp.Type = vbext_ct_StdModule Or vbComp.Type = vbext_ct_Document Then
            sModName = vbComp.Name
            sModTrueName = sModName
            For I = 1 To aWb.Worksheets.Count
                If aWb.Worksheets(I).CodeName = sModName Then
                   sModTrueName = aWb.Worksheets(I).Name
                   Exit For
                End If
            Next I
            GCollModules.Add sModName & "|" & sModTrueName
            Set objCode = vbComp.CodeModule
            ' Get all modules and First Procedure.
            strProcName = objCode.ProcOfLine(objCode.CountOfDeclarationLines + 1, vbext_pk_Proc)
            If strProcName <> vbNullString Then
               ' 2020-03-19 Added IF, to prevent false Object-as-proc declararions
               If InStr(1, objCode.Lines(objCode.CountOfDeclarationLines, 1), "Object", vbTextCompare)
 = 0 Then
                  GCollProcedures.Add sModName & "|" & strProcName
               End If
            End If
            For I = objCode.CountOfDeclarationLines + 1 To objCode.CountOfLines
               If strProcName <> objCode.ProcOfLine(I, vbext_pk_Proc) Then
                  strProcName = objCode.ProcOfLine(I, vbext_pk_Proc)
                  If InStr(1, strProcName, "Public Property", vbTextCompare) = 0 Then
                     GCollProcedures.Add sModName & "|" & strProcName
                  End If
               End If
            Next I
            ' Get all source-code lines
            For I = 1 To objCode.CountOfLines
               sLine = objCode.Lines(I, 1)
               If Left(sLine, 1) = "'" Then sLine = "'" & sLine
               GCollSourceCode.Add sModName & "|" & sLine
            Next I
         End If
      Next
   End With
   Call EU_VBP_Lib.GetFullProcDeclarations
```

```
   Set objCode = Nothing
   Set vbComp = Nothing
   Set myVBAProject = Nothing
   Set oVBAProject = Nothing
End Sub
' ===============================================================================


' ===============================================================================
' ===============================================================================
```

```
' ====================================================================
'                            EU_WIN_Lib.bas
' ====================================================================
' Author:  Ronald Bax          Version: 3.00          Date: 14-03-2021
' Action:  Library with OS and Windows-related functions
' Uses:    See note
' History: 21-10-2016 Ronald Bax     First version
'          25-10-2016 Ronald Bax     1.10 Update libraries added
'          02-02-2017 Ronald Bax     1.20 Separate Win_Lib
'          21-04-2020 Ronald Bax     2.60 Corrected RemoveFilesOlderThan
' ====================================================================
' Note:    This Lib is GENERIC and INDEPENDENT of any other libs/sheets
' ====================================================================
' Public  Function GetVersion() As String
' Public  Function GetDTUpdate() As Date
' Public  Function LibFileExists(sLibDir As String) As Boolean
' Public  Function GetLibFileDate(sLibDir As String) As Date
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Private Function GetRegistryValue(sRegKey As String) As String
' Public  Function GetEnvVar(sVar As String) As String
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Public  Function GetFileDateTime(sFilePath As String) As Date
' Public  Function SetFileDateTime(sFilePath As String, sNewFileDate As String) As Boolean
' Public  Function RemoveDirBackslash(sDirPath As String) As String
' Public  Function AddDirBackslash(sDirPath As String) As String
' Public  Function FileExists(sFilePath As String) As Boolean
' Public  Function DirExists(sDirPath As String, Optional ByVal bForceCreate As Boolean = False) As Bo
olean
' Public  Function ForceCreateDirIfNotThere(sDirPath As String) As Boolean
' Public  Function SaveOldVersionOfFile(sFilePath As String, Optional ByVal bShowMsg As Boolean = Fals
e) As Boolean
' Public  Function Path2UNC(sFilePath As String) As String
' Public  Function ListfilesInDir(sDirPath As String, Optional sMask As String = "*.*", Optional sUnMa
sk As String = vbNullString) As Variant
' Public  Function GetFileNameDlg(Optional sFilePath As String = vbNullString) As String
' Public  Sub      PickAFilePathInCell(aCell As Range)
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Public  Sub      Pause(ByVal iSeconds As Single)
' Public  Sub      ExecBAT(sBAT As String, Optional ByVal iWinStyle As Long = vbHide, Optional ByVal b
WaitForReturn As Boolean = False)
' Public  Sub      ExploreDirectory(sDirPath As String)
' Public  Sub      RemoveFilesOlderThan(sDirPath As String, Optional sMask As String = vbNullString, O
ptional ByVal dtBeforeDate As Date, Optional ByVal MinFilesToKeep As Long = 10)
' Public  Sub      Log(sMsg AS String, Optional sLogFile AS String = vbNullString)
' Public  Sub      LogTruncate(Optional sLogFile As String = vbNullString, Optional iLines As Integer
= 100)
' Public  Sub      OpenTextFileWithNotepad(sFilePath As String)
' Public  Sub      MapNetworkDrive(sDrive As String, sShare As String, Optional ByVal bPersistent As B
oolean = True, Optional sUser As String = vbNullString, Optional sPWD As String = vbNullString)
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Public  Sub      ClearRandNumArray()
' Public  Function RandNum1to100(ByVal iMax As Integer, Optional ByVal bRemember As Boolean = True) As
 Integer
' Public  Sub      GetRandomOrder(ByRef aMyArray() As Integer)
' Public  Sub      ResortMessages(ByRef aMsgArray() As String)
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Public  Sub      MsgCrit(sMsg As String, sTitle As String, Optional bBeep As Boolean = False)
' Public  Sub      MsgExcl(sMsg As String, sTitle As String, Optional bBeep As Boolean = False)
' Public  Sub      MsgInfo(sMsg As String, sTitle As String, Optional bBeep As Boolean = False)
' Public  Sub      MsgFNF(sFile As String, sTitle As String, Optional bBeep As Boolean = False)
' Public  Sub      MsgDNF(sDir  As String, sTitle As String, Optional bBeep As Boolean = False)
' Public  Function AskYN (sMsg  As String, sTitle As String, Optional bBeep As Boolean = False) As Boo
lean
' Public  Function AskNY (sMsg  As String, sTitle As String, Optional bBeep As Boolean = False) As Boo
lean
' ====================================================================
Option Explicit

' ====================================================================
'                      Globals, Constants and Types
' ====================================================================
Global RandNumArray(100) As Boolean
Public Const vbSpace = " "
```

```
Public Const vbDSpace = "  "
Public Const vbDQuote = """"
Public Const vbSQuote = "'"
Public Const vbPathSep = "\"
Public Const vbURLRoot = "\\"
Private Const EU_WIN_Lib_LVersion As String = "3.00"
Private Const EU_WIN_Lib_DTUpdate As String = "14-03-2021"
Private Const EU_WIN_Lib_FileName As String = "EU_WIN_Lib.bas"

Private Type SYSTEMTIME
   wYear         As Integer
   wMonth        As Integer
   wDayOfWeek    As Integer
   wDay          As Integer
   wHour         As Integer
   wMinute       As Integer
   wSecond       As Integer
   wMilliseconds As Integer
End Type

Private Type FILETIME
   dwLowDateTime  As Long
   dwHighDateTime As Long
End Type

Private Declare PtrSafe Function SystemTimeToFileTime Lib "kernel32" (lpSystemTime As SYSTEMTIME, lpFi
leTime As FILETIME) As Long
Private Declare PtrSafe Function LocalFileTimeToFileTime Lib "kernel32" (lpLocalFileTime As FILETIME,
lpFileTime As FILETIME) As Long
Private Declare PtrSafe Function CreateFile Lib "kernel32" Alias "CreateFileA" (lpFileName As String,
ByVal dwDesiredAccess As Long, ByVal dwShareMode As Long, ByVal lpSecurityAttributes As Long, ByVal dw
CreationDisposition As Long, ByVal dwFlagsAndAttributes As Long, ByVal hTemplateFile As Long) As Long
Private Declare PtrSafe Function SetFileTime Lib "kernel32" (ByVal hFile As Long, lpCreationTime As An
y, lpLastAccessTime As Any, lpLastWriteTime As Any) As Long
Private Declare PtrSafe Function CloseHandle Lib "kernel32" (ByVal hObject As Long) As Long
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00          Date: 14-03-2021
' Action:  Get info about this library
' Uses:    EU_WIN_Lib.FileExists
'          EU_WIN_Lib.GetFileDateTime
' Param:   sLibDir
' History: 02-02-2016 Ronald Bax     First version
'          02-02-2019 Ronald Bax     2.00 Update
' --------------------------------------------------------------------
Public Function GetVersion() As String
   GetVersion = EU_WIN_Lib_LVersion
End Function
' --------------------------------------------------------------------
Public Function GetDTUpdate() As Date
   GetDTUpdate = CDate(EU_WIN_Lib_DTUpdate)
End Function
' --------------------------------------------------------------------
Public Function LibFileExists(sLibDir As String) As Boolean
   LibFileExists = EU_WIN_Lib.FileExists(sFilePath:=sLibDir & EU_WIN_Lib_FileName)
End Function
' --------------------------------------------------------------------
Public Function GetLibFileDate(sLibDir As String) As Date
   GetLibFileDate = EU_WIN_Lib.GetFileDateTime(sFilePath:=sLibDir & EU_WIN_Lib_FileName)
End Function
' ====================================================================

' ====================================================================
'                           PRIVATE FUNCTIONS
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00          Date: 14-03-2021
' Action:  Get a value from the registry
' Uses:    n.a.
' Param:   sRegKey
' History: 02-02-2016 Ronald Bax     First version
```

```
'          02-02-2019 Ronald Bax     2.00 Update
' ------------------------------------------------------------------------
Private Function GetRegistryValue(sRegKey As String) As String
   Dim oReg As Object
   GetRegistryValue = vbNullString
   On Error Resume Next
   Set oReg = CreateObject("WScript.Shell")
   GetRegistryValue = oReg.RegRead(sRegKey)
   Set oReg = Nothing
End Function
' ========================================================================

' ========================================================================
'                            PUBLIC FUNCTIONS
' ========================================================================

' ------------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  Get DateTime of given file
' Uses:    EU_WIN_Lib.FileExists
'          EU_WIN_Lib.GetFileDateTime
' Param:   sFilePath --
' History: 22-01-2019 Ronald Bax     First version
' ------------------------------------------------------------------------
Public Function GetFileDateTime(sFilePath As String) As Date
   Const cMe = "EU_WIN_Lib.GetFileDateTime"
   Dim MinDate As Date
   MinDate = DateSerial(1900, 1, 1)
   If EU_WIN_Lib.FileExists(sFilePath:=sFilePath) Then
      GetFileDateTime = FileDateTime(sFilePath)
   Else
      Call EU_WIN_Lib.MsgFNF(sFile:=sFilePath, sTitle:=cMe, bBeep:=True)
      GetFileDateTime = MinDate
   End If
End Function
' ========================================================================

' ------------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  Set File Date (and optionally time) for a given file)
'          Returns True if successful, false otherwise
' Uses:    EU_WIN_Lib.FileExists
'          EU_WIN_Lib.GetFileDateTime
' Param:   sFilePath    -- The File Name
'          sNewFileDate -- Date to Set File's Modified Date/Time ''yyyy-mm-dd hh:nn:ss'
' History: 22-01-2019 Ronald Bax     First version
' ------------------------------------------------------------------------
Public Function SetFileDateTime(sFilePath As String, sNewFileDate As String) As Boolean
   Const cMe = "EU_WIN_Lib.SetFileDateTime"
   Const GENERIC_WRITE = &H40000000
   Const OPEN_EXISTING = 3
   Const FILE_SHARE_READ = &H1
   Const FILE_SHARE_WRITE = &H2

   Dim lFileHnd      As Long
   Dim lRet          As Long
   Dim typSystemTime As SYSTEMTIME
   Dim typLocalTime  As FILETIME
   Dim typFileTime   As FILETIME

   If Not EU_WIN_Lib.FileExists(sFilePath:=sFilePath) Then
      Call EU_WIN_Lib.MsgFNF(sFile:=sFilePath, sTitle:=cMe, bBeep:=True)
      Exit Function
   End If
   If Not IsDate(sNewFileDate) Then
      Call EU_WIN_Lib.MsgCrit(sMsg:="Param 'sNewFileDate' contains an incorrect DateTime (" & sNewFile
Date & ").", sTitle:=cMe, bBeep:=True)
      Exit Function
   End If
   With typSystemTime
       .wYear = Year(sNewFileDate)
       .wMonth = Month(sNewFileDate)
       .wDayOfWeek = Weekday(sNewFileDate) - 1
```

```
       .wDay = Day(sNewFileDate)
       .wHour = Hour(sNewFileDate)
       .wMinute = Minute(sNewFileDate)
       .wSecond = Second(sNewFileDate)
       .wMilliseconds = 0
   End With
   lRet = SystemTimeToFileTime(typSystemTime, typLocalTime)  ' Convert system time to local time
   lRet = LocalFileTimeToFileTime(typLocalTime, typFileTime) ' Convert local time to GMT
   lFileHnd = CreateFile(sFilePath, GENERIC_WRITE, FILE_SHARE_READ Or FILE_SHARE_WRITE, ByVal 0&, OPEN
_EXISTING, 0, 0)
   lRet = SetFileTime(lFileHnd, 0&, 0&, typFileTime)
   CloseHandle lFileHnd
   SetFileDateTime = (lRet > 0) And Format(CDate(EU_WIN_Lib.GetFileDateTime(sFilePath:=sFilePath)), "d
d-mm-yyyy hh:nn:ss") = Format(sNewFileDate, "dd-mm-yyyy hh:nn:ss")
End Function
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  Return a Dir-name without a "\" at the end
' Uses:    n.a.
' Param:   sDirPath -- Name to process
' History: 02-02-2017 Ronald Bax     First version
' --------------------------------------------------------------------
Public Function RemoveDirBackslash(sDirPath As String) As String
   RemoveDirBackslash = IIf((Len(sDirPath) > 3 And Right(sDirPath, 1) = vbPathSep), Mid(sDirPath, 1, L
en(sDirPath) - 1), sDirPath)
End Function
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  Return a Dir-name with a "\" at the end
' Uses:    n.a.
' Param:   sDirPath Name to process
' History: 02-02-2017 Ronald Bax     First version
' --------------------------------------------------------------------
Public Function AddDirBackslash(sDirPath As String) As String
   AddDirBackslash = IIf(Right(sDirPath, 1) = vbPathSep, sDirPath, sDirPath + vbPathSep)
End Function
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  Returns if sFilePath Exists as File
' Uses:    n.a.
' Param:   sFilePath -- Name to process
' History: 02-02-2017 Ronald Bax     First version
' --------------------------------------------------------------------
Public Function FileExists(sFilePath As String) As Boolean
   Dim oFSO As Object
   Set oFSO = CreateObject("Scripting.FileSystemObject")
   FileExists = oFSO.FileExists(sFilePath)
   Set oFSO = Nothing
End Function
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  Returns if sDirPath Exists as Directory
' Uses:    EU_WIN_Lib.RemoveDirBackslash
' Param:   sDirPath     -- Name to process
'          bForceCreate -- True: Try to create the Dir if not already there
' History: 02-02-2017 Ronald Bax     First version
' --------------------------------------------------------------------
Public Function DirExists(sDirPath As String, Optional ByVal bForceCreate As Boolean = False) As Boole
an
   Dim oFSO As Object
   Dim bRes As Boolean
   Dim sDir As String
   sDir = EU_WIN_Lib.RemoveDirBackslash(sDirPath:=sDirPath)
   Set oFSO = CreateObject("Scripting.FileSystemObject")
   bRes = oFSO.FolderExists(sDir)
```

```
   If bRes = False And bForceCreate = True Then
      On Error Resume Next
      oFSO.CreateFolder sDir
      On Error GoTo 0
   End If
   DirExists = oFSO.FolderExists(sDir)
   Set oFSO = Nothing
End Function
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  Returns if sDirPath Exists as Directory. If not Create it first
' Uses:    EU_WIN_Lib.DirExists
' Param:   sDirPath -- Name to process
' History: 02-02-2017 Ronald Bax     First version
' --------------------------------------------------------------------
Public Function ForceCreateDirIfNotThere(sDirPath As String) As Boolean
   ForceCreateDirIfNotThere = EU_WIN_Lib.DirExists(sDirPath:=sDirPath, bForceCreate:=True)
End Function
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  Create a backup-copy of the given file in .\Archive
'          with the same name, and the FORMER file-date-time as prefix
'          Note: The directory .\Archive must already exist
' Uses:    EU_WIN_Lib.GetFileDateTime
'          EU_WIN_Lib.AddDirBackslash
'          EU_WIN_Lib.FileExists
'          EU_WIN_Lib.DirExists
'          EU_WIN_Lib.RemoveFilesOlderThan
' Param:   sFilePath -- Name to process
'          bShowMsg  -- True = show messages if errors
' History: 02-02-2017 Ronald Bax     First version
' --------------------------------------------------------------------
Public Function SaveOldVersionOfFile(sFilePath As String, Optional ByVal bShowMsg As Boolean = False)
As Boolean
   Const cArchDir = "Archive"
   Const cMe = "EU_WIN_Lib.SaveOldVersionOfFile"
   Dim oFSO      As Object
   Dim sDT       As String
   Dim sDir      As String
   Dim sFile     As String
   Dim sFil      As String
   Dim sExt      As String
   Dim sMask     As String
   Dim sFileName As String

   SaveOldVersionOfFile = EU_WIN_Lib.FileExists(sFilePath:=sFilePath)
   If SaveOldVersionOfFile = False Then
      If bShowMsg Then Call EU_WIN_Lib.MsgFNF(sFile:=sFilePath, sTitle:=cMe, bBeep:=True)
      Exit Function
   End If
   sDT = Format$(EU_WIN_Lib.GetFileDateTime(sFilePath:=sFilePath), "yyyymmddhhnn")
   sFile = Right(sFilePath, Len(sFilePath) - InStrRev(sFilePath, vbPathSep))
   sDir = EU_WIN_Lib.AddDirBackslash(sDirPath:=Left(sFilePath, Len(sFilePath) - Len(sFile)))
   sExt = Right(sFile, Len(sFile) - InStrRev(sFile, ".") + 1)
   sFil = Left(sFile, InStrRev(sFile, "."))
   sFileName = EU_WIN_Lib.AddDirBackslash(sDirPath:=sDir & cArchDir) & sFil & sDT & sExt
   sMask = sFil & "????????????" & sExt
   If Not EU_WIN_Lib.DirExists(sDirPath:=sDir & cArchDir, bForceCreate:=True) Then
      If bShowMsg Then Call EU_WIN_Lib.MsgCrit(sMsg:="Could not create " & sDir & cArchDir, sTitle:=cM
e, bBeep:=True)
   Else
      Set oFSO = CreateObject("Scripting.FileSystemObject")
      If oFSO.FileExists(sFilePath) And Not oFSO.FileExists(sFileName) Then
         oFSO.CopyFile sFilePath, sFileName, False
      End If
   End If
   ' Keep at least files younger than 28 days and at least 10 files
   Call EU_WIN_Lib.RemoveFilesOlderThan(sDirPath:=sDir & cArchDir, sMask:=sMask, dtBeforeDate:=Now - 2
8, iMinFilesToKeep:=10)
```

```
   SaveOldVersionOfFile = EU_WIN_Lib.FileExists(sFilePath:=sFileName)

   Set oFSO = Nothing
End Function
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  Converts the mapped drive path in sFilePath, to an UNC path if one exists.
'          If not, just returns sFilePath
' Uses:    n.a.
' Param:   sFilePath
' History: 02-02-2016 Ronald Bax     First version
' --------------------------------------------------------------------
Public Function Path2UNC(sFilePath As String) As String
   Dim sDrive As String
   Dim I      As Long
   sDrive = UCase(Left(sFilePath, 2))
   Path2UNC = sFilePath
   With CreateObject("WScript.Network").EnumNetworkDrives
      For I = 0 To .Count - 1 Step 2
         If .Item(I) = sDrive Then
            Path2UNC = .Item(I + 1) & Mid(sFilePath, 3)
            Exit For
         End If
      Next
   End With
End Function
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  List files in given Directory
' Uses:    n.a.
' Param:   sDirPath --
'          sMask    --
'          sUnMask  --
' History: 02-02-2016 Ronald Bax     First version
' --------------------------------------------------------------------
Public Function ListfilesInDir(sDirPath As String, Optional sMask As String = "*.*", Optional sUnMask
As String = vbNullString) As Variant
   Dim sDir    As String
   Dim sFile   As String
   Dim vaArray As Variant
   Dim I       As Integer
   sDir = LTrim(RTrim(sDirPath))
   If Right(sDir, 1) <> vbPathSep Then sDir = sDir & vbPathSep
   If sMask = vbNullString Then sMask = "*.*"
   If sUnMask = "*.*" Then sMask = vbNullString
   sFile = Dir(sDir & sMask)   ' Call with path "initializes" the Dir function and returns the first f
ile
   I = 0
   Do Until sFile = vbNullString ' Call it again until there are no more files
      If Not sFile Like sUnMask Then I = I + 1
      sFile = Dir              ' Subsequent calls without params, return next file
   Loop
   If I = 0 Then
      ReDim vaArray(1 To 1)
      vaArray(1) = vbNullString
   Else
      ReDim vaArray(1 To I)
      I = 1
      sFile = Dir(sDir & sMask)
      Do Until sFile = vbNullString
         If Not sFile Like sUnMask Then
            vaArray(I) = sDir & sFile
            I = I + 1
         End If
         sFile = Dir
      Loop
   End If
   ListfilesInDir = vaArray
End Function
```

```
' ============================================================================

' ----------------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  ...
' Uses:    n.a.
' Param:   sFilePath --
' History: 02-02-2016 Ronald Bax     First version
' ----------------------------------------------------------------------------
Public Function GetFileNameDlg(Optional sFilePath As String = vbNullString) As String
   With Application.FileDialog(msoFileDialogFilePicker)
      .InitialFileName = sFilePath
      .AllowMultiSelect = False
      .Title = "Select a file"
      .Filters.Clear
      .Filters.Add "All files", "*.*", 1
      .Filters.Add "Text-files", "*.txt", 2
      .Filters.Add "Log-files", "*.log", 3
      Select Case Right(LCase(sFilePath), 4)
         Case ".txt": .FilterIndex = 2
         Case ".log": .FilterIndex = 3
         Case Else:   .FilterIndex = 1
      End Select
      GetFileNameDlg = vbNullString
      If .Show Then
         GetFileNameDlg = .SelectedItems(1)
      End If
   End With
End Function
' ============================================================================

' ----------------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 30-01-2019 Ronald Bax     First version
' ----------------------------------------------------------------------------
Public Sub PickAFilePathInCell(aCell As Range)
   Dim wks   As Worksheet
   Dim sFile As String
   Dim sDir  As String
   Set wks = aCell.Worksheet
   With Application
      sFile = wks.Cells(aCell.Row, aCell.Column).Value2
      sDir = Left(sFile, InStrRev(sFile, .PathSeparator))
      If Not EU_WIN_Lib.FileExists(sFilePath:=sFile) Then
         sFile = .Workbooks(1).Path & .PathSeparator & Right(sFile, Len(sFile) - Len(sDir))
         sDir = Left(sFile, InStrRev(sFile, .PathSeparator))
      End If
   End With
   ChDir (sDir)
   sFile = EU_WIN_Lib.GetFileNameDlg(sFilePath:=sFile)
   If sFile <> vbNullString Then wks.Cells(aCell.Row, aCell.Column).Value2 = sFile
   Set wks = Nothing
End Sub
' ============================================================================

' ----------------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  Return an Environment variable
' Uses:    n.a.
' Param:   sVar --
' History: 02-02-2016 Ronald Bax     First version
' ----------------------------------------------------------------------------
Public Function GetEnvVar(sVar As String) As String
   GetEnvVar = Environ(sVar)
   If UCase(sVar) = "USERNAME" Then GetEnvVar = Replace(GetEnvVar, ".", vbSpace)
End Function
' ============================================================================

' ----------------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
```

```
' Action:  Wait for iSeconds seconds while yielding to Windows
' Uses:    n.a.
' Param:   iSeconds --
' History: 02-02-2016 Ronald Bax     First version
' ------------------------------------------------------------------------
Public Sub Pause(ByVal iSeconds As Single)
   Dim sngEnd As Single
   sngEnd = Timer + iSeconds
   While Timer < sngEnd
      DoEvents
   Wend
End Sub
' ========================================================================

' ------------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  ...
' Uses:    EU_WIN_Lib.FileExists
' Param:   sBAT           -- Batchfile to execute
'          iWinStyle      --
'          0 = vbHide             Window is hidden and focus is passed to the hidden window. The vbHid
e constant is not applicable on Macintosh platforms.
'          1 = vbNormalFocus      Window has focus and is restored to its original size and position.
'          2 = vbMinimizedFocus   Window is displayed as an icon with focus.
'          3 = vbMaximizedFocus   Window is maximized with focus.
'          4 = vbNormalNoFocus    Window is restored to its most recent size and position. The current
ly active window remains active.
'          6 = vbMinimizedNoFocus Window is displayed as an icon. The currently active window remains
active.
'          bWaitForReturn -- True = wait, False = return to VBA immediately
' History: 02-02-2016 Ronald Bax     First version
' ------------------------------------------------------------------------
Public Sub ExecBAT(sBAT As String, Optional ByVal iWinStyle As Long = vbHide, Optional ByVal bWaitForR
eturn As Boolean = False)
   Const cMe = "EU_WIN_Lib.ExecBAT"
   Dim oShell As Object
   Dim sStr   As String
   If iWinStyle < 0 Or iWinStyle > 6 Or iWinStyle = 5 Then iWinStyle = vbHide
   If EU_WIN_Lib.FileExists(sFilePath:=sBAT) = False Then
      Call EU_WIN_Lib.MsgCrit(sMsg:="BatchFile not found '" & sBAT & "'.", sTitle:=cMe, bBeep:=True)
      Exit Sub
   End If
   sStr = IIf(Left(sBAT, 1) <> vbDQuote And Left(sBAT, 1) <> vbDQuote, vbDQuote & vbDQuote & sBAT & vb
DQuote & vbDQuote, sBAT)
   Set oShell = CreateObject("WScript.Shell")
   Call oShell.Run("cmd /k " & sStr, iWinStyle, bWaitForReturn)
   Set oShell = Nothing
End Sub
' ========================================================================

' ------------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  ...
' Uses:    EU_WIN_Lib.DirExists
' Param:   sDirPath --
' History: 02-02-2016 Ronald Bax     First version
' ------------------------------------------------------------------------
Public Sub ExploreDirectory(sDirPath As String)
   Const cMe = "EU_WIN_Lib.ExploreDirectory"
   Dim oShell As Object
   Dim sDir   As String
   sDir = sDirPath
   If EU_WIN_Lib.DirExists(sDirPath:=sDir, bForceCreate:=False) = False Then
      Call EU_WIN_Lib.MsgDNF(sDir:=sDir, sTitle:=cMe, bBeep:=True)
      Exit Sub
   End If
   Set oShell = CreateObject("WScript.Shell")
   If Left(sDir, 1) <> vbDQuote And Left(sDir, 1) <> vbDQuote Then sDir = vbDQuote & sDir & vbDQuote
   Call oShell.Run("explorer.exe " & sDir, vbNormalFocus, False)
   Set oShell = Nothing
End Sub
' ========================================================================
```

```
' ---------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  Delete all files in sPath, older than dtBeforeDate, and keep the youngest iMinFilesToKeep f
iles
' Uses:    EU_WIN_Lib.DirExists
' Param:   sDirPath        --
'          sMask           --
'          dtBeforeDate    --
'          iMinFilesToKeep -- Default 10
' History: 02-02-2016 Ronald Bax     First version
'          21-04-2020 Ronald Bax     2.60 Corrected RemoveFilesOlderThan
' ---------------------------------------------------------------------
Public Sub RemoveFilesOlderThan(sDirPath As String, Optional sMask As String = vbNullString, Optional
ByVal dtBeforeDate As Date = CDate("00:00:00"), Optional ByVal iMinFilesToKeep As Long = 10)
   Const cMe = "EU_WIN_Lib.RemoveFilesOlderThan"
   Dim vaArray   As Variant
   Dim SortArr   As Object
   Dim oFSO      As Object
   Dim oFile     As Object
   Dim I         As Integer
   If EU_WIN_Lib.DirExists(sDirPath:=sDirPath, bForceCreate:=False) = False Then
      Call EU_WIN_Lib.MsgDNF(sDir:=sDirPath, sTitle:=cMe, bBeep:=True)
      Exit Sub
   End If
   If dtBeforeDate = CDate("00:00:00") Or dtBeforeDate > (Now - 1) Then dtBeforeDate = Now - 1
   vaArray = EU_WIN_Lib.ListfilesInDir(sDirPath:=sDirPath, sMask:=sMask, sUnMask:=vbNullString)
   Set SortArr = CreateObject("System.Collections.ArrayList")
   For I = 1 To UBound(vaArray)
      SortArr.Add vaArray(I)
   Next I
   Erase vaArray ' Each element set to Empty.
   Set oFSO = CreateObject("Scripting.FileSystemObject")
   If SortArr.Count > iMinFilesToKeep Then
      ' Now First sort and then reverse the order
      SortArr.Sort
      SortArr.Reverse
      For I = SortArr.Count To iMinFilesToKeep + 1 Step -1
         Set oFile = oFSO.GetFile(SortArr(I - 1))
         If (oFile.DateLastModified < dtBeforeDate) Then
            Debug.Print "Delete " & oFile.Path
            Kill oFile.Path
         End If
      Next I
   End If
   Set SortArr = Nothing
   Set oFile = Nothing
   Set oFSO = Nothing
End Sub
' =====================================================================

' ---------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  Add a record to the sLogFile; Newest line is on top
' Uses:    n.a.
' Param:   sMsg     -- Message to add to the sLogFile
'          sLogFile -- Optional, default is the workbookname, ".xlsm" is replaced by ".log"
' History: 02-02-2016 Ronald Bax     First version
' ---------------------------------------------------------------------
Public Sub Log(sMsg As String, Optional sLogFile As String = vbNullString)
   Dim sFilePath     As String
   Dim sFileContents As String
   Dim hFile         As Long
   If LTrim(sMsg) = vbNullString Then Exit Sub
   sFilePath = IIf(LTrim(sLogFile) = vbNullString, Replace(Application.ActiveWorkbook.FullName, ".xlsm
", ".log"), sLogFile)
   hFile = FreeFile
   Open sFilePath For Binary As #hFile
   sFileContents = Input(LOF(hFile), #hFile)
   Close #hFile
   sFileContents = Format(Now(), "yyyy-mm-dd hh:nn:ss ") & sMsg & " [" & Environ("USERNAME") & "]" & v
bCrLf & sFileContents
   Open sFilePath For Binary As #hFile
   Put #hFile, , sFileContents
```

```
   Close #hFile
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  Truncate the sLogFile to a maximum of iLines
' Uses:    n.a.
' Param:   iLines   -- Optional, default 100
'          sLogFile -- Optional, default is the workbookname, ".xlsm" is replaced by ".log"
' History: 02-02-2016 Ronald Bax     First version
' --------------------------------------------------------------------
Public Sub LogTruncate(Optional ByVal iLines As Long = 100, Optional sLogFile As String = vbNullString
)
   Dim sFilePath     As String
   Dim sFileContents As String
   Dim aLineArray()  As String
   Dim hFile         As Long
   Dim I             As Long
   Dim iCnt          As Long
   sFilePath = IIf(LTrim(sLogFile) = vbNullString, Replace(Application.ActiveWorkbook.FullName, ".xlsm
", ".log"), sLogFile)
   hFile = FreeFile
   Open sFilePath For Binary As #hFile
   sFileContents = Input(LOF(hFile), #hFile)
   aLineArray() = Split(sFileContents, vbCrLf)
   Close #hFile
   iCnt = 0
   sFileContents = vbNullString
   For I = LBound(aLineArray) To UBound(aLineArray)
      If iCnt >= iLines Then Exit For
      sFileContents = sFileContents & aLineArray(I) & vbCrLf
      iCnt = iCnt + 1
   Next I
   Kill sFilePath
   Open sFilePath For Binary As #hFile
   Put #hFile, , sFileContents
   Close #hFile
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  Open sFilePath with "Notepad++" is present, otherwise open with "Notepad"
' Uses:    EU_WIN_Lib.GetRegistryValue
' Param:   sFilePath -- File to open
' History: 02-02-2016 Ronald Bax     First version
' --------------------------------------------------------------------
Public Sub OpenTextFileWithNotepad(sFilePath As String)
   Dim sRegVal        As String
   Dim sEditor        As String
   Dim sStr           As String
   Dim winShell       As Object
   Dim bWaitForReturn As Boolean
   If sFilePath = vbNullString Then Exit Sub
   sRegVal = EU_WIN_Lib.GetRegistryValue(sRegKey:="HKCU\Software\Classes\Applications\notepad++.exe\sh
ell\open\command\")
   sEditor = IIf(sRegVal = vbNullString, "Notepad.exe", Replace(Replace(sRegVal, " ""%1""", vbNullStri
ng), """", vbNullString))
   bWaitForReturn = False
   Set winShell = CreateObject("WScript.Shell")
   sStr = """ & sEditor & """ "" & sFilePath & """
   Call winShell.Run(sStr, vbNormalFocus, bWaitForReturn)
   Set winShell = Nothing
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  Open sFilePath with "Notepad++" is present, otherwise open with "Notepad"
' Uses:    EU_WIN_Lib.GetRegistryValue
' Param:   sFilePath -- File to open
' History: 02-02-2016 Ronald Bax     First version
```

```
' ------------------------------------------------------------------------
Public Sub MapNetworkDrive(sDrive As String, sShare As String, Optional ByVal bPersistent As Boolean =
 True, Optional sUser As String = vbNullString, Optional sPWD As String = vbNullString)
   Const cMe = "EU_WIN_Lib.NetUseDrive"
   Dim oNetwork As Object
   Dim sUNC     As String
   If Len(sDrive) = 1 Then sDrive = sDrive & ":"
   If Len(sDrive) = 2 Then sDrive = sDrive & vbPathSep
   If Len(sDrive) <> 3 Or Right(sDrive, 2) <> ":" & vbPathSep Then
      Call EU_WIN_Lib.MsgCrit(sMsg:="Incorrect drive: '" & sDrive & "'", sTitle:=cMe, bBeep:=True)
      Exit Sub
   End If
   sUNC = EU_WIN_Lib.Path2UNC(sFilePath:=sDrive & vbPathSep)
   ' If it's not connected to the correct share alread...
   If UCase(sUNC) <> UCase(sShare & vbURLRoot) Then
      ' Test if it's already connected (to a fileserver-path)
      If Left(sUNC, 2) = vbURLRoot Then
         Call EU_WIN_Lib.MsgCrit(sMsg:="Drive: '" & sDrive & "' already connected to " & sUNC, sTitle:
=cMe, bBeep:=True)
         Exit Sub
      End If
      ' Try to connect now
      On Error Resume Next
      Set oNetwork = CreateObject("WScript.Network")
      oNetwork.MapNetworkDrive Left(sDrive, 2), sShare, bPersistent, sUser, sPWD
      DoEvents
      If Err.Number <> 0 Then
         Call EU_WIN_Lib.MsgCrit(sMsg:="Drive: '" & sDrive & "' already mapped or not available.", sTi
tle:=cMe, bBeep:=True)
      End If
   End If
   Set oNetwork = Nothing
End Sub
' ========================================================================

' ------------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  Clear the internal array used by RandNum1to100
' Uses:    n.a.
' Param:   n.a.
' History: 17-07-2019 Ronald Bax     First version
' ------------------------------------------------------------------------
Public Sub ClearRandNumArray()
   Dim I As Integer
   For I = LBound(RandNumArray) To UBound(RandNumArray)
      RandNumArray(I) = False
   Next I
End Sub
' ========================================================================

' ------------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  Get a random number between 1 and iMax
' Uses:    EU_WIN_Lib.ClearRandNumArray()
' Param:   iMax   where iMax >= 2 and iMax <= 100
'          bRemember=True:  Don't return the same number twice, until every number has been returned
'          bRemember=False: Returns just a random number, but that does not seem to have a 'realy rand
om' distribution
' History: 17-07-2019 Ronald Bax     First version
' ------------------------------------------------------------------------
Public Function RandNum1to100(ByVal iMax As Integer, Optional ByVal bRemember As Boolean = True) As In
teger
   Const cMe = "EU_WIN_Lib.RandNum1to100"
   Dim iSec       As Integer
   Dim I          As Integer
   Dim bFoundFree As Boolean

   If iMax < 2 Or iMax > 100 Then
      Call EU_WIN_Lib.MsgCrit(sMsg:="iMax must be between 2 and 100.", sTitle:=cMe, bBeep:=True)
      RandNum1to100 = 0
      Exit Function
   End If
```

```
   Randomize
   iSec = (iSec + Int((iMax * Rnd) + 1)) Mod iMax
   iSec = IIf(iSec = 0, iMax, iSec)

   If bRemember Then
      While RandNumArray(iSec) And (iSec < iMax)
         Randomize
         iSec = (iSec + Int((iMax * Rnd) + 1)) Mod iMax
         iSec = IIf(iSec = 0, iMax, iSec)
      Wend
      If RandNumArray(iSec) Then
         bFoundFree = False
         For I = 1 To iMax
            bFoundFree = Not RandNumArray(I)
            If bFoundFree Then
               iSec = I
               Exit For
            End If
         Next I
         If Not bFoundFree Then
            Call EU_WIN_Lib.ClearRandNumArray
            iSec = EU_WIN_Lib.RandNum1to100(iMax:=iMax, bRemember:=True) ' Itterative call!!
         End If
      End If
      RandNumArray(iSec) = True
   End If
   RandNum1to100 = iSec
End Function
' ===========================================================================

' ---------------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  Fill a given Array of Integer with random sorted numbers
'          Suppose Array(5) of Integer,  could result in [4,1,5,3,2]
' Uses:    EU_WIN_Lib.RandNum1to100
' Param:   Array(x) of Integer  where x >= 2 and x <= 100
' History: 17-07-2019 Ronald Bax     First version
' ---------------------------------------------------------------------------
Public Sub GetRandomOrder(ByRef aMyArray() As Integer)
   Const cMe = "EU_WIN_Lib.GetRandomOrder"
   Dim I    As Integer
   Dim iMax As Integer
   Dim iRes As Integer

   iMax = UBound(aMyArray)
   If iMax < 2 Or iMax > 100 Then
      Call EU_WIN_Lib.MsgCrit(sMsg:="The array-dimensions must be between 2 and 100.", sTitle:=cMe, bB
eep:=True)
      Exit Sub
   End If

   Call EU_WIN_Lib.ClearRandNumArray
   For I = 1 To iMax
      iRes = EU_WIN_Lib.RandNum1to100(iMax:=iMax, bRemember:=True)
      If iRes = 0 Then Exit For ' make sure we dont have iMax messages :)
      aMyArray(I) = iRes
   Next I
End Sub
' ===========================================================================

' ---------------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00           Date: 14-03-2021
' Action:  Resort an array with messages randomly
' Uses:    EU_WIN_Lib.GetRandomOrder
' Param:   aMsgArray(x) of String where x >= 2 and x <= 100, Filled with messages
' History: 17-07-2019 Ronald Bax     First version
' ---------------------------------------------------------------------------
Public Sub ResortMessages(ByRef aMsgArray() As String)
   Dim iMax       As Integer
   Dim I          As Integer
   Dim MyArr()    As Integer
   Dim MyMsgArr() As String
   Dim S          As String
```

```
   iMax = UBound(aMsgArray)
   ReDim MyArr(iMax)
   ReDim MyMsgArr(iMax)
   Call EU_WIN_Lib.GetRandomOrder(aMyArray:=MyArr)
   For I = 1 To iMax ' Save the messages resorted
      MyMsgArr(I) = aMsgArray(MyArr(I))
   Next I
   For I = 1 To iMax ' Put the messages back
      aMsgArray(I) = MyMsgArr(I)
   Next I
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.00          Date: 14-03-2021
' Action:  Various Messages
' Uses:    n.a.
' Param:   sMsg=Message, sTitle=Title
' History: 17-07-2019 Ronald Bax     First version
'          14-03-2021 Ronald Bax     Beep added
' --------------------------------------------------------------------
Public Sub MsgCrit(sMsg As String, sTitle As String, Optional bBeep As Boolean = True)
   If bBeep Then Beep
   Call MsgBox(prompt:=sMsg, Buttons:=vbOKOnly + vbCritical, Title:=sTitle)
End Sub
' --------------------------------------------------------------------
Public Sub MsgExcl(sMsg As String, sTitle As String, Optional bBeep As Boolean = True)
   If bBeep Then Beep
   Call MsgBox(prompt:=sMsg, Buttons:=vbOKOnly + vbExclamation, Title:=sTitle)
End Sub
' --------------------------------------------------------------------
Public Sub MsgInfo(sMsg As String, sTitle As String, Optional bBeep As Boolean = True)
   If bBeep Then Beep
   Call MsgBox(prompt:=sMsg, Buttons:=vbOKOnly + vbInformation, Title:=sTitle)
End Sub
' --------------------------------------------------------------------
Public Sub MsgFNF(sFile As String, sTitle As String, Optional bBeep As Boolean = True)
   If bBeep Then Beep
   Call MsgBox(prompt:="File not found." + vbCrLf + vbSQuote + sFile + vbSQuote, Buttons:=vbOKOnly + v
bCritical, Title:=sTitle)
End Sub
' --------------------------------------------------------------------
Public Sub MsgDNF(sDir As String, sTitle As String, Optional bBeep As Boolean = True)
   If bBeep Then Beep
   Call MsgBox(prompt:="Directory not found." + vbCrLf + vbSQuote + sDir + vbSQuote, Buttons:=vbOKOnly
 + vbCritical, Title:=sTitle)
End Sub
' --------------------------------------------------------------------
Public Function AskYN(sMsg As String, sTitle As String, Optional bBeep As Boolean = True) As Boolean
   If bBeep Then Beep
   AskYN = MsgBox(prompt:=sMsg, Buttons:=vbYesNo + vbDefaultButton1 + vbQuestion, Title:=sTitle) = vbY
es
End Function
' --------------------------------------------------------------------
Public Function AskNY(sMsg As String, sTitle As String, Optional bBeep As Boolean = True) As Boolean
   If bBeep Then Beep
   AskNY = MsgBox(prompt:=sMsg, Buttons:=vbYesNo + vbDefaultButton2 + vbQuestion, Title:=sTitle) = vbY
es
End Function
' ====================================================================


' ====================================================================
' ====================================================================
```

```
' ====================================================================
'                            EU_XLS_Lib.bas
' ====================================================================
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  Library with generic Excel-related functions
' Uses:    EU_WIN_Lib
' History: 02-02-2016 Ronald Bax     First version
'          30-01-2019 Ronald Bax     Update
'          02-06-2021 Ronald Bax     VLookupBackToo:  .Find added LookIn:=xlValues
' ====================================================================
' Note:    This Lib is dependent of EU_WIN_Lib
' ====================================================================
' Public  Function GetVersion() As String
' Public  Function GetDTUpdate() As Date
' Public  Function LibFileExists(sLibDir As String) As Boolean
' Public  Function GetLibFileDate(sLibDir As String) As Date
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Public  Sub     SetDebugModeOn(ByVal bSwitchOn As Boolean)
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Private Sub     ListAllLibs()
' Private Sub     AddLib(sLibName As String, sGUID As String, ByVal iMajor As Long, ByVal iMinor As L
ong)
' Public  Sub     addVBIDEExt()
' Public  Sub     addVBINetCtrls()
' Public  Sub     addVBHTMLObjLib()
' Public  Sub     addVBRegExpLib()
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Public  Function ActivateAndUnprotect(aWks as Worksheet) As Boolean
' Public  Sub     ProtectSheet(aWks As Worksheet)
' Public  Sub     ProtectAllWorksheets(ByVal bProtect As Boolean)
' Public  Function SetScreenUpdates(ByVal bSwitchOn As Boolean) As Boolean
' Public  Sub     ShowProcessing(ByVal bSwitchOn As Boolean)
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Public  Function Workbook_Open_AutoBackup(aWks As Worksheet, ByVal bWantBackup As Boolean, Optional
sSelCell As String = vbNullString) As String
' Public  Function TestIfImOnlyActiveWorkbook(Optional ByVal bShowMsg As Boolean = True) As Boolean
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Public  Function RangeTextToRange(sRngText As String) As Range
' Public  Function RangeToRangeText(aRng As Range, Optional ByVal bLong As Boolean = True) As String
' Public  Function Sheetname() As String
' Public  Function NameToWorksheet(sWksName As String) As Worksheet
' Public  Sub     SetStdWksCodeNames()
' Public  Sub     ListDefinedNames(Optional aRng As Range = Nothing, Optional sSepChar As String = ",
")
' Public  Sub     CleanupDefinedNames()
' Public  Sub     ListWksNames(Optional aRng As Range = Nothing, Optional sSepChar As String = ",")
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Public  Function RFill(sString As String, sChar As String, iLen As Integer) As String
' Public  Function LFill(sString As String, sChar As String, iLen As Integer) As String
' Public  Function ColTxt2Num(sColTxt As String) As Long
' Public  Function ColNum2Txt(ByVal iColNum As Long) As String
' Public  Function LastCell(aWks As Worksheet) As Range
' Public  Function LastRow(aWks As Worksheet) As Long
' Public  Function LastCol(aWks As Worksheet) As Long
' Public  Function FromCellToEnd(aWks As Worksheet, sCell As String) As Range
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Private Sub     CircleCellWithOval(ByVal oCell As Range)
' Public  Sub     UnlockedCellsClear(aWks As Worksheet)
' Public  Sub     UnlockedCellsShowOval(aWks As Worksheet)
' Public  Sub     RemoveOvalsFromWks(aWks As Worksheet)
' Public  Sub     InvertRangeColor(aRng As Range)
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Private Sub     AddActCircle(aWks As Worksheet, Optional aRng As Range, Optional aBtn As Shape = No
thing)
' Public  Sub     DelActCircles(aWks As Worksheet)
' Public  Sub     AddActCircleToCell(aWks As Worksheet, aRng As Range, Optional ByVal bDelOld As Bool
ean = False)
' Public  Sub     AddActCircleToBtn(aWks As Worksheet, aBtn As Shape, Optional ByVal bDelOld As Boole
an = False)
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Public  Sub     RemoveConditionalFormatting(aRng As Range)
' Public  Sub     AddConditionalFormatting(aRng As Range, sFormula As String, ByVal iColor As Long, O
ptional ByVal iFontColor As Long = 0, Optional ByVal bStopIfTrue As Boolean = False)
```

```
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Public  Sub      MakeAllWksVisible()
' Public  Sub      ClearSheet(aWks As Worksheet, Optional sCell As String = "A1", Optional sRowCol As
String = "row")
' Public  Sub      ScrollTopLeftInView(aWks As Worksheet)
' Public  Sub      AutoFitWorksheet(aWks As Worksheet)
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Public  Function CreateSheetIfNotPresent(wksName As String, Optional ByVal bEmpty As Boolean = False
) As Worksheet
' Public  Function VLookupBackToo(sSearch As String, aRng As Range, ByVal iColumn As Long, ByVal iResC
olumn As Long) As String
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Public  Function SaveRangeToTXTFile(aRng As Range, sFilePath As String, Optional sSepChar As String
= ",") As Boolean
' Public  Function SaveSheetToTXTFile(aWks As Worksheet, sFilePath As String, Optional sSepChar As Str
ing = ",") As Boolean
' Public  Sub      ExportRangeToHTML(sFilePath As String, aRng As Range)
' Public  Sub      SetHTMLBackgroundColor(sFilePath As String, ByVal iColor As Long)
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Public  Sub      FormatColumn(aWks As Worksheet, sRngStart As String, sFormat As String)
' Public  Sub      SetStyleIsCurrency(aRng As Range)
' Public  Sub      SetValidation(aWks As Worksheet, sCol As String, iStartRow As Long, sLookupRange As
 String, Optional iEndRow As Long = 0)
' Public  Sub      FillWithFormula(aWks As Worksheet, sRngStart As String, sFormula As String)
' Public  Sub      FillRowDownward(ByVal rngFrom As Range)
' Public  Sub      DrawBorders(ByVal rng As Range, ByVal bOnOff As Boolean)
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Public  Sub      PivotTableFormat(aPivTab As PivotTable)
' Public  Sub      PivotFiltersRemove (aPivTab As PivotTable)
' Public  Sub      PivotFiltersNotBlankEmpty (aPivTab As PivotTable, sPivFieldName AS String)
' Public  Sub      PivotClearBelow (aPivTab As PivotTable) as Boolean
' Public  Sub      AutoFiltersRemove(aWks As Worksheet)
' Public  Sub      AutoFiltersAdd(aWks As Worksheet, Optional ByVal iTopRows As Integer = 1)
' Public  Sub      SortSheetWithOneHeaderRow(aWks As Worksheet, sColumns As String)
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Public  Sub      HeaderFooterClearWks(aWks As Worksheet)
' Public  Sub      HeaderFooterClearAll()
' Public  Sub      HeaderFooterAddStandardWks(aWks As Worksheet)
' Public  Sub      HeaderFooterAddStandardAll()
' Public  Sub      PageInLandscapeWks(aWks As Worksheet)
' Public  Sub      PageInLandscapeAll()
' Public  Sub      PageInPortraitWks(aWks As Worksheet)
' Public  Sub      PageInPortraitAll()
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Public  Sub      GetUniqueValues(aRng As Range, aTargetCell As Range)
' ===========================================================================
Option Explicit

' ===========================================================================
'                           Constants and Types
' ===========================================================================
Private Const EU_XLS_Lib_LVersion As String = "3.01"
Private Const EU_XLS_Lib_DTUpdate As String = "02-06-2021"
Private Const EU_XLS_Lib_FileName As String = "EU_XLS_Lib.bas"
Private Const GcActCircle = "ActCircle"
' ===========================================================================

' ===========================================================================
'                                GLOBALS
' ===========================================================================
Public GbDebugMode As Boolean
' ===========================================================================

' ---------------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  Get info about this library
' Uses:    .
' Param:   .
' History: 02-02-2016 Ronald Bax     First version
'          02-02-2019 Ronald Bax     2.00 Update
' ---------------------------------------------------------------------------
Public Function GetVersion() As String
   GetVersion = EU_XLS_Lib_LVersion
```

```
EU_XLS_Lib - 3

End Function
' ---------------------------------------------------------------------
Public Function GetDTUpdate() As Date
   GetDTUpdate = CDate(EU_XLS_Lib_DTUpdate)
End Function
' ---------------------------------------------------------------------
Public Function LibFileExists(sLibDir As String) As Boolean
   LibFileExists = EU_WIN_Lib.FileExists(sFilePath:=sLibDir & EU_XLS_Lib_FileName)
End Function
' ---------------------------------------------------------------------
Public Function GetLibFileDate(sLibDir As String) As Date
   GetLibFileDate = EU_WIN_Lib.GetFileDateTime(sFilePath:=sLibDir & EU_XLS_Lib_FileName)
End Function
' =====================================================================

' ---------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  Switch Debug mode on:
'          Functions will output results to "Direct" window in VB for Applications
' Uses:    n.a.
' Param:   bSwitchOn = True:  Debugmode is on
' History: 02-02-2016 Ronald Bax     First version
'          30-01-2019 Ronald Bax     Update
' ---------------------------------------------------------------------
Public Sub SetDebugModeOn(ByVal bSwitchOn As Boolean)
   GbDebugMode = bSwitchOn
End Sub
' =====================================================================

' ---------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 05-07-2016 Ronald Bax     First version
' ---------------------------------------------------------------------
Private Sub ListAllLibs()
   Dim oExcel As Object: Set oExcel = GetObject(, "Excel.Application")
   Dim vbProj As Object: Set vbProj = oExcel.ActiveWorkbook.VBProject
   Dim chkRef As Object
   For Each chkRef In vbProj.References
      Debug.Print chkRef.Name & vbSpace & chkRef.major & "." & chkRef.minor & chkRef.guid & "'" & chkR
ef.Description & "'"
   Next chkRef
   Set chkRef = Nothing
   Set vbProj = Nothing
   Set oExcel = Nothing
End Sub
' =====================================================================

' ---------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 02-02-2016 Ronald Bax     First version
'          30-01-2019 Ronald Bax     Update
' ---------------------------------------------------------------------
Private Sub AddLib(sLibName As String, sGUID As String, ByVal iMajor As Long, ByVal iMinor As Long)
   Const cMe = "EU_XLS_Lib.AddLib"
   Dim oExcel As Object: Set oExcel = GetObject(, "Excel.Application")
   Dim vbProj As Object: Set vbProj = oExcel.ActiveWorkbook.VBProject
   Dim chkRef As Object
   ' Remove any broken references
   For Each chkRef In vbProj.References
      If chkRef.Name = sLibName Then
         If chkRef.IsBroken Then vbProj.References.Remove chkRef
      End If
   Next
   For Each chkRef In vbProj.References ' Check if the library has already been added
      If chkRef.Name = sLibName Then GoTo Cleanup
   Next
   vbProj.References.AddFromGuid sGUID, iMajor, iMinor ' If not, Add it now
```

```
   For Each chkRef In vbProj.References ' Check if the library has now been added
      If chkRef.Name = sLibName Then
         MsgBox "Reference added: " & chkRef.Name & vbSpace & chkRef.major & "." & chkRef.minor & vbCr
Lf & "'" & chkRef.Description & "'", vbInformation + vbOKOnly, cMe
      End If
   Next
Cleanup:
   Set chkRef = Nothing
   Set vbProj = Nothing
   Set oExcel = Nothing
End Sub
' ===============================================================================

' -------------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  Add the "Microsoft Visual Basic for Applications Extensibility 5.3"
'          Add the "Microsoft Internet Controls"
'          Add the "Microsoft HTML Object Library"
'          Add the "Microsoft VBScript Regular Expressions 5.5"
' Uses:    EU_XLS_Lib.AddLib()
' Param:   ...
' History: 02-02-2016 Ronald Bax     First version
'          30-01-2019 Ronald Bax     Update
' -------------------------------------------------------------------------
Public Sub addVBIDEExt()
   Call EU_XLS_Lib.AddLib(sLibName:="VBIDE", sGUID:="{0002E157-0000-0000-C000-000000000046}", iMajor:=
5, iMinor:=3)
End Sub
' -------------------------------------------------------------------------
Public Sub addVBINetCtrls()
   Call EU_XLS_Lib.AddLib(sLibName:="SHDocVw", sGUID:="{EAB22AC0-30C1-11CF-A7EB-0000C05BAE0B}", iMajor
:=1, iMinor:=1)
End Sub
' -------------------------------------------------------------------------
Public Sub addVBHTMLObjLib()
   Call EU_XLS_Lib.AddLib(sLibName:="MSHTML", sGUID:="{3050F1C5-98B5-11CF-BB82-00AA00BDCE0B}", iMajor:
=4, iMinor:=0)
End Sub
' -------------------------------------------------------------------------
Public Sub addVBRegExpLib()
   Call EU_XLS_Lib.AddLib(sLibName:="VBScript_RegExp_55", sGUID:="{3F4DACA7-160D-11D2-A8E9-00104B365C9
F}", iMajor:=5, iMinor:=5)
End Sub
' ===============================================================================

' -------------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  Protcet/Unprotect a sheet with standard settings
' Uses:    n.a.
' Param:   aWks:   The Worksheet to Protect/Unprotect
' Returns: Boolean If the sheet was protceted before
' History: 05-07-2016 Ronald Bax     First version
' -------------------------------------------------------------------------
Public Function ActivateAndUnprotect(aWks As Worksheet) As Boolean
   If aWks Is Nothing Then Exit Function ' Ready if aWks is not valid
   With aWks
      ActivateAndUnprotect = .ProtectContents
      If .ProtectContents Then
         .Activate
         .Unprotect
      End If
   End With
End Function
' -------------------------------------------------------------------------
Public Sub ProtectSheet(aWks As Worksheet)
   If aWks Is Nothing Then Exit Sub ' Ready if aWks is not valid
   aWks.Protect DrawingObjects:=True, Contents:=True, Scenarios:=True, AllowFiltering:=True, AllowUsin
gPivotTables:=True
End Sub
' ===============================================================================

' -------------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
```

```
' Action:  Protect all worksheets in the ActiveWorkbook (on or off)
' Uses:    n.a.
' Param:   bProtect=True Protect, bProtect=False Unprotect
' History: 02-02-2016 Ronald Bax     First version
'          30-01-2019 Ronald Bax     Update
' ----------------------------------------------------------------------
Public Sub ProtectAllWorksheets(ByVal bProtect As Boolean)
   Dim wks As Worksheet
   For Each wks In ActiveWorkbook.Worksheets
      If bProtect Then
         Call EU_XLS_Lib.ProtectSheet(aWks:=wks)
      Else
         Call EU_XLS_Lib.ActivateAndUnprotect(aWks:=wks)
      End If
   Next wks
   Set wks = Nothing
End Sub
' ======================================================================

' ----------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  Show/Hide all processing and show the correct cursor
' Uses:    n.a.
' Param:   bSwitchOn=False Prevent screenUpdates and show 'hourglass' cursor
'          bSwitchOn=True  Allow   screenUpdates and show 'normal' cursor
' History: 02-02-2016 Ronald Bax     First version
'          30-01-2019 Ronald Bax     Update
' ----------------------------------------------------------------------
Public Function SetScreenUpdates(ByVal bSwitchOn As Boolean) As Boolean
   Application.Cursor = IIf(bSwitchOn, xlDefault, xlWait)
   If bSwitchOn Then ActiveWorkbook.RefreshAll
   SetScreenUpdates = Application.ScreenUpdating
   Application.ScreenUpdating = bSwitchOn
End Function
' ======================================================================

' ----------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  Show/Hide all processing and Protect/Unprotect all sheets
' Uses:    EU_XLS_Lib.ScrollTopLeftInView()
'          EU_XLS_Lib.ProtectAllWorksheets(True)
' Param:   bSwitchOn=False Prevent screenUpdates and show 'hourglass' cursor
'          bSwitchOn=True  Allow   screenUpdates and show 'normal'    cursor
' History: 02-02-2016 Ronald Bax     First version
'          30-01-2019 Ronald Bax     Update
' ----------------------------------------------------------------------
Public Sub ShowProcessing(ByVal bSwitchOn As Boolean)
   Dim wks As Worksheet
   With Application
      If bSwitchOn Then ' Switch ON Excel settings to original state.
         For Each wks In .Worksheets ' Note: this is a sheet-level setting
            wks.Activate
            wks.DisplayPageBreaks = bSwitchOn
            Call EU_XLS_Lib.ScrollTopLeftInView(aWks:=wks)
         Next wks
         Call EU_XLS_Lib.SetScreenUpdates(bSwitchOn:=bSwitchOn)
         Call EU_XLS_Lib.ProtectAllWorksheets(bProtect:=bSwitchOn)
         .DisplayStatusBar = bSwitchOn
         .Calculation = xlAutomatic
         .CalculateFull
         .EnableEvents = bSwitchOn
      Else ' Turn OFF Excel functionality to improve performance.
         .EnableEvents = bSwitchOn
         .Calculation = xlCalculationManual
         .DisplayStatusBar = bSwitchOn
         Call EU_XLS_Lib.SetScreenUpdates(bSwitchOn:=bSwitchOn)
         Call EU_XLS_Lib.ProtectAllWorksheets(bProtect:=bSwitchOn)
         For Each wks In .Worksheets ' Note: this is a sheet-level setting.
            wks.Activate
            wks.DisplayPageBreaks = bSwitchOn
         Next wks
      End If
   End With
```

```
   Set wks = Nothing
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  ...
' Uses:    EU_WIN_Lib.Pause()
'          EU_WIN_Lib.saveOldVersionOfFile()
' Param:   ...
' History: 02-02-2016 Ronald Bax     First version
'          30-01-2019 Ronald Bax     Update
' --------------------------------------------------------------------
Public Function Workbook_Open_AutoBackup(aWks As Worksheet, ByVal bWantBackup As Boolean, Optional sSe
lCell As String = vbNullString) As String
   Dim bSvProtStat As Boolean
   Dim sRes        As String
   sRes = vbNullString
   On Error GoTo Anyway
   Call EU_WIN_Lib.Pause(iSeconds:=1)
   If Application.ActiveWorkbook Is Nothing Then
      sRes = "Workbook inactive."
   Else
      If Not aWks Is Nothing Then
         With aWks
            bSvProtStat = EU_XLS_Lib.ActivateAndUnprotect(aWks:=aWks)
            If bWantBackup Then
               sRes = IIf(EU_WIN_Lib.SaveOldVersionOfFile(sFilePath:=.Application.ThisWorkbook.FullNam
e, bShowMsg:=False) = True, Now, "Backup failed.")
            Else
               sRes = "Backup is off."
            End If
            If sSelCell <> vbNullString Then .Range(sSelCell).Activate
            If bSvProtStat Then Call EU_XLS_Lib.ProtectSheet(aWks:=aWks)
         End With
      End If
   End If
Anyway:
   Workbook_Open_AutoBackup = sRes
End Function
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 02-02-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Public Function TestIfImOnlyActiveWorkbook(Optional ByVal bShowMsg As Boolean = True) As Boolean
   Const cMe = "EU_XLS_Lib.TestIfImOnlyActiveWorkbook"
   TestIfImOnlyActiveWorkbook = (Application.Workbooks.Count <> 1)
   If bShowMsg And TestIfImOnlyActiveWorkbook Then MsgBox "Please close all other workbooks, before op
ening this one!", vbCritical + vbOKOnly, cMe
End Function
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 30-01-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Public Function Sheetname() As String
   Sheetname = vbNullString
   On Error Resume Next
   Sheetname = Application.ThisCell.Worksheet.Name
   On Error GoTo 0
End Function
' ====================================================================

' --------------------------------------------------------------------
```

```
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:
' Uses:
' Param:   sWksName
' History: 30-01-2019 Ronald Bax     First version
' ---------------------------------------------------------------------
Public Function NameToWorksheet(sWksName As String) As Worksheet
   Const cMe = "EU_XLS_Lib.NameToWorksheet"
   Dim wks    As Worksheet
   Dim SelWks As Worksheet
   For Each wks In ActiveWorkbook.Worksheets
      If StrComp(wks.Name, sWksName, vbTextCompare) = 0 Or StrComp(wks.CodeName, sWksName, vbTextCompa
re) = 0 Then
         Set SelWks = wks
         Exit For
      End If
   Next wks
   If SelWks Is Nothing Then
      MsgBox "Worksheet '" & sWksName & "' does not exist in ActiveWorkbook '" & ActiveWorkbook.Name &
 "'.", vbOKOnly, cMe
   End If
   Set NameToWorksheet = SelWks
   Set SelWks = Nothing
   Set wks = Nothing
End Function
' =====================================================================

' ---------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:
' Uses:
' Param:   sWksName
' History: 30-01-2019 Ronald Bax     First version
' ---------------------------------------------------------------------
Public Function RangeTextToRange(sRngText As String) As Range
   Dim S As String
   Dim R As String
   Dim I As Integer
   Dim J As Integer
   R = sRngText
   I = InStr(1, R, "!", vbTextCompare)
   If I > 0 Then
      S = Left(R, I - 1)
      J = InStr(1, S, "]", vbTextCompare)
      If J > 0 Then S = Mid(S, J + 1, Len(S) - J - 1)
      R = Mid(R, I + 1, Len(R) - I)
   Else
      S = ActiveSheet.Name
   End If
   On Error Resume Next
   Set RangeTextToRange = Worksheets(S).Range(R)
End Function
' =====================================================================

' ---------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:
' Uses:
' Param:   sWksName
' History: 30-01-2019 Ronald Bax     First version
' ---------------------------------------------------------------------
Public Function RangeToRangeText(aRng As Range, Optional ByVal bLong As Boolean = True) As String
   Dim I As Integer
   If aRng Is Nothing Then Exit Function ' Ready if aRng is not valid
   RangeToRangeText = vbNullString
   RangeToRangeText = aRng.Worksheet.Parent.FullName
   I = InStrRev(RangeToRangeText, Application.PathSeparator, , vbTextCompare)
   If bLong And I > 0 Then
      RangeToRangeText = "'" & Left(RangeToRangeText, I) & "[" & Mid(RangeToRangeText, I + 1, Len(Rang
eToRangeText) - 1) & "]"
   Else
      RangeToRangeText = "'" & "[" & aRng.Worksheet.Parent.Name & "]"
   End If
```

```
   RangeToRangeText = RangeToRangeText & aRng.Worksheet.Name & "'!" & aRng.Address
End Function
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 07-03-2021 Ronald Bax     First version
' --------------------------------------------------------------------
Public Function RFill(sString As String, sChar As String, iLen As Integer) As String
   Dim C As String * 1
   C = IIf(sChar = vbNullString, vbSpace, Left(sChar, 1))
   If Len(sString) >= iLen Then
      RFill = Left(sString, iLen)
   Else
      If Len(sString) > 0 And iLen > 0 Then RFill = sString & String(iLen - Len(sString), C)
   End If
End Function
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 07-03-2021 Ronald Bax     First version
' --------------------------------------------------------------------
Public Function LFill(sString As String, sChar As String, iLen As Integer) As String
   Dim C As String * 1
   C = IIf(sChar = vbNullString, vbSpace, Left(sChar, 1))
   If Len(sString) >= iLen Then
      LFill = Right(sString, iLen)
   Else
      If Len(sString) > 0 And iLen > 0 Then LFill = String(iLen - Len(sString), C) & sString
   End If
End Function
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 02-02-2019 Ronald Bax     First version
'          14-03-2021 Ronald Bax     Rewrite
' --------------------------------------------------------------------
Public Function ColTxt2Num(sColTxt As String) As Long
   On Error Resume Next
   ColTxt2Num = Columns(sColTxt).Column
   If Err.Number <> 0 Then ColTxt2Num = 0
   On Error GoTo 0
End Function
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 02-02-2019 Ronald Bax     First version
'          14-03-2021 Ronald Bax     Rewrite
' --------------------------------------------------------------------
Public Function ColNum2Txt(ByVal iColNum As Long) As String
   Dim S As String
   On Error Resume Next
   S = Columns(iColNum).Address
   If Err.Number <> 0 Then ColNum2Txt = vbNullString
   On Error GoTo 0
   If InStr(S, ":") <> 0 Then ColNum2Txt = Trim(Replace(Left(S, InStr(S, ":") - 1), "$", vbNullString)
)
End Function
```

```
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  Return the last cell of the sheet (bottom-right-cell)
' Uses:    n.a.
' Param:   aWks       The worksheet to research
' History: 02-02-2016 Ronald Bax     First version
'          30-01-2019 Ronald Bax     Update
'          14-03-2021 Ronald Bax     Rewrite
' --------------------------------------------------------------------
Public Function LastCell(aWks As Worksheet) As Range
   Dim rng         As Range
   Dim oSvActSheet As Worksheet
   Dim bSvProtStat As Boolean
   Dim bSvUpdtStat As Boolean
   Dim iLR         As Long
   Dim iLC         As Long
   If aWks Is Nothing Then Exit Function ' Ready if aWks is not valid
   bSvUpdtStat = Application.ScreenUpdating
   Application.ScreenUpdating = False
   Set oSvActSheet = ActiveSheet
   bSvProtStat = EU_XLS_Lib.ActivateAndUnprotect(aWks:=aWks)
   With aWks
      On Error Resume Next
      iLR = .Cells.Find(What:="*", After:=.Range("A1"), LookAt:=xlPart, LookIn:=xlFormulas, SearchOrde
r:=xlByRows, SearchDirection:=xlPrevious, MatchCase:=False).Row
      iLC = .Cells.Find(What:="*", After:=.Range("A1"), LookAt:=xlPart, LookIn:=xlFormulas, SearchOrde
r:=xlByColumns, SearchDirection:=xlPrevious, MatchCase:=False).Column
      On Error GoTo 0
      If iLR = 0 Then iLR = 1
      If iLC = 0 Then iLC = 1
      Set rng = .Cells(iLR, iLC)
   End With
   oSvActSheet.Activate
   If bSvProtStat Then Call EU_XLS_Lib.ProtectSheet(aWks:=aWks)
   Application.ScreenUpdating = bSvUpdtStat
   Set LastCell = rng
   Set oSvActSheet = Nothing
   Set rng = Nothing
End Function
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  Return the last Row-number of the sheet (bottom-right-cell)
' Uses:    n.a.
' Param:   aWks       The worksheet to research
' History: 02-02-2016 Ronald Bax     First version
'          30-01-2019 Ronald Bax     Update
'          14-03-2021 Ronald Bax     Rewrite
' --------------------------------------------------------------------
Public Function LastRow(aWks As Worksheet) As Long
   On Error Resume Next
   LastRow = EU_XLS_Lib.LastCell(aWks:=aWks).Row
   If Err.Number <> 0 Then LastRow = 0
   On Error GoTo 0
End Function
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  Return the last Column-number of the sheet (bottom-right-cell)
' Uses:    n.a.
' Param:   aWks       The worksheet to research
' History: 02-02-2016 Ronald Bax     First version
'          30-01-2019 Ronald Bax     Update
'          14-03-2021 Ronald Bax     Rewrite
' --------------------------------------------------------------------
Public Function LastCol(aWks As Worksheet) As Long
   On Error Resume Next
   LastCol = EU_XLS_Lib.LastCell(aWks:=aWks).Column
   If Err.Number <> 0 Then LastCol = 0
```

```
   On Error GoTo 0
End Function
' ===========================================================================

' ---------------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  Return a Range from sCell to the last Cell of the sheet (bottom-right-cell)
' Uses:    n.a.
' Param:   aWks       The worksheet to research
'          sCell      The cell to start with in "A1"-format
' History: 02-02-2016 Ronald Bax     First version
'          30-01-2019 Ronald Bax     Update
'          14-03-2021 Ronald Bax     Rewrite
' ---------------------------------------------------------------------------
Public Function FromCellToEnd(aWks As Worksheet, sCell As String) As Range
   Dim rng         As Range
   If sCell = vbNullString Then sCell = "A1"
   With aWks
      Set rng = .Range(.Range(sCell), EU_XLS_Lib.LastCell(aWks:=aWks))
      If rng Is Nothing Then Set rng = .Range(sCell)
      If EU_XLS_Lib.LastRow(aWks:=aWks) < .Range(sCell).Row Then
         Set rng = .Range(.Range(sCell), Cells(.Range(sCell).Row, EU_XLS_Lib.LastCol(aWks:=aWks)))
      End If
   End With
   Set FromCellToEnd = rng
   Set rng = Nothing
End Function
' ===========================================================================

' ---------------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 06-04-2019 Ronald Bax     First version
' ---------------------------------------------------------------------------
Private Sub CircleCellWithOval(aRng As Range)
   If aRng Is Nothing Then Exit Sub ' Ready if aRng is not valid
   With aRng.Worksheet.Shapes.AddShape(msoShapeOval, .Left - (.Width / 6), .Top - (.Height / 6), .Widt
h + (.Width / 3), .Height + (.Height / 3))
      .LockAspectRatio = msoTrue
      .Line.Weight = 0.25
      .Line.ForeColor.RGB = RGB(179, 129, 218)
      .Line.BackColor.RGB = vbWhite
      .Fill.Transparency = 1
   End With
End Sub
' ===========================================================================

' ---------------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 06-04-2019 Ronald Bax     First version
' ---------------------------------------------------------------------------
Public Sub UnlockedCellsClear(aWks As Worksheet)
   Dim rng         As Range
   Dim bSvProtStat As Boolean
   If aWks Is Nothing Then Exit Sub ' Ready if aWks is not valid
   bSvProtStat = EU_XLS_Lib.ActivateAndUnprotect(aWks:=aWks)
   With aWks
      Set rng = EU_XLS_Lib.LastCell(aWks:=aWks)
      For Each rng In Range(.Range("A1"), rng)
         If Not rng.Locked Then rng.Value2 = vbNullString
      Next rng
   End With
   If bSvProtStat Then Call EU_XLS_Lib.ProtectSheet(aWks:=aWks)
   Set rng = Nothing
End Sub
' ===========================================================================

' ---------------------------------------------------------------------------
```

```
' Author:  Ronald Bax          Version: 3.01          Date: 02-06-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 06-04-2019 Ronald Bax     First version
' ---------------------------------------------------------------------
Public Sub UnlockedCellsShowOval(aWks As Worksheet)
   Dim rng         As Range
   Dim bSvProtStat As Boolean
   If aWks Is Nothing Then Exit Sub ' Ready if aWks is not valid
   bSvProtStat = EU_XLS_Lib.ActivateAndUnprotect(aWks:=aWks)
   With aWks
      Call EU_XLS_Lib.RemoveOvalsFromWks(aWks:=aWks)
      Set rng = EU_XLS_Lib.LastCell(aWks:=aWks)
      For Each rng In Range(.Range("A1"), rng)
         If Not rng.Locked Then Call EU_XLS_Lib.CircleCellWithOval(aRng:=rng)
      Next rng
   End With
   If bProt Then Call EU_XLS_Lib.ProtectSheet(aWks:=aWks)
   Set rng = Nothing
End Sub
' =====================================================================

' ---------------------------------------------------------------------
' Author:  Ronald Bax          Version: 3.01          Date: 02-06-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 06-04-2019 Ronald Bax     First version
' ---------------------------------------------------------------------
Public Sub RemoveOvalsFromWks(aWks As Worksheet)
   Dim shp As Shape
   If aWks Is Nothing Then Exit Sub ' Ready if aWks is not valid
   For Each shp In aWks.Shapes
      If shp.AutoShapeType = msoShapeOval Then shp.Delete
   Next shp
   Set shp = Nothing
End Sub
' =====================================================================

' ---------------------------------------------------------------------
' Author:  Ronald Bax          Version: 3.01          Date: 02-06-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 06-04-2019 Ronald Bax     First version
' ---------------------------------------------------------------------
Public Sub InvertRangeColor(aRng As Range)
    Dim lColorBack As Long
    Dim lColorFont As Long
    With aRng
       lColorBack = .Interior.Color
       lColorFont = .Font.Color
       .Interior.Color = lColorFont
       .Font.Color = lColorBack
       If .Interior.Color = 16777215 Then  ' Put the default border back
          .Style = "Normal"
          .Locked = False
       End If
   End With
End Sub
' =====================================================================

' ---------------------------------------------------------------------
' Author:  Ronald Bax          Version: 3.01          Date: 02-06-2021
' Action:
' Uses:    n.a.
' Param:
' History: 05-06-2016 Ronald Bax     First version
' ---------------------------------------------------------------------
Public Sub DelActCircles(aWks As Worksheet)
   Dim shp As Shape
   If aWks Is Nothing Then Exit Sub ' Ready if aWks is not valid
```

```
   For Each shp In aWks.Shapes
      If StrComp(Left(shp.Name, Len(GcActCircle)), GcActCircle, vbTextCompare) = 0 Then shp.Delete
   Next shp
   Set shp = Nothing
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:
' Uses:    n.a.
' Param:
' History: 05-06-2016 Ronald Bax     First version
' --------------------------------------------------------------------
Public Sub AddActCircleToCell(aWks As Worksheet, aRng As Range, Optional ByVal bDelOld As Boolean = Fa
lse)
   If aWks Is Nothing Or aRng Is Nothing Then Exit Sub ' Ready if aWks or aRng is not valid
   If bDelOld Then Call EU_XLS_Lib.DelActCircles(aWks:=aWks)
   Call EU_XLS_Lib.AddActCircle(aWks:=aWks, aRng:=aRng, aBtn:=Nothing)
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:
' Uses:    n.a.
' Param:
' History: 05-06-2016 Ronald Bax     First version
' --------------------------------------------------------------------
Public Sub AddActCircleToBtn(aWks As Worksheet, aBtn As Shape, Optional ByVal bDelOld As Boolean = Fal
se)
   If aWks Is Nothing Then Exit Sub ' Ready if aWks is not valid
   If bDelOld Then Call EU_XLS_Lib.DelActCircles(aWks:=aWks)
   Call EU_XLS_Lib.AddActCircle(aWks:=aWks, aRng:=Nothing, aBtn:=aBtn)
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:
' Uses:    n.a.
' Param:
' History: 05-06-2016 Ronald Bax     First version
' --------------------------------------------------------------------
Public Sub RemoveConditionalFormatting(aRng As Range)
   If aRng Is Nothing Then Exit Sub ' Ready if aRng is not valid
   aRng.FormatConditions.Delete
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:
' Uses:    n.a.
' Param:
' History: 05-06-2016 Ronald Bax     First version
' --------------------------------------------------------------------
Public Sub AddConditionalFormatting(aRng As Range, sFormula As String, ByVal iColor As Long, Optional
ByVal iFontColor As Long = 0, Optional ByVal bStopIfTrue As Boolean = False)
   Dim bSvScrnUpd As Boolean
   Dim idx        As Integer
   If aRng Is Nothing Then Exit Sub ' Ready if aRng is not valid
   bSvScrnUpd = EU_XLS_Lib.SetScreenUpdates(bSwitchOn:=False)
   With aRng
      .FormatConditions.Add Type:=xlExpression, Formula1:=sFormula
      idx = .FormatConditions.Count
      With .FormatConditions(idx)
         .SetFirstPriority
         .Font.Color = iFontColor
         With .Interior
            .PatternColorIndex = xlAutomatic
            .Color = iColor
            .TintAndShade = 0
```

```
         End With
         .StopIfTrue = bStopIfTrue
      End With
   End With
   Call EU_XLS_Lib.SetScreenUpdates(bSwitchOn:=bSvScrnUpd)
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:
' Uses:    n.a.
' Param:
' History: 05-06-2016 Ronald Bax     First version
' --------------------------------------------------------------------
Private Sub AddActCircle(aWks As Worksheet, Optional aRng As Range, Optional aBtn As Shape = Nothing)
   Dim rng   As Range
   Dim shp   As Shape
   Dim idx   As Integer
   Dim nam   As String
   Dim cLeft As Long
   Dim cTop  As Long
   If aWks Is Nothing Then Exit Sub ' Ready if aWks is not valid
   If aRng Is Nothing And aBtn Is Nothing Then
      'msgbox ....
      Exit Sub
   End If
   If Not aRng Is Nothing And Not aBtn Is Nothing Then
      'msgbox ....
      Exit Sub
   End If
   cLeft = 0
   cTop = 0
   Set rng = Selection
   If Not aRng Is Nothing Then
      Set rng = aRng
      cLeft = aRng.Left - 2
      cTop = aRng.Top - 2
   Else
      If Not aBtn.Type = msoFormControl Or Not aBtn.FormControlType = xlButtonControl Then
         'msgbox ....
         Exit Sub
      End If
      cLeft = aBtn.Left - 1
      cTop = aBtn.Top - 1
   End If

   With aWks
      idx = 0
      For Each shp In .Shapes
         If StrComp(Left(shp.Name, Len(GcActCircle)), GcActCircle, vbTextCompare) = 0 Then idx = idx +
 1
      Next shp
      idx = idx + 1
      nam = GcActCircle & Right("000" & idx, 3)
      .Shapes.AddShape(Type:=msoShapeOval, Left:=cLeft, Top:=cTop, Width:=10, Height:=10).Name = nam
      With .Shapes(nam)
         .Line.Visible = msoFalse
         .ShapeStyle = msoShapeStylePreset36
         With .Fill
            .Visible = msoTrue
            .ForeColor.RGB = RGB(240, 0, 0)
            .Transparency = 0
            .Solid
         End With
         With .ThreeD
            .ContourColor = RGB(255, 255, 155)
            .ExtrusionColor = RGB(255, 255, 155)
            .BevelTopType = msoBevelCircle
            .BevelTopInset = 1
            .BevelTopDepth = 1
            .PresetMaterial = msoMaterialMetal2
            .PresetLighting = msoLightRigSunset
```

```
         End With
      End With
   End With
   rng.Activate
   Set shp = Nothing
   Set rng = Nothing
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  Make all worksheets in the ActiveWorkbook visible
' Uses:    n.a.
' Param:   n.a.
' History: 02-02-2016 Ronald Bax     First version
'          30-01-2019 Ronald Bax     Update
' --------------------------------------------------------------------
Public Sub MakeAllWksVisible()
   Dim wks As Worksheet
   For Each wks In ActiveWorkbook.Worksheets
      If wks.Name <> "SysInfo" Then wks.Visible = xlSheetVisible
   Next wks
   Set wks = Nothing
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  Clear all rows an columns (left and down from sCell)
' Uses:    n.a.
' Param:   aWks       The worksheet to select/scroll
' History: 02-02-2016 Ronald Bax     First version
'          30-01-2019 Ronald Bax     Update
' --------------------------------------------------------------------
Public Sub ClearSheet(aWks As Worksheet, Optional sCell As String = "A1", Optional sRowCol As String =
 "row")
   Dim bSvAppEvent As Boolean
   Dim oSvActSheet As Worksheet
   Dim bSvProtStat As Boolean
   Dim rng         As Range
   Dim bDone       As Boolean
   If aWks Is Nothing Then Exit Sub ' Ready if aWks is not valid
   bSvAppEvent = Application.EnableEvents
   Application.EnableEvents = False
   Set oSvActSheet = ActiveSheet
   bSvProtStat = EU_XLS_Lib.ActivateAndUnprotect(aWks:=aWks)
   With aWks
      .Activate
      .Range("A1").Activate
      ' Clear the rows and colls of the whole sheet
      sRowCol = LCase(sRowCol)
      If sRowCol <> "row" And sRowCol <> "col" And sRowCol <> "rowcol" Then sRowCol = "row"
      If sRowCol = "row" Or sRowCol = "rowcol" Then
         Set rng = .Range("A1").SpecialCells(xlLastCell)
         Set rng = .Range(sCell & ":" & .Cells(rng.Row, rng.Column).Address)
         If Range(sCell).Row > rng.Row Or Range(sCell).Column > rng.Column Then Set rng = Range(sCell)
         rng.Rows.EntireRow.Delete
      End If
      If sRowCol = "col" Or sRowCol = "rowcol" Then
         Set rng = .Range("A1").SpecialCells(xlLastCell)
         Set rng = .Range(sCell & ":" & .Cells(rng.Row, rng.Column).Address)
         If Range(sCell).Row > rng.Row Or Range(sCell).Column > rng.Column Then Set rng = Range(sCell)
         rng.Columns.EntireColumn.Delete
      End If
   End With
   If bSvProtStat Then Call EU_XLS_Lib.ProtectSheet(aWks:=aWks)
   oSvActSheet.Activate
   Application.EnableEvents = bSvAppEvent
   Set rng = Nothing
   Set oSvActSheet = Nothing
End Sub
' ====================================================================
```

```
' ---------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  Scroll the sheet to "A1" and select the first cell that is 'Editable'
' Uses:    n.a.
' Param:   aWks       The worksheet to select/scroll
' History: 02-02-2016 Ronald Bax     First version
'          30-01-2019 Ronald Bax     Update
' ---------------------------------------------------------------------
Public Sub ScrollTopLeftInView(aWks As Worksheet)
   Dim bSvAppEvent As Boolean
   Dim oSvActSheet As Worksheet
   Dim bSvProtStat As Boolean
   Dim rng         As Range
   Dim rngLast     As Range
   If aWks Is Nothing Then Exit Sub ' Ready if aWks is not valid
   bSvAppEvent = Application.EnableEvents
   Application.EnableEvents = False
   Set oSvActSheet = ActiveSheet
   bSvProtStat = EU_XLS_Lib.ActivateAndUnprotect(aWks:=aWks)
   With aWks
      .Activate
      .Range("A1").Activate ' Scroll to the top-left
      ActiveWindow.ScrollColumn = 1
      ActiveWindow.ScrollRow = 1
      ' Find and select the first cell that is 'Editable'
      Set rngLast = EU_XLS_Lib.LastCell(aWks:=aWks)
      If Not rngLast Is Nothing Then
         For Each rng In Range(.Range("A1"), rngLast)
            If Not rng.Locked Then
               rng.Activate
               Exit For
            End If
         Next rng
      End If
   End With
   If bSvProtStat Then Call EU_XLS_Lib.ProtectSheet(aWks:=aWks)
   oSvActSheet.Activate
   Application.EnableEvents = bSvAppEvent
   Set rngLast = Nothing
   Set rng = Nothing
   Set oSvActSheet = Nothing
End Sub
' =====================================================================

' ---------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  Autofit all columns and rows on the whorksheet
' Uses:    n.a.
' Param:   aWks       The worksheet to AutoFit
' History: 02-02-2016 Ronald Bax     First version
'          30-01-2019 Ronald Bax     Update
' ---------------------------------------------------------------------
Public Sub AutoFitWorksheet(aWks As Worksheet)
   Dim bSvAppEvent As Boolean
   Dim oSvActSheet As Worksheet
   Dim bSvProtStat As Boolean
   Dim rng         As Range
   Dim bDone       As Boolean
   If aWks Is Nothing Then Exit Sub ' Ready if aWks is not valid
   bSvAppEvent = Application.EnableEvents
   Application.EnableEvents = False
   Set oSvActSheet = ActiveSheet
   bSvProtStat = EU_XLS_Lib.ActivateAndUnprotect(aWks:=aWks)
   With aWks ' Autofit the whole sheet
      Set rng = .Range("A1").SpecialCells(xlLastCell)
      Set rng = .Range("A1:" & .Cells(rng.Row, rng.Column).Address)
      rng.Cells.EntireColumn.AutoFit
      rng.Cells.EntireRow.AutoFit
   End With
   If bSvProtStat Then Call EU_XLS_Lib.ProtectSheet(aWks:=aWks)
   oSvActSheet.Activate
   Application.EnableEvents = bSvAppEvent
   Set rng = Nothing
```

```
   Set oSvActSheet = Nothing
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  Set the Codenames of all worksheets in the ActiveWorkbook to "wks" & Name
'          If DebugMode=True, output will go to "Direct" window in VB for Applications
' Uses:    EU_XLS_Lib.ListWksNames
' Param:   n.a.
' History: 02-02-2016 Ronald Bax     First version
'          30-01-2019 Ronald Bax     Update
' --------------------------------------------------------------------
Public Sub SetStdWksCodeNames()
   Dim wks As Worksheet
   For Each wks In ActiveWorkbook.Worksheets
      If wks.CodeName <> "wks" & wks.Name Then
         ActiveWorkbook.VBProject.VBComponents(wks.CodeName).Name = "wks" & wks.Name
      End If
   Next wks
   If GbDebugMode Then Call EU_XLS_Lib.ListWksNames ' If DebugMode then output will go to "Direct" win
dow in VB for Applications
   Set wks = Nothing
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  List the Defined-names in the ActiveWorkbook
' Uses:    n.a.
' Param:   aRng         If not supplied, output will go to "Direct" window in VB for Applications
'          sSepChar     Seperator-char. Default ",". Only used if aRng = Nothing
' History: 02-02-2016 Ronald Bax     First version
'          30-01-2019 Ronald Bax     Update
' --------------------------------------------------------------------
Public Sub ListDefinedNames(Optional aRng As Range = Nothing, Optional sSepChar As String = ",")
   Dim aStr()      As String
   Dim bSvAppEvent As Boolean
   Dim oSvActSheet As Worksheet
   Dim bSvProtStat As Boolean
   Dim wks         As Worksheet
   Dim wksOutput   As Worksheet
   Dim bDone       As Boolean
   Dim nm          As Name
   Dim R           As Long
   Dim C           As Long
   bSvAppEvent = Application.EnableEvents
   Application.EnableEvents = False
   Set oSvActSheet = ActiveSheet
   If Not aRng Is Nothing Then
      Set wksOutput = aRng.Worksheet
      bSvProtStat = EU_XLS_Lib.ActivateAndUnprotect(aWks:=wksOutput)
   End If
   ' Show a line with headers
   If aRng Is Nothing Then
      Debug.Print vbNullString
      Debug.Print "Scope" & sSepChar & "Name" & sSepChar & "RefersTo" & sSepChar & "Value" & sSepChar
& "Visible" & sSepChar & "Comment"
   Else
      With wksOutput
         R = aRng.Row
         C = aRng.Column
         ReDim aStr(5)  ' 0 - 5 = 6 entries
         aStr() = Split("Scope|Name|RefersTo|Value|Visible|Comment", "|", -1, vbBinaryCompare)
         .Range(EU_XLS_Lib.ColNum2Txt(iColNum:=C) & R & ":" & EU_XLS_Lib.ColNum2Txt(iColNum:=C + 5) &
R).Value2 = aStr
         R = R + 1
      End With
   End If
   ' Show the Workbook names
   For Each nm In ActiveWorkbook.Names
      If Not nm.Name Like "_xlfn*" Then
         If nm.Parent.Name = ActiveWorkbook.Name Then
```

```
            If aRng Is Nothing Then
               Debug.Print "Workbook" & sSepChar & nm.Name & sSepChar & vbDQuote & nm.RefersTo & vbDQu
ote & sSepChar & nm.Value & sSepChar & IIf(nm.Visible = 0, 0, 1) & sSepChar & nm.Comment
            Else
               With wksOutput
                  ReDim aStr(5)  ' 0 - 5 = 6 entries
                  aStr() = Split("Workbook" & "|" & nm.Name & "|" & vbDQuote & nm.RefersTo & vbDQuote
& "|" & nm.Value & "|" & IIf(nm.Visible = 0, 0, 1) & "|" & nm.Comment, "|", -1, vbBinaryCompare)
                  .Range(EU_XLS_Lib.ColNum2Txt(iColNum:=C) & R & ":" & EU_XLS_Lib.ColNum2Txt(iColNum:=
C + 5) & R).Value2 = aStr
                  R = R + 1
               End With
            End If
         End If
      End If
   Next nm
   ' Show the Names for individual worksheets
   For Each wks In ActiveWorkbook.Worksheets
      For Each nm In wks.Names
         If Not nm.Name Like "_xlfn*" Then
            If aRng Is Nothing Then
               Debug.Print wks.Name & sSepChar & nm.Name & sSepChar & vbDQuote & nm.RefersTo & vbDQuot
e & sSepChar & nm.Value & sSepChar & IIf(nm.Visible = 0, 0, 1) & sSepChar & nm.Comment
            Else
               With wksOutput
                  ReDim aStr(5)  ' 0 - 5 = 6 entries
                  aStr() = Split("Workbook" & "|" & nm.Name & "|" & vbDQuote & nm.RefersTo & vbDQuote
& "|" & nm.Value & "|" & IIf(nm.Visible = 0, 0, 1) & "|" & nm.Comment, "|", -1, vbBinaryCompare)
                  .Range(EU_XLS_Lib.ColNum2Txt(iColNum:=C) & R & ":" & EU_XLS_Lib.ColNum2Txt(iColNum:=
C + 5) & R).Value2 = aStr
                  R = R + 1
               End With
            End If
         End If
      Next nm
   Next wks
   If Not aRng Is Nothing Then
      If bSvProtStat Then Call EU_XLS_Lib.ProtectSheet(aWks:=wksOutput)
   End If
   oSvActSheet.Activate
   Application.EnableEvents = bSvAppEvent
   Set nm = Nothing
   Set oSvActSheet = Nothing
   Set wksOutput = Nothing
   Set wks = Nothing
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  Clean-up the Defined-names in the ActiveWorkbook
' Uses:    n.a.
' Param:   n.a.
' History: 08-03-2021 Ronald Bax     First version
' --------------------------------------------------------------------
Public Sub CleanupDefinedNames()
   Dim nm As Name
   For Each nm In ActiveWorkbook.Names
      If InStr(1, nm.RefersTo, "#REF!") > 0 Then
         On Error GoTo ITried
         nm.Delete
ITried:  On Error GoTo 0
      End If
   Next
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  List the names of all worksheets in the ActiveWorkbook
' Uses:    n.a.
' Param:   aRng         If not supplied, output will go to "Direct" window in VB for Applications
'          sSepChar     Seperator-char. Default ",". Only used if aRng = Nothing
```

```
' History: 02-02-2016 Ronald Bax     First version
'          30-01-2019 Ronald Bax     Update
' ----------------------------------------------------------------------
Public Sub ListWksNames(Optional aRng As Range = Nothing, Optional sSepChar As String = ",")
   Dim aStr()      As String
   Dim bSvAppEvent As Boolean
   Dim oSvActSheet As Worksheet
   Dim bSvProtStat As Boolean
   Dim wks         As Worksheet
   Dim wksOutput   As Worksheet
   Dim bDone       As Boolean
   Dim R           As Long
   Dim C           As Long
   Dim bProtCont   As Boolean
   bSvAppEvent = Application.EnableEvents
   Application.EnableEvents = False
   Set oSvActSheet = ActiveSheet
   If Not aRng Is Nothing Then
      Set wksOutput = aRng.Worksheet
      bSvProtStat = EU_XLS_Lib.ActivateAndUnprotect(aWks:=wksOutput)
   End If
   ' Show a line with headers
   If aRng Is Nothing Then
      Debug.Print vbNullString
      Debug.Print "Index" & sSepChar & "Name" & sSepChar & "CodeName" & sSepChar & "Visible" & sSepCha
r & "Protected"
   Else
      With wksOutput
         R = aRng.Row
         C = aRng.Column
         ReDim aStr(4)  ' 0 - 4 = 5 entries
         aStr() = Split("Index|Name|CodeName|Visible|Protected", "|", -1, vbBinaryCompare)
         .Range(EU_XLS_Lib.ColNum2Txt(iColNum:=C) & R & ":" & EU_XLS_Lib.ColNum2Txt(iColNum:=C + 4) &
R).Value2 = aStr
         R = R + 1
      End With
   End If
   ' Show the individual lines
   For Each wks In ActiveWorkbook.Worksheets
      bProtCont = IIf(wks.Name <> wksOutput.Name, wks.ProtectContents, bSvProtStat)
      If aRng Is Nothing Then
         Debug.Print wks.Index & sSepChar & wks.Name & sSepChar & wks.CodeName & sSepChar & IIf(wks.Vi
sible = 0, 0, 1) & sSepChar & IIf(bProtCont = 0, 0, 1)
      Else
         With wksOutput
            ReDim aStr(4)  ' 0 - 4 = 5 entries
            aStr() = Split(wks.Index & "|" & wks.Name & "|" & wks.CodeName & "|" & IIf(wks.Visible = 0
, 0, 1) & "|" & IIf(bProtCont = 0, 0, 1), "|", -1, vbBinaryCompare)
            .Range(EU_XLS_Lib.ColNum2Txt(iColNum:=C) & R & ":" & EU_XLS_Lib.ColNum2Txt(iColNum:=C + 4)
 & R).Value2 = aStr
            R = R + 1
         End With
      End If
   Next wks
   If Not aRng Is Nothing Then
      If bSvProtStat Then Call EU_XLS_Lib.ProtectSheet(aWks:=wksOutput)
   End If
   oSvActSheet.Activate
   Application.EnableEvents = bSvAppEvent
   Set oSvActSheet = Nothing
   Set wksOutput = Nothing
   Set wks = Nothing
End Sub
' ======================================================================

' ----------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  Save a range of cells to a TXT (or CSV) file
'          NOTE: The file will be overwritten without any question!!
' Uses:    EU_WIN_Lib.addDirBackslash()
'          EU_WIN_Lib.DirExists()
' Param:   aRng       The range of cells to save
'          sFilePath  The fully qualyfied pathname top save the data to
```

```
'          sSepChar   Seperator-char. Default ",".
' History: 02-02-2016 Ronald Bax     First version
'          30-01-2019 Ronald Bax     Update
' ----------------------------------------------------------------------
Public Function SaveRangeToTXTFile(aRng As Range, sFilePath As String, Optional sSepChar As String = "
,") As Boolean
   Dim sLine As String
   Dim sFile As String
   Dim sDir  As String
   Dim hFile As Long
   Dim R     As Long
   Dim C     As Long
   If aRng Is Nothing Then Exit Function ' Ready if aRng is not valid
   ' Print every row and column within this range to the file
   On Error Resume Next
   sFile = Right(sFilePath, Len(sFilePath) - InStrRev(sFilePath, Application.PathSeparator))
   sDir = EU_WIN_Lib.AddDirBackslash(sDirPath:=Left(sFilePath, Len(sFilePath) - Len(sFile)))
   If EU_WIN_Lib.DirExists(sDirPath:=sDir, bForceCreate:=True) = False Then
      MsgBox "Can't create directory '" & sDir & "'.", vbCritical + vbOKOnly, "Save to file"
      SaveRangeToTXTFile = False
      Exit Function
   End If
   hFile = FreeFile
   Open sFilePath For Output As #hFile
   For R = 1 To aRng.Rows.Count
      For C = 1 To aRng.Columns.Count
          sLine = IIf(C = 1, vbNullString, sLine & sSepChar) & aRng.Cells(R, C)
      Next C
      If Left(sLine, 1) = vbDQuote Then sLine = Mid(sLine, 2, Len(sLine) - 1)
      If Right(sLine, 1) = vbDQuote Then sLine = Left(sLine, Len(sLine) - 1)
      Print #hFile, sLine
   Next R
   Close #hFile
   SaveRangeToTXTFile = (Err.Number = 0)
   On Error GoTo 0
End Function
' ======================================================================

' ----------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  Save an entire Worksheet to a TXT (or CSV) file
'          NOTE: The file will be overwritten without any question!!
' Uses:    n.a.
' Param:   aWks       The worksheet with the data to save
'          sFilePath  The fully qualyfied pathname top save the data to
'          sSepChar   Seperator-char. Default ",".
' History: 02-02-2016 Ronald Bax     First version
'          30-01-2019 Ronald Bax     Update
' ----------------------------------------------------------------------
Public Function SaveSheetToTXTFile(aWks As Worksheet, sFilePath As String, Optional sSepChar As String
 = ",") As Boolean
   Dim bSvAppEvent As Boolean
   Dim oSvActSheet As Worksheet
   Dim bSvProtStat As Boolean
   Dim rng         As Range
   Dim bDone       As Boolean
   If aWks Is Nothing Then Exit Function ' Ready if aWks is not valid
   bSvAppEvent = Application.EnableEvents
   Application.EnableEvents = False
   Set oSvActSheet = ActiveSheet
   bSvProtStat = EU_XLS_Lib.ActivateAndUnprotect(aWks:=aWks)
   With aWks ' Get a range with all cells, and use the SaveRangeToTXTFile function
      Set rng = .Range("A1").SpecialCells(xlLastCell)
      Set rng = .Range("A1:" & .Cells(rng.Row, rng.Column).Address)
      SaveSheetToTXTFile = EU_XLS_Lib.SaveRangeToTXTFile(aRng:=rng, sFilePath:=sFilePath, sSepChar:=sS
epChar)
   End With
   If bSvProtStat Then Call EU_XLS_Lib.ProtectSheet(aWks:=aWks)
   oSvActSheet.Activate
   Application.EnableEvents = bSvAppEvent
   Set rng = Nothing
   Set oSvActSheet = Nothing
End Function
```

```
' ============================================================================

' ----------------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  ...
' Uses:    EU_WIN_Lib.addDirBackslash()
'          EU_WIN_Lib.DirExists()
' Param:   ...
' History: 02-02-2016 Ronald Bax     First version
'          30-01-2019 Ronald Bax     Update
' ----------------------------------------------------------------------------
Public Sub ExportRangeToHTML(sFilePath As String, aRng As Range)
   Dim FSO         As Object
   Dim bSvProtStat As Boolean
   Dim expDir      As String
   Set FSO = CreateObject("Scripting.FilesystemObject")
   If FSO.FolderExists(sFilePath) Then
      MsgBox "'" & sFilePath & "' is a directory.", vbCritical + vbOKOnly, "Export to HTML."
      Set FSO = Nothing
      Exit Sub
   End If
   expDir = EU_WIN_Lib.AddDirBackslash(sDirPath:=FSO.GetParentFolderName(sFilePath))
   If Not EU_WIN_Lib.DirExists(sDirPath:=expDir, bForceCreate:=False) Then
      MsgBox "Directory '" & expDir & "' does not exist.", vbCritical + vbOKOnly, "Export to HTML."
      Set FSO = Nothing
      Exit Sub
   End If
   With Application
      Call EU_XLS_Lib.SetScreenUpdates(bSwitchOn:=False)
      .DisplayAlerts = False
      On Error Resume Next
      Kill sFilePath
      Err.Clear
      On Error GoTo 0
      bSvProtStat = EU_XLS_Lib.ActivateAndUnprotect(aWks:=aRng.Worksheet)
      With aRng.Worksheet
         ThisWorkbook.RefreshAll
         ThisWorkbook.PublishObjects.Add( _
            SourceType:=xlSourceRange, _
            Filename:=sFilePath, _
            Sheet:=aRng.Worksheet.Name, _
            Source:=aRng.Address, _
            HtmlType:=xlHtmlStatic).Publish
      End With
      If bSvProtStat Then Call EU_XLS_Lib.ProtectSheet(aWks:=aRng.Worksheet)
      .DisplayAlerts = True
      Call EU_XLS_Lib.SetScreenUpdates(bSwitchOn:=True)
   End With
   Set FSO = Nothing
End Sub
' ============================================================================

' ----------------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 02-02-2016 Ronald Bax     First version
'          30-01-2019 Ronald Bax     Update
' ----------------------------------------------------------------------------
Public Sub SetHTMLBackgroundColor(sFilePath As String, ByVal iColor As Long)
   Dim hFile         As Long
   Dim sFileContents As String
   Dim aLineArray()  As String
   Dim I             As Long
   hFile = FreeFile
   Open sFilePath For Binary As #hFile
   sFileContents = Input(LOF(hFile), #hFile)
   aLineArray() = Split(sFileContents, vbCrLf)
   Close #hFile
   For I = LBound(aLineArray) To UBound(aLineArray)
      If aLineArray(I) = "<body>" Then
         aLineArray(I) = "<body style=""background:black;"">"
```

```
         Exit For
      End If
   Next I
   Open sFilePath For Output As #hFile
   For I = LBound(aLineArray) To UBound(aLineArray)
      Print #hFile, aLineArray(I)
   Next I
   Close #hFile
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  Create a new sheet if it's not already there
' Uses:    n.a.
' Param:   wksName The name of the sheet to create
'          bEmpty  If True, clear the entire sheet (default = False)
' History: 02-02-2016 Ronald Bax     First version
'          30-01-2019 Ronald Bax     Update
' --------------------------------------------------------------------
Public Function CreateSheetIfNotPresent(sWksName As String, Optional ByVal bEmpty As Boolean = False)
As Worksheet
   Dim aWks    As Worksheet
   Dim wks     As Worksheet
   Dim rngLast As Range
   Dim bAlerts As Boolean
   Set aWks = ActiveSheet
   Set wks = Nothing
   bAlerts = Application.DisplayAlerts
   Application.DisplayAlerts = False
   On Error Resume Next
      Set wks = ActiveWorkbook.Worksheets(sWksName)
      If (Not wks Is Nothing) And (bEmpty = True) Then
         Set rngLast = wks.Range("A1").SpecialCells(xlLastCell)
         rngLast.Rows.EntireRow.Delete
      End If
      Set wks = ActiveWorkbook.Worksheets(sWksName)
      If wks Is Nothing Then
         Worksheets.Add(After:=Worksheets(Worksheets.Count)).Name = sWksName
         Set wks = ActiveWorkbook.Worksheets(sWksName)
      End If
   On Error GoTo 0
   Application.DisplayAlerts = bAlerts
   Set CreateSheetIfNotPresent = wks
   If Not wks Is Nothing Then
      wks.Activate
      wks.Range("A1").Activate
   End If
   aWks.Activate
   Set rngLast = Nothing
   Set wks = Nothing
   Set aWks = Nothing
End Function
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  VLookup a value sSearch in Column sColumn of aWks
'          and return the range in sResColumn with matching values
' Uses:    n.a.
' Param:   wksName     The name of the sheet to create
'          sSearch     The string to search for in column sColumn
'          sColumn     Column where the value should be sSearch
'          sResColumn  Column with the values we want to find
'          iStartRow   Start at 1 by default
' History: 02-02-2016 Ronald Bax     First version
'          30-01-2019 Ronald Bax     Update
'          02-06-2021 Ronald Bax     .Find added LookIn:=xlValues
' --------------------------------------------------------------------
Public Function VLookupBackToo(sSearch As String, aRng As Range, ByVal iColumn As Long, ByVal iResColu
mn As Long) As String
   Dim iStartCol As Long
   Dim iRowFound As Long
```

```
   If aRng Is Nothing Then
      VLookupBackToo = "#Name?"
   Else
      With aRng
         If iColumn > .Columns.Count Or iResColumn > .Columns.Count Then
            VLookupBackToo = "#COL?"
         Else
            iStartCol = .Column + iColumn - 1
            iRowFound = -1
            On Error Resume Next
            iRowFound = .Find(What:=sSearch, After:=Cells(.Row, .Column), LookIn:=xlValues).Row
            On Error GoTo 0
            If iRowFound = -1 Then
               VLookupBackToo = "#N/A"
            Else
               VLookupBackToo = .Worksheet.Cells(iRowFound, .Column + (iResColumn - 1)).Value2
            End If
         End If
      End With
   End If
End Function
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 02-02-2016 Ronald Bax     First version
'          30-01-2019 Ronald Bax     Update
' --------------------------------------------------------------------
Public Sub FormatColumn(aWks As Worksheet, sRngStart As String, sFormat As String)
   Dim rng       As Range
   Dim aRngStart As Range
   Dim aRngEnd   As Range
   If aWks Is Nothing Then Exit Sub ' Ready if aWks is not valid
   With aWks
      Set aRngStart = .Range(sRngStart)
      Set aRngEnd = .Cells(EU_XLS_Lib.LastRow(aWks:=aWks), aRngStart.Column)
      Set rng = .Range(aRngStart, aRngEnd)
      rng.NumberFormat = sFormat
      rng.Value2 = rng.Value2 ' Needs to be 'refreshed'
   End With
   Set rng = Nothing
   Set aRngEnd = Nothing
   Set aRngStart = Nothing
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 07-03-2021 Ronald Bax     First version
' --------------------------------------------------------------------
Public Sub SetStyleIsCurrency(aRng As Range)
   Dim oCell As Range
   For Each oCell In aRng
      If oCell.Value2 <> vbNullString Then oCell.Value = CDec(oCell.Value2)
   Next oCell
   aRng.Style = "Currency"
   Set oCell = Nothing
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 07-03-2021 Ronald Bax     First version
' --------------------------------------------------------------------
```

```
Public Sub SetValidation(aWks As Worksheet, sCol As String, iStartRow As Long, sLookupRange As String,
 Optional iEndRow As Long = 0)
   Dim iLR As Long
   With aWks
      .Activate
      If iEndRow <> 0 And iEndRow >= iStartRow Then
         iLR = iEndRow
      Else
         iLR = EU_XLS_Lib.LastRow(aWks:=aWks)
      End If
      .Range(sCol & iStartRow & ":" & sCol & iLR).Select
      Selection.Activate
      With Selection.Validation
         .Delete
         .Add Type:=xlValidateList, AlertStyle:=xlValidAlertStop, Operator:=xlBetween, Formula1:=sLook
upRange
      End With
   End With
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 02-02-2016 Ronald Bax     First version
'          30-01-2019 Ronald Bax     Update
' --------------------------------------------------------------------
Public Sub FillWithFormula(aWks As Worksheet, sRngStart As String, sFormula As String)
   Dim aRngStart As Range
   Dim aRngEnd   As Range
   If aWks Is Nothing Then Exit Sub ' Ready if aWks is not valid
   With aWks
      Set aRngStart = .Range(sRngStart)
      Set aRngEnd = .Cells(EU_XLS_Lib.LastRow(aWks:=aWks), aRngStart.Column)
      aRngStart.FormulaR1C1 = sFormula
      aRngStart.AutoFill Destination:=.Range(aRngStart, aRngEnd)
   End With
   Set aRngEnd = Nothing
   Set aRngStart = Nothing
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  ...
' Uses:    EU_XLS_Lib.DrawBorders()
' Param:   ...
' History: 30-01-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Public Sub FillRowDownward(aRng As Range)
   Const cMe = "EU_XLS_Lib.FillRowDownward"
   Dim rngDest As Range
   Dim iLR     As Long
   With aRng
      If .Rows.Count <> 1 Then
         MsgBox "Range '" & .Address & "' may only contain one row!", vbCritical + vbOKOnly, cMe
         Exit Sub
      End If
      .Worksheet.Activate
      iLR = EU_XLS_Lib.LastRow(aWks:=.Worksheet)
      Set rngDest = Range(Cells(.Rows(1).Row, .Columns(1).Column), Cells(iLR, .Columns(.Columns.Count)
.Column))
      If .Address <> rngDest.Address Then
         .AutoFill Destination:=rngDest
         Call EU_XLS_Lib.DrawBorders(aRng:=rngDest, bOnOff:=True)
      End If
      .Activate
   End With
   Set rngDest = Nothing
End Sub
' ====================================================================
```

```
' ---------------------------------------------------------------------
' Author:  Ronald Bax          Version: 3.01          Date: 02-06-2021
' Action:  ...
' Uses:    n.a.
' Param:   rng
'          bOnOff
' History: 30-01-2019 Ronald Bax     First version
' ---------------------------------------------------------------------
Public Sub DrawBorders(aRng As Range, ByVal bOnOff As Boolean)
   With aRng
      .Borders(xlDiagonalDown).LineStyle = xlNone
      .Borders(xlDiagonalUp).LineStyle = xlNone
      If bOnOff = False Then
         .Borders(xlEdgeLeft).LineStyle = xlNone
         .Borders(xlEdgeTop).LineStyle = xlNone
         .Borders(xlEdgeBottom).LineStyle = xlNone
         .Borders(xlEdgeRight).LineStyle = xlNone
         .Borders(xlInsideVertical).LineStyle = xlNone
         .Borders(xlInsideHorizontal).LineStyle = xlNone
      Else
         With .Borders(xlEdgeLeft)
            .LineStyle = xlContinuous
            .ColorIndex = 0
            .TintAndShade = 0
            .Weight = xlThin
         End With
         With .Borders(xlEdgeTop)
            .LineStyle = xlContinuous
            .ColorIndex = 0
            .TintAndShade = 0
            .Weight = xlThin
         End With
         With .Borders(xlEdgeBottom)
            .LineStyle = xlContinuous
            .ColorIndex = 0
            .TintAndShade = 0
            .Weight = xlThin
         End With
         With .Borders(xlEdgeRight)
            .LineStyle = xlContinuous
            .ColorIndex = 0
            .TintAndShade = 0
            .Weight = xlThin
         End With
         With .Borders(xlInsideVertical)
            .LineStyle = xlContinuous
            .ColorIndex = 0
            .TintAndShade = 0
            .Weight = xlThin
         End With
         With .Borders(xlInsideHorizontal)
            .LineStyle = xlContinuous
            .ColorIndex = 0
            .TintAndShade = 0
            .Weight = xlThin
         End With
         Cells(.Row, .Column).Activate
      End If
   End With
End Sub
' =====================================================================

' ---------------------------------------------------------------------
' Author:  Ronald Bax          Version: 3.01          Date: 02-06-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 30-01-2019 Ronald Bax     First version
' ---------------------------------------------------------------------
Public Sub PivotTableFormat(aPivTab As PivotTable, Optional ByVal bUpdateToo As Boolean = False)
   Dim pif As PivotField
   With aPivTab
```

```
      .RowAxisLayout xlTabularRow
      .RepeatAllLabels xlRepeatLabels
      .ColumnGrand = False
      .RowGrand = False
      For Each pif In .PivotFields
         pif.Subtotals(1) = False
      Next pif
      If bUpdateToo Then .Update
   End With
   Set pif = Nothing
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 30-01-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Public Sub PivotFiltersRemove(aPivTab As PivotTable, Optional ByVal bUpdateToo As Boolean = False)
   Dim pif As PivotField
   Dim pit As PivotItem
   With aPivTab
      ' Set to manual update for speed
      .ManualUpdate = True
      ' Ensure there are no old ghost items in the item lists
      .PivotCache.MissingItemsLimit = xlMissingItemsNone
      .PivotCache.Refresh
      For Each pif In .PivotFields
         With pif
            .ClearValueFilters
            If .Orientation <> 0 Then
               If .Orientation = xlPageField Then
                  .CurrentPage = "(All)"
               Else
                  For Each pit In .PivotItems
                     If pit.RecordCount > 0 Then pit.Visible = True
                  Next pit
               End If
            End If
         End With
      Next pif
      If bUpdateToo Then .Update
      .ManualUpdate = False
   End With
   Set pit = Nothing
   Set pif = Nothing
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 30-01-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Public Sub PivotFiltersNotBlankEmpty(aPivTab As PivotTable, sPivFieldName As String)
   Const cMe = "EU_XLS_Lib.PivotFiltersNotBlankEmpty"
   Dim pif    As PivotField
   Dim pit    As PivotItem
   Dim bFound As Boolean
   bFound = False
   With aPivTab
      For Each pif In .PivotFields
         If pif.Name = sPivFieldName Then
            With pif
               For Each pit In .PivotItems
                  If pit.Name = vbNullString Or pit.Name = "(blank)" Then
                     If pit.RecordCount > 0 Then
                        pit.Visible = False
                     End If
```

```
                End If
             Next pit
          End With
          bFound = True
          Exit For
       End If
    Next pif
 End With
 If bFound = False Then
    MsgBox ".PivotFields(""" & sPivFieldName & """) not found.", vbCritical + vbOKOnly, cMe
 End If
 Set pit = Nothing
 Set pif = Nothing
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  ...
' Uses:    EU_XLS_Lib.PivotFiltersRemove()
'          EU_XLS_Lib.LastRow()
'          EU_XLS_Lib.ClearSheet()
' Param:   ...
' History: 30-01-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Public Sub PivotClearBelow(aPivTab As PivotTable)
 Dim iLR As Long
 With aPivTab
    Call EU_XLS_Lib.PivotFiltersRemove(aPivTab:=aPivTab, bUpdateToo:=False)
    iLR = EU_XLS_Lib.LastRow(aWks:=aPivTab.Parent)
    If .TableRange1.Cells(.TableRange1.Cells.Count).Row < iLR Then
       iLR = .TableRange1.Cells(.TableRange1.Cells.Count).Row
    End If
    Call EU_XLS_Lib.ClearSheet(aWks:=.Parent, sCell:="A" & iLR + 1, sRowCol:="row")
 End With
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 30-01-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Public Sub AutoFiltersRemove(aWks As Worksheet)
 If aWks Is Nothing Then Exit Sub ' Ready if aWks is not valid
 With aWks
    If .FilterMode Then .ShowAllData
    If .AutoFilterMode Then .AutoFilterMode = False
 End With
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 30-01-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Public Sub AutoFiltersAdd(aWks As Worksheet, Optional ByVal iTopRows As Integer = 1)
 Dim iLC As Integer
 If aWks Is Nothing Then Exit Sub ' Ready if aWks is not valid
 iLC = EU_XLS_Lib.LastCol(aWks:=aWks)
 With aWks
    .Range("A1:" & .Cells(iTopRows, iLC).Address).AutoFilter
    .Rows(iTopRows).EntireRow.AutoFit
    .Rows(iTopRows).RowHeight = .Rows(iTopRows).RowHeight * 2
    .Rows(iTopRows).VerticalAlignment = xlTop
 End With
End Sub
' ====================================================================
```

```
' ---------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 30-01-2019 Ronald Bax     First version
' ---------------------------------------------------------------------
Public Sub SortSheetWithOneHeaderRow(aWks As Worksheet, sColumns As String, Optional iNumHeaderRows As
 Long = 1)
   Dim wks   As Worksheet
   Dim rng   As Range
   Dim sCols As String
   Dim sCol  As String
   If aWks Is Nothing Then Exit Sub ' Ready if aWks is not valid

   Set wks = ActiveSheet
   sCols = LTrim(UCase(sColumns))
   If Right(sCols, 1) <> "," Then sCols = sCols + ","
   Set rng = EU_XLS_Lib.LastCell(aWks:=aWks)
   With aWks
      Set rng = Range(.Range("A" & iNumHeaderRows), rng)
      .Sort.SortFields.Clear
      While InStr(1, sCols, ",", vbTextCompare) > 0
         sCol = Left(sCols, InStr(1, sCols, ",", vbTextCompare) - 1)
         sCols = Mid(sCols, InStr(1, sCols, ",", vbTextCompare) + 1)
         On Error Resume Next
         .Sort.SortFields.Add2 key:=Range(sCol & "2:" & sCol & rng.Row), SortOn:=xlSortOnValues, Order
:=xlAscending, DataOption:=xlSortNormal
         On Error GoTo 0
      Wend
      With .Sort
         .SetRange rng
         .Header = xlYes
         .MatchCase = False
         .Orientation = xlTopToBottom
         .SortMethod = xlPinYin
         .Apply
      End With
      .Activate
      .Range("A" & iNumHeaderRows + 1).Activate
   End With
   wks.Activate
   Set rng = Nothing
   Set wks = Nothing
End Sub
' =====================================================================

' ---------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 30-01-2019 Ronald Bax     First version
' ---------------------------------------------------------------------
Public Sub HeaderFooterClearWks(aWks As Worksheet)
   If aWks Is Nothing Then Exit Sub ' Ready if aWks is not valid
   Call EU_XLS_Lib.SetScreenUpdates(bSwitchOn:=False)
   Application.PrintCommunication = True
   aWks.Activate
   With aWks.PageSetup
      .LeftHeader = vbNullString
      .CenterHeader = vbNullString
      .RightHeader = vbNullString
      .LeftFooter = vbNullString
      .CenterFooter = vbNullString
      .RightFooter = vbNullString
   End With
   Call EU_XLS_Lib.SetScreenUpdates(bSwitchOn:=True)
End Sub
' =====================================================================

' ---------------------------------------------------------------------
```

```
' Author:  Ronald Bax          Version: 3.01          Date: 02-06-2021
' Action:  ...
' Uses:    EU_XLS_Lib.HeaderFooterClearWks()
' Param:   ...
' History: 30-01-2019 Ronald Bax     First version
' ---------------------------------------------------------------------
Public Sub HeaderFooterClearAll()
   Dim wks As Worksheet
   For Each wks In ThisWorkbook.Worksheets
      Call EU_XLS_Lib.HeaderFooterClearWks(aWks:=wks)
   Next wks
   ThisWorkbook.Worksheets(1).Activate
   Set wks = Nothing
End Sub
' =====================================================================

' ---------------------------------------------------------------------
' Author:  Ronald Bax          Version: 3.01          Date: 02-06-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 30-01-2019 Ronald Bax     First version
' ---------------------------------------------------------------------
Public Sub HeaderFooterAddStandardWks(aWks As Worksheet)
   If aWks Is Nothing Then Exit Sub ' Ready if aWks is not valid
   Call EU_XLS_Lib.SetScreenUpdates(bSwitchOn:=False)
   Application.PrintCommunication = True
   aWks.Activate
   With aWks.PageSetup
      .LeftHeader = "Domino's Pizza"
      .CenterHeader = vbNullString
      .RightHeader = "Printed &D &T"
      .LeftFooter = "&Z&F"
      .CenterFooter = vbNullString
      .RightFooter = "&P / &N"
   End With
   ThisWorkbook.Worksheets(1).Activate
   Call EU_XLS_Lib.SetScreenUpdates(bSwitchOn:=True)
End Sub
' =====================================================================

' ---------------------------------------------------------------------
' Author:  Ronald Bax          Version: 3.01          Date: 02-06-2021
' Action:  ...
' Uses:    EU_XLS_Lib.HeaderFooterAddStandardWks()
' Param:   ...
' History: 30-01-2019 Ronald Bax     First version
' ---------------------------------------------------------------------
Public Sub HeaderFooterAddStandardAll()
   Dim wks As Worksheet
   For Each wks In ThisWorkbook.Worksheets
      Call EU_XLS_Lib.HeaderFooterAddStandardWks(aWks:=wks)
   Next wks
   ThisWorkbook.Worksheets(1).Activate
   Set wks = Nothing
End Sub
' =====================================================================

' ---------------------------------------------------------------------
' Author:  Ronald Bax          Version: 3.01          Date: 02-06-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 30-01-2019 Ronald Bax     First version
' ---------------------------------------------------------------------
Public Sub PageInLandscapeWks(aWks As Worksheet)
   If aWks Is Nothing Then Exit Sub ' Ready if aWks is not valid
   Call EU_XLS_Lib.SetScreenUpdates(bSwitchOn:=False)
   Application.PrintCommunication = True
   With aWks
      .Activate
      With .PageSetup
         .Orientation = xlLandscape
```

```
         .LeftMargin = Application.InchesToPoints(0.25)
         .RightMargin = Application.InchesToPoints(0.25)
         .TopMargin = Application.InchesToPoints(0.75)
         .BottomMargin = Application.InchesToPoints(0.75)
         .HeaderMargin = Application.InchesToPoints(0.3)
         .FooterMargin = Application.InchesToPoints(0.3)
      End With
   End With
   ThisWorkbook.Worksheets(1).Activate
   Call EU_XLS_Lib.SetScreenUpdates(bSwitchOn:=True)
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 30-01-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Public Sub PageInLandscapeAll()
   Dim wks As Worksheet
   For Each wks In ThisWorkbook.Worksheets
      Call EU_XLS_Lib.PageInLandscapeWks(aWks:=wks)
   Next wks
   Set wks = Nothing
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 30-01-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Public Sub PageInPortraitWks(aWks As Worksheet)
   If aWks Is Nothing Then Exit Sub ' Ready if aWks is not valid
   Call EU_XLS_Lib.SetScreenUpdates(bSwitchOn:=False)
   Application.PrintCommunication = True
   With aWks
      .Activate
      With .PageSetup
         .Orientation = xlPortrait
         .LeftMargin = Application.InchesToPoints(0.25)
         .RightMargin = Application.InchesToPoints(0.25)
         .TopMargin = Application.InchesToPoints(0.75)
         .BottomMargin = Application.InchesToPoints(0.75)
         .HeaderMargin = Application.InchesToPoints(0.3)
         .FooterMargin = Application.InchesToPoints(0.3)
      End With
   End With
   ThisWorkbook.Worksheets(1).Activate
   Call EU_XLS_Lib.SetScreenUpdates(bSwitchOn:=True)
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 3.01           Date: 02-06-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 30-01-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Public Sub PageInPortraitAll()
   Dim wks As Worksheet
   For Each wks In ThisWorkbook.Worksheets
      Call EU_XLS_Lib.PageInPortraitWks(aWks:=wks)
   Next wks
   Set wks = Nothing
End Sub
' ====================================================================
```

```
' ---------------------------------------------------------------------
' Author:  Ronald Bax          Version: 3.01          Date: 02-06-2021
' Action:  ...
' Uses:    n.a.
' Param:   ...
' History: 30-01-2019 Ronald Bax     First version
' ---------------------------------------------------------------------
Public Sub GetUniqueValues(aRng As Range, aTargetCell As Range)
   Const cMe = "EU_XLS_Lib.GetUniqueValues"
   Dim bSvScrnUpd  As Boolean
   Dim bSvProtStat As Boolean
   Dim rngSort     As Range
   Dim rngTarget   As Range
   Dim rngSel      As Range
   Dim rngLoop     As Range
   Dim sSaveHdr    As String
   Dim iLR         As Long
   If aRng Is Nothing Or aTargetCell Is Nothing Then Exit Sub ' Ready if aRng is not valid
   If aTargetCell.Worksheet.CodeName <> aRng.Worksheet.CodeName Then
      MsgBox "Param 'aRng' and 'aTargetCell' must be in the same worksheet.", vbCritical + vbOKOnly, c
Me
      Exit Sub
   End If
   If aRng.Cells(1, 1).Value2 = vbNullString Then
      MsgBox "The first cell of Param 'aRng' must be the header and cannot be empty.", vbCritical + vb
OKOnly, cMe
      Exit Sub
   End If
   If aRng.Columns.Count <> 1 Then
      MsgBox "Param 'aRng' must be a single column.", vbCritical + vbOKOnly, cMe
      Exit Sub
   End If
   If aTargetCell.Rows.Count <> 1 Or aTargetCell.Columns.Count <> 1 Then
      MsgBox "Param 'aTargetCell' must be a single cell.", vbCritical + vbOKOnly, cMe
      Exit Sub
   End If
   If aTargetCell.Column = aRng.Column Then
      MsgBox "Param 'aRng' and 'aTargetCell' must be in a differet column.", vbCritical + vbOKOnly, cM
e
      Exit Sub
   End If
   Set rngSort = Range(Cells(aTargetCell.Row + 1, aTargetCell.Column), Cells(aTargetCell.Row + aRng.Ro
ws.Count - 1, aTargetCell.Column))
   For Each rngLoop In rngSort.Cells
      If rngLoop.Value2 <> vbNullString Then
         On Error Resume Next
         rngLoop.Cells.Activate
         On Error GoTo 0
         MsgBox "The target column must be empty.", vbCritical + vbOKOnly, cMe
         Exit Sub
      End If
   Next rngLoop

   bSvScrnUpd = EU_XLS_Lib.SetScreenUpdates(bSwitchOn:=False)
   bSvProtStat = EU_XLS_Lib.ActivateAndUnprotect(aWks:=rngSort.Worksheet)

   Set rngTarget = Range(Cells(rngSort.Row, rngSort.Column).Address)
   sSaveHdr = aTargetCell.Value2
   aTargetCell.ClearContents
   aRng.AdvancedFilter Action:=xlFilterCopy, CopyToRange:=aTargetCell, Unique:=True
   If sSaveHdr <> vbNullString Then
      aTargetCell.Value2 = sSaveHdr
   Else
      aTargetCell.Value2 = aRng.Cells(1, 1).Value2 & " (unique)"
   End If
   rngSort.Sort Key1:=rngTarget, Order1:=xlAscending, Header:=xlNo

   iLR = rngSort.Row + rngSort.Count - 1
   Set rngSel = rngSort.Worksheet.Range(aTargetCell.Address, Cells(iLR, rngSort.Column).Address)
   Call EU_XLS_Lib.DrawBorders(aRng:=rngSel, bOnOff:=False)

   iLR = rngSort.Row + rngSort.Count - 1
   For Each rngLoop In rngSort.Worksheet.Range(aTargetCell.Address, Cells(iLR, rngSort.Column).Address
```

```
EU_XLS_Lib - 31

)
      If rngLoop.Value2 = vbNullString Then
         iLR = iLR - 1
      End If
   Next rngLoop
   Set rngSel = rngSort.Worksheet.Range(aTargetCell.Address, Cells(iLR, rngSort.Column).Address)
   Call EU_XLS_Lib.DrawBorders(aRng:=rngSel, bOnOff:=True)
   Call EU_XLS_Lib.DrawBorders(aRng:=aRng, bOnOff:=True)

   If bSvProtStat Then Call EU_XLS_Lib.ProtectSheet(aWks:=rngSort.Worksheet)
   Call EU_XLS_Lib.SetScreenUpdates(bSwitchOn:=bSvScrnUpd)

   Set rngLoop = Nothing
   Set rngSel = Nothing
   Set rngTarget = Nothing
   Set rngSort = Nothing
End Sub
' ====================================================================


' ====================================================================
' ====================================================================
```

```
' Last Cleaned (Export): 20-03-2023 11:34
' ====================================================================
'                             QAGeneric.bas
' ====================================================================
' Author:  Ronald Bax           Version: 4.00          Date: 16-02-2023
' Action:  GENERIC module for various workbooks that work in all stores
' Uses:    EU_DB_Lib, EU_Dom_Lib, EU_XLS_Lib, EU_WIN_Lib
' History: 23-01-2019 Ronald Bax     First version
' ====================================================================
' Private Sub InitProc()
' Private Sub DoneProc()
' Private Sub PingAllStores()
' Private Sub ReplaceRef(sCorrectName As String)
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Public  Sub SetGlobals()
' Public  Sub ReLoadStores()
' Public  Sub PingInAllStores()
' Public  Sub ReloadStandards()
' Public  Sub RunInAllStores()
' Public  Sub LoadStoreFile()
' Public  Sub LoadStandFile()
' - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
' Public  Sub AddSQLResultsToResult(sSQL As String)
' ====================================================================
Option Explicit

' ====================================================================
'                               GLOBALS
' ====================================================================
Public GsMyDisk      As String ' = Left(ThisWorkbook.Path,2)
Public GsMyDir       As String ' = ThisWorkbook.Path
Public GsMyFile      As String ' = Replace(ThisWorkbook.FullName, ThisWorkbook.Path, vbNullstring)
Public GsBaseDir     As String ' = Left(GsMyDir, InStrRev(GsMyDir, Application.PathSeparator))
Public GsUserName    As String ' = Application.UserName
Public GsDescription As String ' = Replace(ThisWorkbook.FullName, ThisWorkbook.Path, vbNullstring)

' ====================================================================
'                           PRIVATE FUNCTIONS
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 4.00          Date: 16-02-2023
' Action:  Disable the screen-updating etc.
' Uses:    EU_XLS_Lib.ShowProcessing
' Param:   n.a.
' History: 23-01-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Private Sub InitProc()
   Call EU_XLS_Lib.ShowProcessing(bSwitchOn:=False)
   wksStores.Activate
   wksStores.Range("F9").Value2 = Format(Time, "hh:nn:ss")
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 4.00          Date: 16-02-2023
' Action:  re-enable the screen-updating etc.
' Uses:    EU_XLS_Lib.ShowProcessing
' Param:   n.a.
' History: 23-01-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Private Sub DoneProc()
   Dim TmStart As Date
   Dim TmEnd   As Date
   TmEnd = Time
   With wksStores
      TmStart = .Range("F9").Value2
      .Range("F10").Value2 = Format(TmEnd, "hh:nn:ss")
      .Range("G9").Value2 = Format((TmEnd - TmStart), "hh:nn:ss")
      If .Range("H8").Value2 = 0 Then
         .Range("H10").Value2 = 0
      Else
         .Range("H10").Value2 = Format((TmEnd - TmStart) / .Range("H8").Value2, "hh:nn:ss") & " per st
```

```
ore."
      End If
      Call EU_XLS_Lib.ShowProcessing(bSwitchOn:=True)
      .Activate
      .Range("E3").Select
   End With
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 4.00           Date: 16-02-2023
' Action:  Ping all stores and show if they're connected
' Uses:    EU_DB_Lib.getPingResult
'          EU_Dom_Lib.getStoreToDNSName
'          EU_XLS_Lib.LastRow
'          EU_WIN_Lib.Pause
' Param:   n.a.
' History: 23-01-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Private Sub PingAllStores()
   Dim sHost As String
   Dim sOK   As String
   Dim I     As Long
   Dim iLR   As Long
   With wksStores
      iLR = EU_XLS_Lib.LastRow(aWks:=wksStores)
      For I = 2 To iLR
         .Range("D" & I).Value2 = vbNullString
         .Range("E" & I).Value2 = vbNullString
         If .Range("A" & I).Value2 = "V" Then
            sHost = EU_DOM_Lib.GetStoreToDNSName(sStoreNum:=.Range("B" & I).Value2)
            sOK = EU_DB_Lib.getPingResult(sHost:=sHost)
            If sOK <> "Connected" Then
               Call EU_WIN_Lib.Pause(iSeconds:=1)
               sOK = EU_DB_Lib.getPingResult(sHost:=sHost)
            End If
            .Range("D" & I).Value2 = sOK
            If sOK <> "Connected" Then .Range("A" & I).Value2 = vbNullString
            .Range("H8").Formula = "=COUNTIF(D:D,""Connected"")"
         End If
      Next I
   End With
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 4.00           Date: 16-02-2023
' Action:  Relace the filename in the references in wksStandard
' Uses:    EU_XLS_Lib.LastRow
'          EU_XLS_Lib.LastCol
'          EU_XLS_Lib.ColNum2Txt
' Param:   sCorrectName -- Correct name of the worksheet to reference
'          sColumns     -- columns in which to replace the reference
' History: 23-01-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Private Sub ReplaceRef(sCorrectName As String)
   Dim sSource As String
   Dim sReplac As String
   Dim iLR     As Long
   Dim iLC     As Long
   Dim iRow    As Long
   Dim iCol    As Long
   Dim sCol    As String
   Dim I       As Long
   Dim F       As String
   Dim S       As String
   ' Find the worksheet-name that is there already
   iLR = EU_XLS_Lib.LastRow(aWks:=wksStandard)
   iLC = EU_XLS_Lib.LastCol(aWks:=wksStandard)
   F = vbNullString
   For iRow = 1 To iLR
      S = wksStandard.Range("B" & iRow).Formula
      If InStr(1, S, "]", vbTextCompare) > 0 Then
```

```
         I = InStr(1, S, "[", vbTextCompare)
         F = Mid(S, I, Len(S) - I + 1)
         F = Left(F, InStr(1, F, "!", vbTextCompare))
         S = Mid(S, 1, I - 1)
         Exit For
      End If
   Next iRow
   ' Determine the new worksheet-name
   If F <> vbNullString Then
      sSource = S & F
      S = sCorrectName
      I = InStrRev(S, "\", , vbTextCompare)
      F = Mid(S, I + 1, Len(S) - I)
      sReplac = "='" & Left(S, InStrRev(S, "\", , vbTextCompare)) & "[" & F
      I = InStr(1, sSource, "]", vbTextCompare)
      S = Mid(sSource, I, Len(sSource) - 1)
      sReplac = sReplac & S
   End If
   ' Replace the old name with the new name, in all relevant columns
   sCol = vbNullString
   For iCol = 2 To iLC
      If InStr(1, wksStandard.Cells(iLR, iCol).Formula, sReplac) = 0 Then
         sCol = EU_XLS_Lib.ColNum2Txt(iColNum:=iCol - 1)
         Exit For
      End If
   Next iCol
   If sSource <> sReplac Then
      wksStandard.Columns("B:" & sCol).Replace What:=sSource, Replacement:=sReplac, SearchOrder:=xlByC
olumns, MatchCase:=False
   End If
End Sub
' ============================================================================


' ============================================================================
'                              PUBLIC FUNCTIONS
' ============================================================================

' ----------------------------------------------------------------------------
' Author:  Ronald Bax           Version: 4.00          Date: 16-02-2023
' Action:  Sets global variables
' Uses:    n.a.
' Param:   n.a.
' History: 16-02-2023 Ronald Bax     First version
' ----------------------------------------------------------------------------
Public Sub SetGlobals(Optional sDescription As String = vbNullString)
   Dim WS As Worksheet

   GsMyDisk = Left(ThisWorkbook.Path, 2)
   GsMyDir = ThisWorkbook.Path
   GsMyFile = Replace(ThisWorkbook.FullName, ThisWorkbook.Path & Application.PathSeparator, vbNullStri
ng)
   GsBaseDir = Left(GsMyDir, InStrRev(GsMyDir, Application.PathSeparator))
   GsUserName = Application.UserName
   GsDescription = IIf(sDescription <> vbNullString, sDescription, Replace(Replace(GsMyFile, ".xlsm",
vbNullString), ".xlsb", vbNullString))

   Call EU_XLS_Lib.ShowProcessing(bSwitchOn:=False)
   For Each WS In Worksheets
      With WS
         If .Name = "Summary" Then
            .Unprotect
            .Range("A2").Value2 = "Summary " & GsDescription
         End If
         If .Name = "Stores" Then
            .Unprotect
            If .Range("G2").Value2 <> vbNullString Then
               If Left(.Range("G2").Value2, 2) <> GsMyDisk And Mid(.Range("G2").Value2, 2, 2) = ":\" T
hen
                  .Range("G2").Value2 = GsMyDisk & Mid(.Range("G2").Value2, 3)
               End If
            End If
            If .Range("G4").Value2 <> vbNullString Then
```

```
               If Left(.Range("G4").Value2, 2) <> GsMyDisk And Mid(.Range("G4").Value2, 2, 2) = ":\" T
hen
                  .Range("G4").Value2 = GsMyDisk & Mid(.Range("G4").Value2, 3)
               End If
            End If
         End If
      End With
   Next WS
   Call EU_XLS_Lib.ProtectAllWorksheets(bProtect:=True)
   Call EU_XLS_Lib.ShowProcessing(bSwitchOn:=True)
   Set WS = Nothing
End Sub

' ---------------------------------------------------------------------
' Author:  Ronald Bax           Version: 4.00           Date: 16-02-2023
' Action:  Reload a list of all stores from .\Data\StoreXX.dat
' Uses:    EU_XLS_Lib.AutoFiltersRemove
'          EU_XLS_Lib.ClearSheet
'          EU_XLS_Lib.LastRow
'          EU_XLS_Lib.DrawBorders
'          EU_XLS_Lib.AutoFitWorksheet
'          EU_WIN_Lib.FileExists
' Param:   n.a.
' History: 23-01-2019 Ronald Bax     First version
' ---------------------------------------------------------------------
Public Sub ReLoadStores()
   Dim adoStream As Object
   Dim aStrArr   As Variant
   Dim sFilePath As String
   Dim sLine     As String
   Dim sNum      As String
   Dim sNam      As String
   Dim I         As Long
   Dim iRow      As Long
   Dim iLR       As Long

   Call QAGeneric.InitProc
   ' Clear the sheets Stores and Result
   Call EU_XLS_Lib.AutoFiltersRemove(aWks:=wksResult)
   Call EU_XLS_Lib.ClearSheet(aWks:=wksResult, sCell:="A3", sRowCol:="row")
   Call EU_XLS_Lib.ClearSheet(aWks:=wksStores, sCell:="A11", sRowCol:="row")

   ActiveWorkbook.Save

   sFilePath = wksStores.Range("G2").Value2 ' Check if the file even exists...
   If Not EU_WIN_Lib.FileExists(sFilePath:=sFilePath) Then
      MsgBox "File not found '" & sFilePath & "'.", vbOKOnly + vbCritical, "wksStores.ReLoadStores"
      Exit Sub
   End If
   With wksStores ' Read all stores, and format the cells
      iLR = EU_XLS_Lib.LastRow(aWks:=wksStores)
      Call EU_XLS_Lib.DrawBorders(aRng:=.Range("A2:D" & iLR), bOnOff:=False)
      .Range("A2:E" & iLR).ClearContents

      Set adoStream = CreateObject("ADODB.Stream")
      adoStream.Charset = "iso-8859-1"
      adoStream.Open
      adoStream.LoadFromFile sFilePath
      aStrArr = Split(adoStream.ReadText, vbCrLf) ' Split entire file into array - lines delimited by
CRLF
      Set adoStream = Nothing
      iRow = 2
      For I = LBound(aStrArr) To UBound(aStrArr)
         sLine = aStrArr(I)
         If Len(sLine) > 6 Then
            sNum = IIf(Left(sLine, 1) = "*", Left(sLine, 6), Left(sLine, 5))  ' Select all, except whe
n column B starts with "*"
            sNam = Mid(sLine, Len(sNum) + 1)
            .Range("A" & iRow).Value2 = IIf(sNum <> vbNullString And Len(sNum) = 5, "V", vbNullString)
            .Range("B" & iRow).Value2 = sNum
            .Range("C" & iRow).Value2 = sNam
            iRow = iRow + 1
         End If
```

```
      Next I

      iLR = EU_XLS_Lib.LastRow(aWks:=wksStores)
      Call EU_XLS_Lib.DrawBorders(aRng:=.Range("A2:D" & iLR), bOnOff:=True)
      ' Create some formulae and some formatting
      .Range("H8").Formula = "=COUNTIF(A:A,""V"")"
      .Range("G3").Value2 = Date + Time
      .Range("E1").FormulaR1C1 = "=COUNTA(C[-3])-1"
      Call EU_XLS_Lib.AutoFitWorksheet(aWks:=wksStores)
      .Columns("E").ColumnWidth = 7
      .Columns("F").ColumnWidth = 20
      .Columns("G").ColumnWidth = 40
      .Columns("H").ColumnWidth = 20
      .Columns("I").ColumnWidth = 40
      With .Range("F2,F3,F4,F5,H5,H3").Font
         .ThemeColor = xlThemeColorLight2
         .TintAndShade = 0.4
      End With
   End With
   wksStores.Activate
   Call EU_XLS_Lib.AddActCircleToBtn(aWks:=wksStores, aBtn:=wksStores.Shapes("btnPing"), bDelOld:=True
)
   Call QAGeneric.DoneProc
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 4.00           Date: 16-02-2023
' Action:  Ping all stores, and make sure Result is empty (better performance when saved now)
' Uses:    EU_XLS_Lib.AutoFiltersRemove
'          EU_XLS_Lib.ClearSheet
' Param:   n.a.
' History: 23-01-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Public Sub PingInAllStores()
   Call QAGeneric.InitProc
   Call EU_XLS_Lib.AutoFiltersRemove(aWks:=wksResult)
   Call EU_XLS_Lib.ClearSheet(aWks:=wksResult, sCell:="A3", sRowCol:="row")
   Call QAGeneric.PingAllStores
   With wksStores
      .Activate
      .Columns("D:D").EntireColumn.AutoFit
      On Error Resume Next
      Call EU_XLS_Lib.AddActCircleToBtn(aWks:=wksStores, aBtn:=.Shapes("btnStand"), bDelOld:=True)
      If Err <> 0 Then
         Call EU_XLS_Lib.AddActCircleToBtn(aWks:=wksStores, aBtn:=.Shapes("btnRunInStores"), bDelOld:=
True)
      End If
   End With
   Call QAGeneric.DoneProc
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 4.00           Date: 16-02-2023
' Action:  Reload the standard-values into wksStandard
' Uses:    EU_XLS_Lib.AutoFiltersRemove
'          EU_XLS_Lib.ClearSheet
'          EU_XLS_Lib.FillRowDownward(.Range("B" & iRow & ":G" & iRow))
'          EU_WIN_Lib.FileExists
' Param:   n.a.
' History: 23-01-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Public Sub ReloadStandards()
   Dim iRow      As Long
   Dim sFilePath As String
   ' Clear Result (better performance when saved now), and check if the file exists
   Call QAGeneric.InitProc
   Call EU_XLS_Lib.AutoFiltersRemove(aWks:=wksResult)
   Call EU_XLS_Lib.ClearSheet(aWks:=wksResult, sCell:="A3", sRowCol:="row")
   sFilePath = wksStores.Range("G4").Value2
   If Not EU_WIN_Lib.FileExists(sFilePath:=sFilePath) Then
      MsgBox "File not found '" & sFilePath & "'.", vbOKOnly + vbCritical, "wksStores.ReloadStandards"
```

```
      Exit Sub
   End If
   Call QAGeneric.ReplaceRef(sCorrectName:=wksStores.Range("G4").Value2)
   With wksStandard  ' find first row with label 'Standard' and Replace all references in wksStandard
with the correct pathname to the file
      .Activate
      .Columns("A:A").Select
      On Error Resume Next
      Selection.Find(What:="Standard", After:=ActiveCell, LookIn:=xlFormulas, LookAt:=xlPart, _
                     SearchOrder:=xlByRows, SearchDirection:=xlNext, MatchCase:=False, SearchFormat:=F
alse).Activate
      On Error GoTo 0
      iRow = ActiveCell.Row
      If iRow > 1 Then Call EU_XLS_Lib.FillRowDownward(aRng:=.Range("B" & iRow & ":G" & iRow))
      .Range("A2").Select
   End With
   With wksStores
      .Activate
      .Range("G5").Value2 = Date + Time
      Call EU_XLS_Lib.AddActCircleToBtn(aWks:=wksStores, aBtn:=.Shapes("btnRunInStores"), bDelOld:=Tru
e)
   End With
   Call QAGeneric.DoneProc
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 4.00           Date: 16-02-2023
' Action:  Call wksResult.GetAndProcessData to do specific processing
' Uses:    EU_XLS_Lib.AutoFiltersRemove
'          EU_XLS_Lib.ClearSheet
' Param:   n.a.
' History: 23-01-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Public Sub RunInAllStores()
   Call QAGeneric.InitProc
   ' Clear Result (better performance when saved now), and Save the sheet before processing
   Call EU_XLS_Lib.AutoFiltersRemove(aWks:=wksResult)
   Call EU_XLS_Lib.ClearSheet(aWks:=wksResult, sCell:="A3", sRowCol:="row")

   ActiveWorkbook.Save

   Call wksResult.GetAndProcessData
   With wksStores
      .Range("H7").Value2 = Date + Time
      .Range("H8").Formula = "=COUNTIF(E:E,""V"")"
      .Activate
      On Error Resume Next
      Call EU_XLS_Lib.AddActCircleToBtn(aWks:=wksStores, aBtn:=.Shapes("btnReloadStores"), bDelOld:=Tr
ue)
      If Err <> 0 Then
         Call EU_XLS_Lib.AddActCircleToBtn(aWks:=wksStores, aBtn:=.Shapes("btnPing"), bDelOld:=True)
      End If
      Call QAGeneric.DoneProc
      Call EU_WIN_Lib.Log(sMsg:="Executed in " & .Range("H8").Text & " stores in " & .Range("G9").Text
 & " (" & .Range("H10").Value2 & ")")
   End With

   With wksStores
      .Activate
      .Unprotect
      If .Range("$E:$E").FormatConditions.Count = 0 Then
         .Range("$E:$E").Select
         With Selection
            .FormatConditions.Add Type:=xlExpression, Formula1:="=E1=""X"""
            With .FormatConditions(.FormatConditions.Count)
               .SetFirstPriority
               With .Font
                  .Color = -16776961
                  .TintAndShade = 0
               End With
               With .Interior
                  .PatternColorIndex = xlAutomatic
```

```
                .Color = 10092543
                .TintAndShade = 0
             End With
          .StopIfTrue = False
          End With
       End With
    End If
    .Range("E1").Select
    Call EU_XLS_Lib.ProtectSheet(aWks:=wksStores)
 End With

 Call wksResult.ActivateResultPage
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 4.00           Date: 16-02-2023
' Action:  Determine the location where the storefile is stored
' Uses:    n.a.
' Param:   n.a.
' History: 23-01-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Public Sub LoadStoreFile()
 wksStores.Unprotect
 Call EU_WIN_Lib.PickAFilePathInCell(aCell:=wksStores.Range("G2"))
 Call EU_XLS_Lib.ProtectSheet(aWks:=wksStores)
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 4.00           Date: 16-02-2023
' Action:  Determine the location where the file with Standards is stored
' Uses:    n.a.
' Param:   n.a.
' History: 23-01-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Public Sub LoadStandFile()
 Call EU_WIN_Lib.PickAFilePathInCell(aCell:=wksStores.Range("G4"))
End Sub
' ====================================================================

' --------------------------------------------------------------------
' Author:  Ronald Bax           Version: 4.00           Date: 16-02-2023
' Action:  AddSQLResultsToResult
' Uses:    EU_XLS_Lib.LastRow
'          EU_Dom_Lib.getStoreToDNSName
'          EU_DB_Lib.SQLToWorksheet
' Param:   n.a.
' History: 23-01-2019 Ronald Bax     First version
' --------------------------------------------------------------------
Public Sub AddSQLResultsToResult(sSQL As String)
 Dim sStoreIP As String
 Dim iRow     As Long
 Dim iLR      As Long
 Dim iStLR    As Long
 Dim bRes     As Boolean
 iStLR = EU_XLS_Lib.LastRow(aWks:=wksStores)
 iLR = EU_XLS_Lib.LastRow(aWks:=wksResult)
 If iLR < 2 Then iLR = 2
 With wksStores
    For iRow = 2 To iStLR
       If .Range("A" & iRow).Value2 = "V" And .Range("B" & iRow).Value2 <> vbNullString Then
          .Range("E" & iRow).Value2 = vbNullString
          sStoreIP = EU_DOM_Lib.GetStoreToDNSName(sStoreNum:=.Range("B" & iRow).Text)
          On Error Resume Next
          bRes = EU_DB_Lib.SQLToWorksheet(aWks:=wksResult, aRng:=wksResult.Range("A" & iLR), sStoreI
P:=sStoreIP, sSQL:=sSQL, bHeaders:=False, bShowMsg:=False)
          On Error GoTo 0
          .Range("E" & iRow).Value2 = IIf(bRes, "V", "X")
          iLR = EU_XLS_Lib.LastRow(aWks:=wksResult) + 1
       End If
    Next iRow
 End With
```

```
QAGeneric - 8

End Sub
' ============================================================================

' ----------------------------------------------------------------------------
' Author:  Ronald Bax           Version: 4.00           Date: 16-02-2023
' Action:  Autorun if env-var says so...
' Uses:    n.a.
' Param:   n.a.
' History: 25-07-2022 Ronald Bax     First version
' ----------------------------------------------------------------------------
Public Sub TryAUTORUN()
   Dim aWks    As Worksheet
   If Environ("DPEU_AUTORUN") = "1" Then
      QAGeneric.ReLoadStores
      QAGeneric.PingInAllStores
      On Error Resume Next
         For Each aWks In ActiveWorkbook.Worksheets
            If aWks.CodeName = "wksStandard" Then
               QAGeneric.ReloadStandards
               Exit For
            End If
         Next aWks
      On Error GoTo 0
      QAGeneric.RunInAllStores
      If Environ("DPEU_AUTOCLOSE") = "1" Then
         ActiveWorkbook.Save
         Application.Quit
      End If
   End If
   Set aWks = Nothing
End Sub
' ============================================================================


' ============================================================================
' ============================================================================
```